

学校代码 : 10286
分类号 : TP393
密 级 : 公开
U D C : 004.9
学 号 : 215200



东南大学

硕士学位论文

基于 DPDK 的高速自适应网络测量方法

研究生姓名: 张家康

导师姓名: 程光 教授

申请学位类别 工学硕士 学位授予单位 东南大学

一级学科名称 网络空间安全 论文答辩日期 2024 年 5 月 26 日

二级学科名称 网络空间安全 学位授予日期 2024 年 6 月 19 日

答辩委员会主席 刘光杰 教授 评 阅 人 YS086240187

YS086240188

2024 年 5 月 26 日

東南大學

硕士学位论文

基于 DPDK 的高速自适应网络测量方法

专业名称： 网络空间安全

研究生姓名： 张家康

导师姓名： 程光 教授

本文的部分工作受到国家自然科学基金面上项目(62172093)与国家自然科学基金联合基金项目(U22B2025)的支持和帮助，在此表示感谢。

HIGH-SPEED ADAPTIVE NETWORK MEASUREMENT METHOD BASED ON DPDK

A Thesis Submitted to

Southeast University

For the Academic Degree of Master of

Engineering

BY

Zhang Jiakang

Supervised by

Prof. Cheng Guang

School of Cyber Science and Engineering

Southeast University

May 26, 2024

摘 要

当前,我国具备万兆网络服务能力的 10G 无源光网络端口数已达 2302 万个,随着网络流量规模和传输速率的日益增长,在通用计算平台上对网络流量多维特征进行细粒度测量面临着前所未有的挑战。在高速网络环境下,高效的数据包处理是网络测量的基础,DPDK 是一个开源的高性能网络数据处理框架,通过采用轮询模式驱动显著提升了数据包处理的效率,然而轮询机制也不可避免地导致了 CPU 的持续满负载运行以及功耗增加的问题。同时,DPDK 还提供了基于 Cuckoo 哈希的高性能哈希库,通过使用多个不同的哈希函数为每个元素提供多个备选存储位置,保证了即使在最坏情况下也能提供常数级的查询时间复杂度,因此被广泛应用于网络测量领域。然而,现有基于 Cuckoo 哈希的网络测量方法仍存在着多次哈希运算复杂度高、多个备选桶查询开销大以及高负载条件下可能出现的循环冲突问题。针对上述问题,本文提出一种基于 DPDK 的高速自适应网络测量方法,平衡 DPDK 轮询收包过程中的功耗与性能,优化 Cuckoo 哈希的效率,在通用计算平台上实现最高 14.88Mpps 数据包速率下网络流量多维特征的细粒度测量。具体而言,本文主要包括以下研究内容:

(1) 针对 DPDK 轮询收包过程中难以在性能和功耗之间实现良好平衡的问题,本文提出了一种基于自适应包间休眠的间歇式轮询方法。本方法通过对 DPDK 轮询收包过程进行建模并分析当前数据包速率下的休眠时长期望,同时基于一维卡尔曼滤波对数据包速率进行实时在线预测,最后根据 RX 接收队列的期望负载长度与实际负载长度之间的差距反馈,使用 PID 控制器动态调整每次空轮询后的休眠时长。实验结果表明,在最高 14.88Mpps 的数据包速率下,与原生 DPDK 相比,本方法能将 CPU 利用率降低 52.3%,并且保证零数据包丢失。

(2) 针对网络测量中哈希运算开销大与缓存利用率低的问题,本文提出了一种基于改进桶式 Cuckoo 哈希的高速网络测量方法。本方法将哈希计算任务卸载至网卡,并对网卡生成的对称 RSS 哈希值进行扩展,有效降低了 CPU 的哈希计算开销,同时使用桶式 Cuckoo 哈希作为网络测量键值对哈希表的索引,并基于计数布隆过滤器加速查询流程,最后基于惰性时间衰减成本算法优化 Cuckoo 哈希的流逐出策略,减少循环冲突。同时,本文还基于 AVX 与 SSE 指令集进行加速,实验结果表明,本文所提出的方法相比于传统 Cuckoo 哈希在高速网络流量细粒度测量上更具优势,在 14.88Mpps 的发包速率下,实现了 2 核 16.18%丢包率与 3 核零丢包率的细粒度网络测量。

(3) 基于上述研究,本文构建了一个基于 DPDK 的高速自适应网络测量原型系统,系统集成了资源配置模块、收包转发模块、网络测量模块与流持久化模块。本文介绍了系统整体架构设计与各模块功能,对系统各个模块进行了展示,并对系统极限性能进行了测试验证,证明了本文方法的有效性,表明在通用计算平台上,本系统能够有效地对网络流量的多维特征进行细粒度测量,具有较强的实用性。

关键词: 网络测量, DPDK, 轮询模式驱动, Cuckoo 哈希, 计数布隆过滤器

Abstract

Currently, China has reached a capability of 23.02 million 10G Passive Optical Network ports, offering ten-gigabit network services. With the continuous growth in network traffic scale and transmission rates, unprecedented challenges have arisen in performing fine-grained measurement of multi-dimensional network traffic features on general-purpose computing platforms. In the context of high-speed networks, efficient packet processing is essential. DPDK (Data Plane Development Kit) is an open-source, high-performance network data processing framework that has significantly enhanced packet processing efficiency through its use of a polling mode. However, the polling mechanism inevitably leads to continuous full CPU load operation and increased power consumption. In addition, DPDK provides a high-efficiency hash library based on Cuckoo Hashing, which, by utilizing multiple distinct hash functions for each element, offers several alternative storage positions, ensuring constant-time search complexity even under the worst scenarios, making it widely adopted for network measurement. Yet, existing measurement methods based on Cuckoo Hashing still face issues with the high complexity of multiple hash operations, the large overhead caused by querying multiple backup buckets, and potential cyclic conflicts under heavy load conditions. To address these concerns, this thesis introduces a high-speed adaptive network measurement method based on DPDK, which balances power consumption with performance during the packet reception process, optimizes the efficiency of Cuckoo Hashing, and implements fine-grained measurement of the network's multi-dimensional traffic features at up to 14.88 Mpps on general-purpose computing platforms. Specifically, this thesis presents the following research contents:

(1) Focusing on the challenge of maintaining a balance between performance and power consumption during the DPDK packet reception process, this study proposes an intermittent polling method based on adaptive packet intersleep. This approach involves modeling the DPDK packet receiving process and analyzing the expected sleep duration under the current packet rate. Furthermore, it incorporates one-dimensional Kalman filtering for real-time online prediction of packet rate, and finally, leverages a PID controller to dynamically adjust sleep duration after each non-receiving polling cycle based on the feedback from the expected versus actual load length of the RX queue. Experimental results demonstrate that, at the peak packet rate of 14.88 Mpps, this method can reduce CPU utilization by 52.3% compared to the native DPDK while ensuring zero packet loss.

(2) Addressing the issue of high hash computation overhead and low cache utilization in network measurement, the thesis proposes a high-speed network measurement method based on an improved bucketized Cuckoo Hashing. This method offloads hashing computations to the network card and extends the symmetric RSS hash values generated by the hardware, effectively reducing CPU hash computation overhead. It employs bucketized Cuckoo Hashing as an index for the key-value pair hash table in network measurement and accelerates the query process using a counting Bloom filter.

The final step involves optimizing the flow eviction strategy in Cuckoo Hashing with a lazy time-decay cost algorithm to reduce cyclic collisions. Additionally, this work utilizes AVX and SSE instruction sets to hasten the process. Experimental findings prove that the proposed method surpasses traditional Cuckoo Hashing in fine-grained measurement of high-speed network traffic, demonstrating fine-grained measurements with a 16.18% packet loss rate on two cores and zero packet loss on three cores at a 14.88 Mpps transmission rate.

(3) Building upon the aforementioned research, this article develops a prototype high-speed adaptive network measurement system based on DPDK, including modules for resource configuration, packet reception and forwarding, network measurement, and flow persistence. The thesis outlines the comprehensive architecture and functionalities of each module, illustrates the system's components, and confirms the system's maximum performance through tests, which proved the effectiveness of the proposed method. It indicates that the system can effectively conduct fine-grained measurements of the multi-dimensional characteristics of network traffic on general-purpose computing platforms, proving to be highly practical.

Key words: Network Measurement, DPDK, Poll Mode Driver, Cuckoo Hashing, Counting Bloom Filter

目 录

第一章 绪论.....	1
1.1 研究背景和意义.....	1
1.2 研究现状.....	3
1.2.1 高速数据包处理技术.....	3
1.2.2 数据流技术.....	5
1.3 研究内容.....	8
1.4 论文组织结构.....	9
第二章 相关背景技术	11
2.1 DPDK.....	11
2.1.1 UIO 技术.....	11
2.1.2 轮询模式驱动技术.....	11
2.1.3 无锁队列技术.....	12
2.1.4 RSS 技术.....	12
2.2 数据流技术.....	13
2.2.1 Bitmap	13
2.2.2 Bloom Fliter	13
2.2.3 Sketch.....	13
2.2.4 Cuckoo 哈希	14
2.3 本章小结.....	16
第三章 基于自适应包间休眠的间歇式轮询方法	17
3.1 问题分析.....	17
3.2 DPDK 轮询批处理分析.....	18
3.3 基于一维卡尔曼滤波的数据包速率预测方法	21
3.4 基于 PID 控制器的自适应轮询调度方法	25
3.4.1 Linux 与 DPDK 的休眠函数	25
3.4.2 基于 PID 控制器的自适应轮询调度方法	26
3.5 实验与分析.....	29
3.5.1 实验与环境.....	29
3.5.2 数据集构建.....	30
3.5.3 对比实验设置.....	30
3.5.4 评价指标.....	30
3.5.5 实验结果.....	31
3.6 本章小结.....	34
第四章 基于改进桶式 Cuckoo 哈希的高速网络测量方法.....	35

4.1 问题分析.....	35
4.2 基于对称 RSS 的软硬协同哈希方法	36
4.2.1 Toeplitz 哈希算法.....	36
4.2.2 对称 RSS 算法.....	38
4.2.3 基于对称 RSS 算法的软硬协同哈希方法	39
4.3 基于改进桶式 Cuckoo 哈希的高速网络测量方法	41
4.3.1 桶单元的内存布局.....	41
4.3.2 基于混合哈希编码的流指纹生成方法	42
4.3.3 基于计数布隆过滤器的查找优化方法	43
4.3.4 基于惰性时间衰减成本的流逐出方法	45
4.4 基于改进桶式 Cuckoo 哈希的高速网络测量方法操作	47
4.4.1 插入操作	48
4.4.2 查询更新操作.....	49
4.4.3 删除操作	50
4.5 实验与分析.....	51
4.5.1 实验与环境.....	51
4.5.2 数据集构建.....	52
4.5.3 对比实验设置.....	52
4.5.4 评价指标.....	52
4.5.5 实验结果.....	53
4.6 本章小结.....	58
第五章 系统设计与实现	59
5.1 系统需求与分析.....	59
5.2 系统整体架构设计.....	59
5.3 系统模块设计.....	61
5.3.1 资源配置模块.....	61
5.3.2 收包转发模块.....	61
5.3.3 网络测量模块.....	62
5.3.4 流持久化模块.....	64
5.4 系统运行结果.....	65
5.5 系统性能测试.....	66
5.6 本章小结.....	67
第六章 总结与展望	69
6.1 工作总结.....	69
6.2 问题和展望.....	70
参考文献.....	71

插图目录

图 1-1 Linux 内核协议栈与 DPDK 的数据包处理流程	2
图 1-2 高速数据包处理研究分类	3
图 1-3 研究内容框架图	8
图 2-1 DPDK 核心组件图	11
图 2-2 DPDK 无锁队列结构图	12
图 2-3 RSS 示意图	12
图 2-4 Bitmap 示意图	13
图 2-5 布隆过滤器示意图	13
图 2-6 Count-Min Sketch 示意图	14
图 2-7 Cuckoo 哈希示意图	14
图 2-8 Cuckoo 哈希插入示意图	15
图 3-1 基于自适应包间休眠的间歇式轮询方法架构图	17
图 3-2 DPDK 轮询收包与空轮询对比图	19
图 3-3 DPDK 轮询与间歇式轮询对比图	20
图 3-4 基于一维卡尔曼滤波的数据包速率预测方法图	22
图 3-5 Linux 与 DPDK 的休眠函数关系图	25
图 3-6 基于 PID 控制器的自适应轮询调度方法图	28
图 3-7 基于 Pktgen 的收发包实验架构图	30
图 3-8 六种休眠函数延迟图	31
图 3-9 数据包速率预测的 MASE 对比图	32
图 3-10 基于自适应包间休眠的间歇式轮询方法性能对比图	32
图 3-11 NUMA 性能对比图	33
图 3-12 不同队列负载因子 α 下的性能对比图	33
图 4-1 基于改进的桶式布谷鸟哈希的高速网络测量方法架构图	35
图 4-2 Toeplitz 与 RSS 图	38
图 4-3 基于 Cuckoo 哈希表的键值对哈希表索引图	42
图 4-4 DPDK Cuckoo 与改进的 Cuckoo 的内存布局图	42
图 4-5 基于计数布隆过滤器的查找优化	44
图 4-6 Cuckoo 哈希的循环冲突	46
图 4-7 Run-To-Completion 与 Pipeline 模型架构图	48
图 4-8 十种哈希函数计算开销对比图	53
图 4-9 哈希函数卡方检验图	54
图 4-10 基于对称 RSS 的软硬协同哈希方法的分布散点图	54
图 4-11 基于循环与基于 AVX 的元素查找对比图	55
图 4-12 基于循环与基于 SSE 的最小查找对比图	55
图 4-13 不同负载因子下的插入与查询周期对比图	56
图 4-14 基于 Run-To-Completion 与 Pipeline 的丢包率图	56
图 4-15 CERNET 与 MAWI 流累计百分比图	57
图 4-16 CERNET 与 MAWI 流量数据集重放的丢包率图	57
图 5-1 基于 DPDK 的高速自适应网络测量原型系统架构图	60
图 5-2 资源配置模块流程图	61
图 5-3 收包转发模块流程图	62
图 5-4 网络测量模块流程图	63
图 5-5 查找更新流程图	63
图 5-6 插入流程图	64

图 5-7 基于 DPDK 的高速自适应网络测量原型系统主界面图	65
图 5-8 网络测量界面图	66
图 5-9 测量结果展示图	66
图 5-10 Pktgen 发包界面图	67
图 5-11 基于 DPDK 的高速自适应网络测量原型系统性能测试图.....	67

表格目录

表 3-1 基于一维卡尔曼滤波的数据包速率预测方法变量定义	22
表 3-2 Linux 与 DPDK 的休眠函数.....	26
表 3-3 基于 DPDK 的收发包系统的软硬件配置	29
表 3-4 DPDK 程序队列长度与批处理大小等参数配置	30
表 3-5 六种休眠函数的 CPU 利用率.....	31
表 4-1 RSS 的哈希报文字段	38
表 4-2 网络测量的流特征	48
表 4-3 基于改进的桶式 Cuckoo 哈希的高速网络测量方法的收发包软硬件配置.....	51
表 4-4 基于改进的桶式 Cuckoo 哈希的高速网络测量方法的参数配置	52
表 4-5 MAWI 与 CERNET 流量数据集.....	52

第一章 绪论

1.1 研究背景和意义

随着我国互联网技术的高速发展,数据中心与国家骨干网络的流量规模不断增长,据第53次《中国互联网络发展状况统计报告》^[1]显示,我国互联网宽带接入端口数量已达11.36亿,具备万兆网络服务能力的10G PON(Passive Optical Network)端口数已达2302万个,1000Mbps及以上的固定互联网宽带接入用户达1.63亿户。随着网络带宽的增长与互联网应用的丰富,网络安全问题也日益严峻,垃圾邮件、病毒蠕虫、DDoS攻击等各类网络安全事件频发,网络信息安全问题受到了广泛重视。

网络流量测量是网络安全管理的基础。流量数据承载了丰富的网络状况和用户行为信息,这些信息对于维护网络运行状态、改善网络运维管理、分析用户行为、挖掘网络漏洞等具有重要意义。通过对网络流量进行测量和分析,可以优化网络配置、及时发现可能的网络攻击、预测网络安全态势,实现对网络行为的有效监管,保障网络空间安全。为了对网络流量的多维特征进行深入的细粒度测量分析,需要足够大的存储空间来容纳海量数据,而基于通用计算平台的网络测量有着灵活性高、成本低与存储空间大的优势,可以有效支持对网络流的多维特征进行细粒度测量分析。然而,目前数据中心与主干网中已广泛使用10Gbps网络链路来连接交换机和路由器,随着网络流量规模和传输速率的不断提高,在通用计算平台上对网络流量多维特征进行细粒度测量面临着巨大的挑战。

数据包处理是网络测量的基础,然而,传统的软件数据包处理通常基于操作系统的内核网络协议栈,在高速网络场景下,内核网络协议栈的处理机制会产生大量的软硬件中断、多次内存拷贝与系统调用等,消耗大量的系统资源,导致处理器无法及时处理网络数据流,从而引起网卡数据堆积,造成服务器网卡在收包时产生大量丢包,难以适用于高速网络测量,如图1-1(a)所示。

为了提升通用计算平台的数据包处理能力,一些高速网络I/O框架被提出,例如Netmap^[2]、DPDK(Data Plane Development Kit)^[3]与PF_RING^[4]等,其中DPDK是由Intel等提出的一套开源框架,包含一套高性能数据包处理函数库与驱动程序集,被广泛应用于高性能数据包处理中。DPDK基于用户态I/O技术,将网卡的内存区域映射到用户空间,当数据包到达网卡时,数据包被分配到映射在用户空间内的内存区域,因此DPDK绕过了内核网络协议栈,可以直接与网卡进行交互并接收数据包,如图1-1(b)所示。同时,DPDK还实现了一个轮询模式驱动(Poll Mode Driver)程序,通过轮询网卡来检索并处理数据包,大大提升了数据包处理的效率。此外,DPDK还利用了CPU亲和性、大页内存与无锁队列等技术,为通用计算平台上的高速网络测量提供了基础。然而,由于DPDK采用轮询模式取代中断,这导致无论网络流量实际负载如何,CPU的使用率都始终保持在100%,繁忙的轮询会消耗大量的CPU周期,导致能源的浪费。

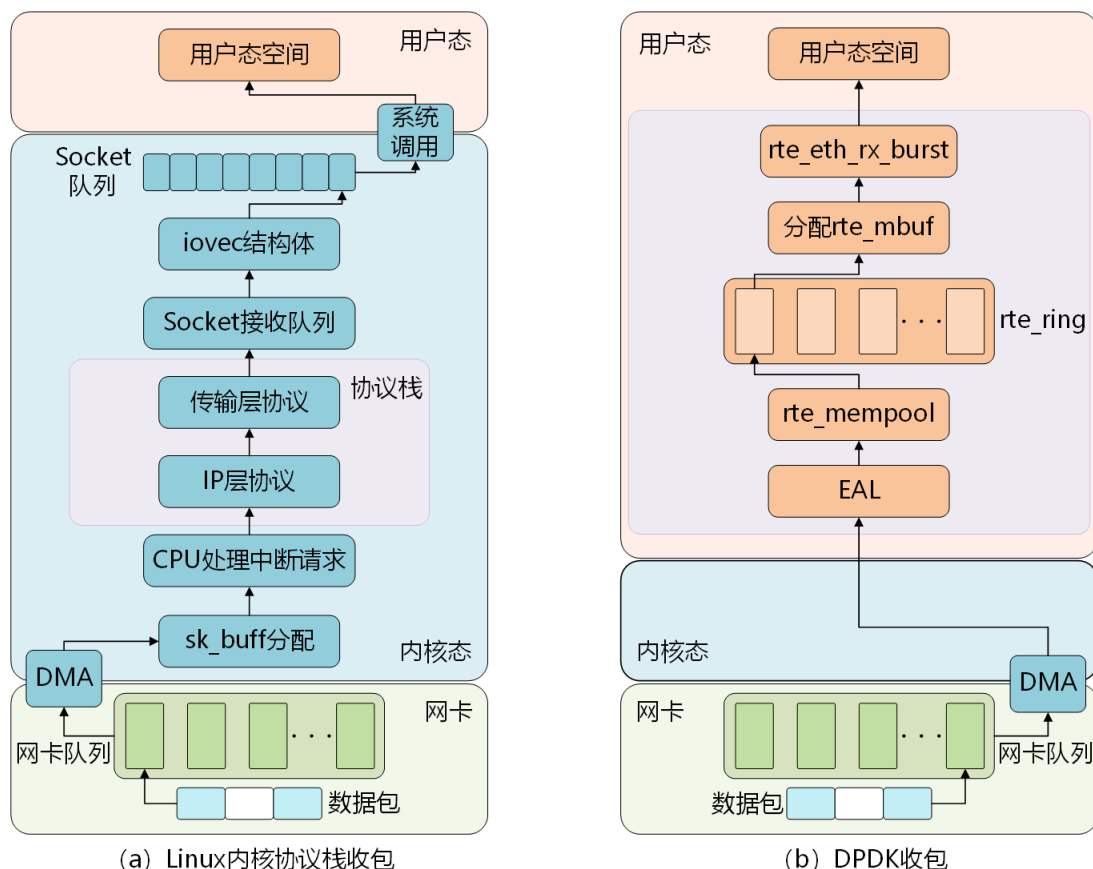


图 1-1 Linux 内核协议栈与 DPDK 的数据包处理流程

同时，在 10Gbps 的网络环境中，数据包速率最高可达 14.88Mpps，为了有效处理大量的流信息，基于哈希的数据结构因其常数时间复杂度的寻址能力，被广泛用于网络测量领域，常用的数据结构有 Sketch 与 Cuckoo 哈希等。Sketch 通过多组哈希运算，降低了哈希冲突的概率，然而，Sketch 是为了优化内存效率而设计的结构，在基于通用计算平台的网络测量中，Sketch 多次的哈希运算反而成为了性能瓶颈。Cuckoo 哈希为每个元素提供多个备选桶，元素可以选择某个为空的备选桶进行插入，当所有备选桶都已被占用时，该元素可以选择备选桶中的某个元素并将其踢出，而被踢出的元素将继续执行插入操作，直到所有元素都能存储在 Cuckoo 哈希表中。但是 Cuckoo 哈希也存在着多次哈希运算复杂度高、多个备选桶查询开销大以及高负载时插入操作的复杂度骤升的问题。

目前，在通用计算平台上，面向高速网络流量的细粒度测量，提取网络流的多维特征信息，还主要面临以下问题：

(1) 基于 DPDK 的高速数据包处理难以在性能和功耗之间实现良好平衡。在传统的基于内核网络协议栈的数据包处理过程中，当数据包到达时，网卡向 CPU 发起中断，随后内核接收数据包并处理，在此过程中还会伴随着系统调用，需要频繁地在用户态与内核态之间进行切换，Marathe 等^[5]的研究表明，这种上下文切换会导致超过 5 倍的 TLB 未命中率以及缓存失效所带来的访问延迟。因此，在 DPDK 中，为了提高数据包处理效率，DPDK 采用了轮询模式驱动，通过自旋锁使 CPU 轮询网卡队列，再结合批处理技术，极大地提升了数据包处

理效率。然而繁忙的轮询也带来了高功耗问题，即使当前的流量规模很小，CPU 也仍然保持满负载运行，Shah 等^[6]指出在 DPDK 的程序中，大部分功耗来自于轮询模式驱动。

(2) 数据流的多维特征处理速度难以匹配高速网络流量。由于片上内存空间有限，难以存储数百万条流的多维特征信息，因此需要依赖容量更大的片外内存。片外内存通常由 DRAM 组成，虽然容量远超片上内存，但其访问速度远慢于片上内存。通过哈希表来对片外内存建立索引，以此来支持数据快速查询等操作，然而多个数据项的哈希冲突会增加操作复杂度。Cuckoo 哈希是一种常被用于缓解哈希冲突的数据结构，但是也存在着多次哈希运算复杂度高、多个备选桶查询开销大以及高负载时可能产生循环冲突的问题。

综上所述，网络流量的高速增长与大规模化为细粒度测量带来了显著挑战，本文旨在研究基于 DPDK 的高速自适应网络测量方法，平衡轮询收包过程中的 CPU 功耗与性能，在通用计算平台上实现 10Gbps 的高速细粒度网络测量，对网络流的多维特征进行统计处理。

1.2 研究现状

1.2.1 高速数据包处理技术

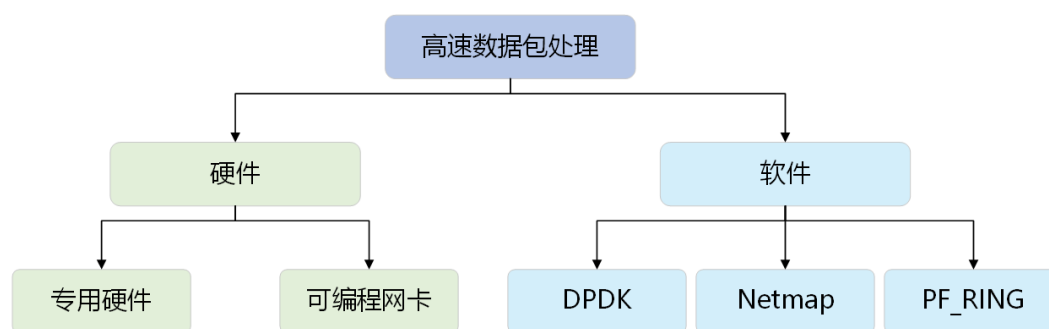


图 1-2 高速数据包处理研究分类

在高速数据包处理技术上，主要可以分为两类^[7]，一类是基于硬件对报文处理进行加速，另一类则是基于软件优化，如零拷贝、内核旁路、批处理与轮询等技术来加速数据包处理。

在基于硬件的报文处理中，又可以分为专用硬件与可编程网卡两种。在专用硬件中，通过将网络功能卸载至定制化的专用硬件设备来加速报文处理，例如将某些功能从内核和通用 CPU 上卸载到专用硬件上。然而，因为这种方法依赖于专用硬件和封闭系统，所以存在着系统功能模块相对单一、扩展性差和硬件成本高等问题。在可编程网卡中，主要是基于 FPGA (Field-Programmable Gate Array) 实现，通过定制数据通路获得更高的流水线并行性，以加速数据包的处理^[8]。Plakalovic 等^[9]提出了一种可扩展的基于 FPGA 的高速以太网流量发生器，能够充分利用 40Gbps 链路，实现在数据包级别上对流量特征的操作。Lin 等^[10]提出 SuperNIC 新型智能网卡平台，利用动态映射的虚拟任务链和公平共享机制，在 FPGA 上实现了高效的网络计算任务卸载，确保了高带宽和低延迟的性能。然而，FPGA 仍然存在着成本高、资源受限和开发门槛高的问题。

另一类方法是基于软件优化，尽管在性能上略逊于硬件解决方案，但在系统成本、灵活性和可编程性方面均有着显著优势。近年来，如 Netmap、PF_RING 和 DPDK 这样的高速数

据包 I/O 框架取得了快速进展,其中 DPDK 是被广泛使用的一个框架。DPDK 通过内核旁路、批处理以及轮询模式驱动等技术,为通用计算平台提供了处理高速网络数据包的能力,因此涌现出了多种基于 DPDK 处理超高速流量的方法。

Emmerich 等^[11]提出 FlowScope,该系统在一个内存环形缓冲区中连续捕获和存储数据包,如果发生指定的触发事件,可以将数据包的过滤子集转储到磁盘中。令等^[12]基于 DPDK 并使用 RSS(Receiver-Side Scaling)技术,设计并实现了一个可扩展的数据包采集系统,支持多核并行化的数据包采集与模块化的上层处理流程。Zhang 等提出 FlowMon-DPDK^[13]与 FloWatcher-DPDK^[14],在数据包级别和流级别都提供了可调整的细粒度统计,能够使用少量的资源提供每条流的精确统计。Zhou 等^[15]提出一个加速软件网络测量的通用框架 Agg-Evict,通过将相同流的数据包聚合并批量处理,减少了后续测量方案中的计算量与内存访问次数,有效降低了每个数据包的平均处理延迟。Ren 等^[16]提出一个高性能的报文处理框架 PacketUser,通过构建 DPDK 封装器作为底层数据包 I/O 引擎来加速数据传输,将每秒报文处理能力提升到了标准 Linux 平台的 4 倍以上。Bhattacharjee 等^[17]基于 DPDK 提出了一个优化的可扩展的报文处理流水线,并设计了一种基于 NUMA 的报文处理架构 NAPA-FP。Ramani 等^[18]设计并实现了一个基于 DPDK 的数据包转发实验,以 AoI (Age-of-Information) 为性能指标,定量地评估了读-写锁与无锁同步的性能。Han 等^[19]将 DPDK 与 FPGA 相结合,设计并实现了一个网络流量全包捕获与测量系统 FPC-NM,该系统具有数据包接收、纳秒时间戳、负载均衡、数据包预处理、应用层协议分析、数据包存储、日志管理等多种功能,并使用 LZ4 压缩算法,可以提供高达 10Gbps 的压缩和存储性能。Huang 等^[20]为 FPGA 设备开发一个 DPDK 驱动程序,实现了对以太网流量以 4ns 精度的时间戳在线速率分析,大大提高了安全事件回溯分析的准确性。Zhao 等^[21]提出了一个基于 DPDK 的新型 DDoS 防御系统,将受害者的流量以机器学习模型形式编码到数据平面分组调度算法中,使受害者流量能以线速优先转发。Zhang 等^[22]提出一种新的数据中心传输方式 Anole,在接近瓶颈的交换机处重新调整数据包优先级,并结合网络遥测和无私调度策略,让下一跳是瓶颈交换机的流量让路于其他流量,最后基于 DPDK 实现了 Anole 原型,相比于 Aeolus^[23],它可以将所有流的平均 FCT (Flow Completion Time) 缩短 61.6%。Sun 等^[24]提出一种网络遥测架构 OmniMon,为大型数据中心实现了资源高效且全面精确的流级遥测,并基于 DPDK 与 P4 平台构建了 OmniMon 原型系统,实验表明 OmniMon 可以有效降低 33%以上的内存使用与 66%以上的操作复杂度。Zhang 等^[25]提出一种专为 RDMA (Remote Direct Memory Access) 设计的拥塞控制算法 PACC, PACC 通过基于权重的分配方法,利用实时队列长度主动生成准确的拥塞反馈,并基于 DPDK 与 P4 交换机构建了 PACC 系统,能够将流的平均 PCT 降低 6%至 69%。

同时,为了在多核系统中更有效地均衡不同 CPU 核心的负载,Iqbal 等^[26]通过保持流局部性来维护数据包顺序,识别激进流 (Aggressive Flows) 以减少流从一个 CPU 核心迁移到另一个核心的频率,并为每个服务分配单独的 CPU 核心以提高指令缓存局部性,最终使得丢包数减少了 60%,乱序数据包数减少了 80%。Wang 等^[27]基于 DPDK 设计并实现了一个面向 NFV 的高性能 4 层网络负载均衡系统 HVLB,在转发端实现多核多队列的高效数据处理架构。

Barbette 等^[28]提出了一种新的负载均衡技术 RSS++, 通过动态修改 RSS 的间接表, 优化 CPU 核心之间的流量负载分布。与 RSS 相比, 在高 CPU 利用率下, RSS++ 的 95% 尾延迟只有 RSS 的十四分之一。Chen 等^[29]提出一种基于流特征的非对称流负载均衡方法, 以解决软件负载均衡系统不能充分发挥多核处理器和网卡硬件性能的问题, 该方法基于会话报文序列的时间与空间特征, 构建大象流识别算法, 并将大象流的负载均衡功能卸载到网卡上, 从而减少 CPU 处理器的负载均衡计算压力。Zhang 等^[30]提出一种主动负载均衡方法 iBalancer, 通过 PTI (Poll Time Interval) 来量化 CPU 核心的负载, 并根据负载情况主动处理网卡端的流调度, 在 500 毫秒的 99% 尾延迟下, iBalancer 的负载能力比 mTCP^[31]和 Shenango^[32]分别高 4.75 倍和 1.55 倍。

虽然 DPDK 通过采用轮询模式驱动显著提升了数据包处理性能, 但是轮询中的忙等待自旋锁不仅会浪费大量的 CPU 周期, 造成一定的功耗, 而且过高的 CPU 使用率会对系统中其他应用的性能造成影响。为了解决这个问题, DPDK 提供了混合中断轮询机制, 允许在网络流量较低时使用中断收包以降低轮询频率, 然而这种方法可能会导致流量突增时的处理延迟增大。此外, DPDK 还允许用户空间应用程序通过动态调整 CPU 频率或进入不同的 C-States 状态来节省电源。但在集群环境下, 降低主机 CPU 频率可能会影响设备上运行的其他应用程序的性能。此外, 在不同 C-States 之间的状态切换也需要更长的时间。在更为深入的研究中, Belkhiri 等^[33]提出一种称为有效 RX 自旋比的指标, 以评估轮询线程的有效 CPU 利用率, 该指标的计算方法是将调用轮询函数中有一个或多个数据包的次数除以该函数的总调用次数, 使用有效 RX 自旋比可以更好地评估轮询线程在处理数据包的繁忙程度。Li 等^[34]建立了一个分析模型以制定高性能数据包 I/O 的处理流程, 根据不同的流量负载推断出实际的 CPU 利用率和平均空闲周期, 同时使用 pause 指令在极短的 CPU 空闲期降低 CPU 功耗, 与原始 DPDK 相比, 实现了超过 30% 的功耗降低。Wu 等^[35]提出一种寻找最优线程休眠时长的模型, 它能够平衡线程 CPU 使用率和传输效率, 找到最优的休眠时长。Faltelli 等^[36]提出 Metronome, 它以间歇性休眠-唤醒模式代替 DPDK 的连续轮询, 并将 RX 接收队列在多个线程之间共享, 线程通过获取锁来获得队列的访问权限, 并处理队列中的所有数据包, 同时为了平衡 CPU 使用率与处理延迟, 建立了一个分析模型, 最终降低了 40% 的 CPU 利用率。

总体而言, 在基于 DPDK 的高速数据包处理上, 已经开展了许多研究。然而, 目前轮询机制会占用过多的 CPU 资源, 并且难以适应流量负载的变化。因此, 进一步研究能够自适应流量负载的轮询机制是十分必要的。

1.2.2 数据流技术

在网络测量领域, 伴随着网络带宽的快速增加, 网络测量的计算和存储需求也随之攀升。为此, 常用的方案有采样技术与数据流技术。采样技术通过选取流量数据的一个有代表性的子集来推测整体流量的特性, 但是采样技术存在着信息丢失、采样偏差以及精度与效率难以平衡的问题。数据流技术主要用于测量高速网络流量的统计信息, 常用的数据结构包括 Bitmap^[37]、Bloom Filter^[38]、Sketch^[39]与 Cuckoo 哈希表^[40]等。

在 Bloom Filter 的基础上, Xu 等^[41]又进一步提出了一种混合计数布隆过滤器 HyCBF, 将数据名前缀与前缀标记维护在同一个 CBF 中, 提高了数据名查找的效率, 达到了单核 12.41Gbps 的吞吐量。Li 等提出 SeeSaw 计数过滤器 (SSCF)^[42]与哈希自适应布隆过滤器 (HABF)^[43], 通过一个统一的计数数组将敏感的假阳键封装起来, 并动态调整所有哈希函数以防止封装的键被误判, 实验表明 SSCF 在准确性方面相较于 CBF 提升了 3 倍。

基于 Sketch, Tang 等^[44]提出一种快速紧凑的可逆 Sketch: MV-Sketch, 基于投票算法, MV-Sketch 实现了静态的小内存分配与轻量级内存访问的更新操作, 实现了线速的包数统计。Zhang 等^[45]设计了 CocoSketch, 它将任意部分关键词查询转化为子集和估计问题, 通过随机方差最小化, CocoSketch 在 CPU 性能和内存效率方面明显优于现有 Sketch, CocoSketch 还进一步去除循环依赖, 对可编程交换机和 FPGA 都具有硬件友好性, 相比于传统的单键 Sketch, 当测量六个流关键词时, CocoSketch 将平均数据包处理吞吐量提高了 27.2 倍, 准确度提高了 10.4 倍。Xiong 等^[46]提出一种网络大流弹性识别方法 RobustSketch, 基于 Sketch 的可伸缩小流过滤器, 根据实时分组到达速率扩增与缩减 Sketch 的数量, 从而确保网络流量波动时仍能精确过滤小流。同时设计基于动态分段哈希的可扩展大流记录表, 根据小流过滤器筛选后的候选大流数量增加或减少分段, 以完整记录所有候选的大流, 实验表明该方法即使在网络流量波动时也仍能稳定保持大流识别准确率在 99% 以上。

然而, Sketch 是为了优化内存效率而设计的结构, 基于 Sketch 的测量算法往往通过多个哈希函数来维护具有紧凑内存的计数器。而在基于软件的网络测量方法中, 内存并不是限制, 反而是多次的哈希运算成为了性能瓶颈。Huang 等^[47]指出虚拟交换机在进行基于 Sketch 的网络测量不可避免地会产生沉重的计算开销。Liu 等^[48]分析了软件中 Sketch 的关键瓶颈, 指出对每个数据包的哈希计算与多次随机内存访问是主要问题, 并设计了 NitroSketch, 通过减少每个数据包的 CPU 与内存操作次数, 减少了 45% 的 CPU 使用率。

在 Cuckoo 哈希的研究中, 为了提高 Cuckoo 哈希在插入与查找方面的性能, 目前主要从四个方向来开展研究: 缓解哈希冲突、简化操作、改进踢出策略与并发性优化。

在缓解哈希冲突方面, Erlingsson 等^[49]提出桶式布谷鸟哈希表 (Bucketized Cuckoo Hash Table, BCHT), BCHT 将每个桶划分为 2 至 8 个槽, 每个槽存储一个元素, 以此来处理哈希冲突。为了避免因递归插入到达阈值而导致扩容重哈希, Kirsch 等^[50]进一步提出 CHS (Cuckoo Hashing with a Stash), CHS 在传统 Cuckoo 哈希的基础上增加一个额外的存储空间, 用以临时存储插入失败的元素, 有效减少了对哈希表的重构次数, 实验表明, 仅需三或四个元素大小的存储空间即可显著改进系统的性能。

在简化操作方面, Guo 等^[51]提出 Single Hash 方法, 他们认为多次哈希计算将占用大量 CPU 资源, 而且对于 32 位的哈希值, Sketch 或 Cuckoo 哈希等并未充分利用 32 位信息, 造成了哈希值信息的损失, 因此他们通过简单的异或与移位运算将一个 32 位哈希值扩展至多个, 显著提升了基于哈希的数据结构的性能。Luo 等^[52]提出 Ark Filter, 通过元素指纹与过滤器长度之间的商或余数来索引两个候选桶, 并存储元素信息, 避免了进一步的哈希运算。同时, 为了进一步减少在查询时访问 CPU 缓存行的次数, Breslow 等^[53]提出了一种增强的桶式

Cuckoo 哈希表 Horton, 在 Horton 中大多数元素仅使用主哈希函数并存储在主桶中, 因此查询时通常只需要访问一个哈希桶。Jiang 等^[54]提出一个新的哈希表设计方案 DCuckoo, 在片内内存中使用指纹与 Bitmap 作为摘要, 在查询时先匹配指纹以减少对片外内存的访问次数。Scouarnec 等^[55]提出 Cuckoo++, 基于计数布隆过滤器进一步减少了查找不存在元素所需要访问的 CPU 缓存行次数, 相比于 DPDK 提供的 Cuckoo 哈希库提高了 45% 的查询性能。

在改进踢出策略方面, Frieze 等^[56]提出了 Random Walk (RW) 方法, 当某个元素的所有备选位置都被其他元素占据时, RW 将随机选择一个元素并踢出, 然后将待插入元素插入, 被踢出的元素则继续执行插入操作。如果递归插入的次数或者插入操作的时间达到预设的阈值, 则判定插入失败。Sun 等提出 SmartCuckoo^[57]与 MinCounter^[58]方法, SmartCuckoo 将哈希关系表示为有向伪森林图, 通过伪森林图的状态来预判是否会发生无限循环冲突, 避免了因陷入循环冲突而导致插入失败。MinCounter 为每个桶设置一个冲突计数器, 记录桶内执行踢出操作的次数, 当所有备选位置都已满时, MinCounter 会选择计数器最小的桶并将其中元素踢出。Wu 等^[59]提出 CostCounter, 不同于 MinCounter 以桶为单位, CostCounter 为每个槽分配一个成本计数器, 在发生冲突时使用成本计数器记录踢出的情况从而选择更好的路径, 实验表明在 95% 的哈希表负载率下, 相比 MinCounter, CostCounter 对读写次数的优化超过了 20%。

在并发性能优化方面, Fan 等^[60]设计了一种乐观 Cuckoo 哈希表 MemC3, MemC3 基于原子操作与版本控制机制实现了多个读线程与单个写线程之间的并发执行, 适用于读取密集型的应用场景。Sun 等^[61]提出 CoCuckoo, CoCuckoo 使用图粒度的锁机制, 避免为每个哈希桶单独分配锁, 降低了系统开销, 并缓解了锁资源争用, 同时支持高并发的读写操作, 显著提升了系统的吞吐量。

由于 Cuckoo 哈希能够保证最坏情况下的常数级查询, 因此也被广泛用于网络测量领域。Reviriego 等^[62]提出了一种新的数据结构 CuCoTrack, 用于执行流跟踪, CuCoTrack 基于布谷鸟哈希技术, 通过使用小型指纹而非存储完整的五元组来实现无冲突方案, 这大幅减少了存储连接所需的内存, 实验表明在使用 16 位指纹时插入失败的概率降低至 10^{-10} 。Yan 等^[63]提出 Cuckoo Sketch 方法, 将大流存储在 Cuckoo 哈希表中, 并存储了流的准确大小, 而小流则存储在 Count-Min Sketch 中, 通过 Cuckoo 哈希的踢出机制驱逐老化的流来为大流在 Cuckoo 哈希表中腾出空间。Shi 等^[64]提出一个基于 Cuckoo 哈希的自适应计数器 Cuckoo Counter (CC), CC 由多个桶组成, 每个桶中槽的大小不同, 以分别统计小流与大流的频次, 当发生溢出时, CC 尝试将流重定位到同一桶中更大的槽中, 当发生冲突时 CC 将存储在最小槽中的流驱逐出去, 实验表明, 相比 ASketch, CC 在吞吐量与准确性上提高 1 至 2 个数量级。Xiao 等^[65]提出一种基于 Cuckoo Filter 的算法 PFD-DW, PFD-DW 将每个桶里的槽组织为最小堆, 并基于时间衰减窗口对 Top-k 持久流进行监测。

总的来说, 在基于数据流的网络测量领域, 已经开展了大量研究工作, 然而无论是 Sketch 还是 Cuckoo 哈希都依赖多次哈希运算, 但在基于通用计算平台的网络测量中, 多次的哈希运

算反而成为了性能瓶颈。此外，当需要对网络流的多维特征进行细粒度测量时，Sketch 或 Cuckoo 哈希均面临着挑战，难以匹配 DPDK 等高速网络数据处理框架的速度。

1.3 研究内容

本文以基于 DPDK 实现高速自适应的细粒度网络测量为目标，着重解决两个研究难点：一是在 DPDK 轮询收包过程中，CPU 始终保持满载状态，难以根据流量规模自适应负载；二是在基于 Cuckoo 哈希的网络测量中，存在着多次哈希运算复杂度高、多个备选桶查询开销大以及高负载时可能产生循环冲突的问题。因此，本文尝试建立 DPDK 轮询收包的分析模型，并研究根据流量规模变化的动态自适应轮询调度方法；接着，本文研究低开销的哈希运算与基于 Cuckoo 哈希的高速网络测量方法；最后，本文结合上述研究实现了原型系统。本文的研究内容框架如图 1-3 所示：

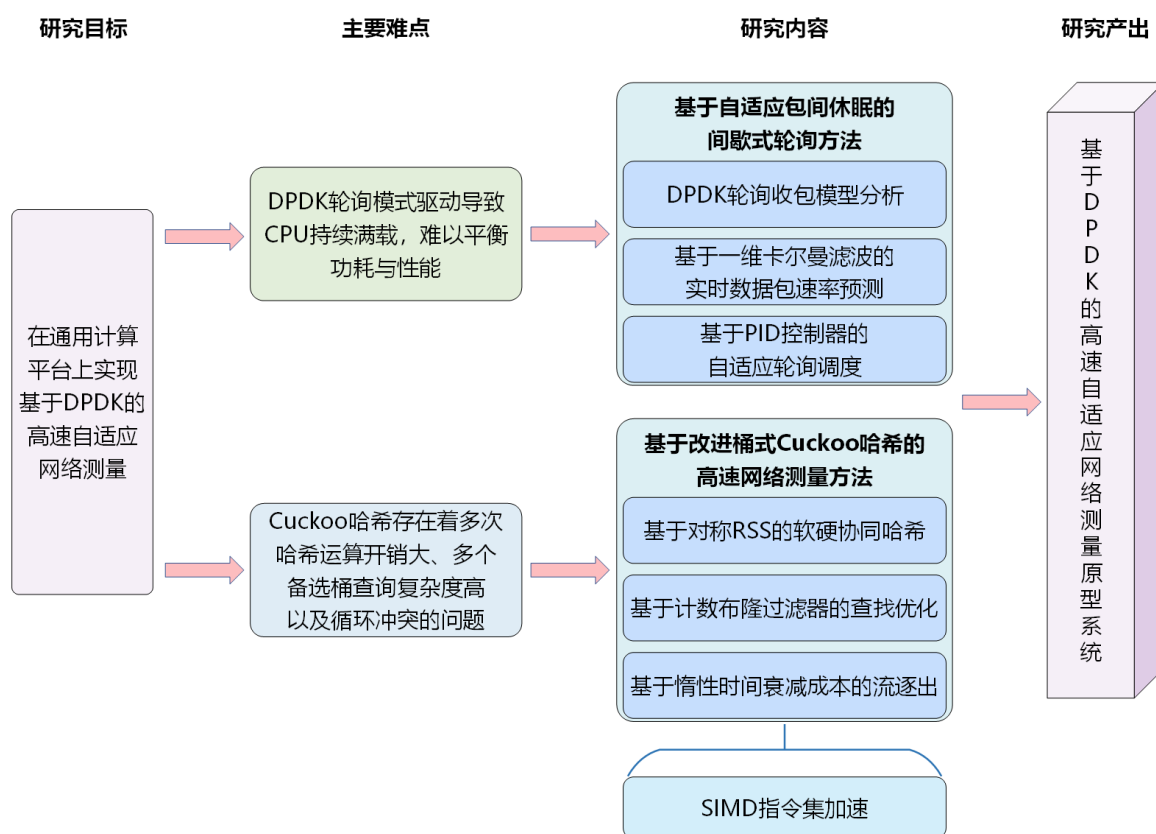


图 1-3 研究内容框架图

具体而言，本文的研究内容可以总结为如下三点：

(1) 基于自适应包间休眠的间歇式轮询方法。针对 DPDK 轮询收包中功耗与性能的平衡问题，本文提出基于自适应包间休眠的间歇式轮询方法，通过对 DPDK 轮询收包过程进行建模，分析当前数据包速率下休眠时长的期望，同时本文基于一维卡尔曼滤波对数据包速率进行实时在线预测，并基于 PID（Proportional-Integral-Derivative）控制器，处理 RX 接收队列的期望负载长度与实际负载长度之间的差距反馈，动态调整每次空轮询后的休眠时长，实现 DPDK 轮询收包过程中对性能与功耗的有效平衡。

(2) 基于改进桶式 Cuckoo 哈希的高速网络测量方法。针对 Cuckoo 哈希的网络测量中,存在着多次哈希运算复杂度高、多个备选桶查询开销大以及高负载时可能产生循环冲突的问题,本文提出基于改进桶式 Cuckoo 哈希的高速网络测量方法,使用网卡硬件生成的对称 RSS 哈希值来减少 CPU 哈希运算的开销,同时基于桶式 Cuckoo 哈希作为网络测量键值对哈希表的索引,并基于计数布隆过滤器减少不必要的查询流程,最后基于惰性时间衰减算法计算流的成本,以优化 Cuckoo 哈希的踢出路径,避免陷入循环冲突。此外,本文还基于 AVX 与 SSE 指令集加速上述方法。

(3) 基于 DPDK 的高速自适应网络测量原型系统。本文对上述两项研究成果进行整合,在通用计算平台上构建了基于 DPDK 的高速自适应网络测量原型系统,实现了最高 14.88Mpps 的细粒度网络测量。

1.4 论文组织结构

本文围绕基于 DPDK 的高速自适应网络测量进行相关研究,主要通过六章来进行阐述说明,各个章节的内容安排如下:

第一章主要介绍了本文的研究背景与研究意义,同时分析了当前国内外在基于 DPDK 的高速网络测量的研究现状,并从整体上介绍了本文的研究内容。

第二章介绍 DPDK 与 Bitmap、Bloom Filter、Sketch、Cuckoo 哈希等相关背景技术知识。

第三章介绍了基于自适应包间休眠的间歇式轮询方法,首先对 DPDK 轮询收包过程进行建模分析,随后基于一维卡尔曼滤波对数据包速率进行快速预测,并结合 PID 控制器动态调整休眠时长,最后开展实验对方法性能进行评估,并与现有方法进行对比。

第四章介绍了基于改进桶式 Cuckoo 哈希的高速网络测量方法,首先对网卡对称 RSS 哈希进行扩展,并设计桶式 Cuckoo 哈希结构作为键值对哈希表的索引,同时基于计数布隆过滤器减少查询开销,并基于惰性时间衰减成本优化踢出路径,最后开展实验对方法性能进行评估。

第五章为基于 DPDK 的高速自适应网络测量原型系统的设计与实现,在需求分析的基础上设计了系统的整体架构,并进一步介绍了各模块的主要功能与流程,最后展示了系统运行结果并对系统极限性能进行了测试。

第六章为总结与展望,对本文的研究内容进行整体总结,思考和分析本次研究工作存在的不足,对今后的研究方向做出展望。

第二章 相关背景技术

本章主要介绍了与本文研究内容相关的背景技术,首先介绍了 DPDK 的主要技术与原理,接着介绍了 Sketch 与 Cuckoo 哈希等的基本结构,以及相应的插入查询操作流程,为本文第三章与第四章的方法提供理论知识。

2.1 DPDK

DPDK 是由 Intel 等提出的一套开源框架,包含一套高性能数据包处理函数库与驱动程序集,被广泛应用于高性能数据包处理中,它通过 UIO (Userspace I/O) 技术绕过操作系统内核网络协议栈,将网卡的地址空间和中断映射至用户态空间,从而支持在用户态空间直接处理数据包,减少了不必要的数据包拷贝与上下文切换,显著降低网络 I/O 的延迟,为通用平台提供了高效可扩展的数据包处理性能。DPDK 还通过轮询模式驱动(PMD, Poll Mode Driver)、无锁队列、大页内存、CPU 亲和性等技术,有效提高了数据包处理的效率。

DPDK 提供了多种核心组件,包括内存管理、队列状态、缓冲区管理和多核处理器支持等,其核心组件的体系架构如图 2-1 所示。

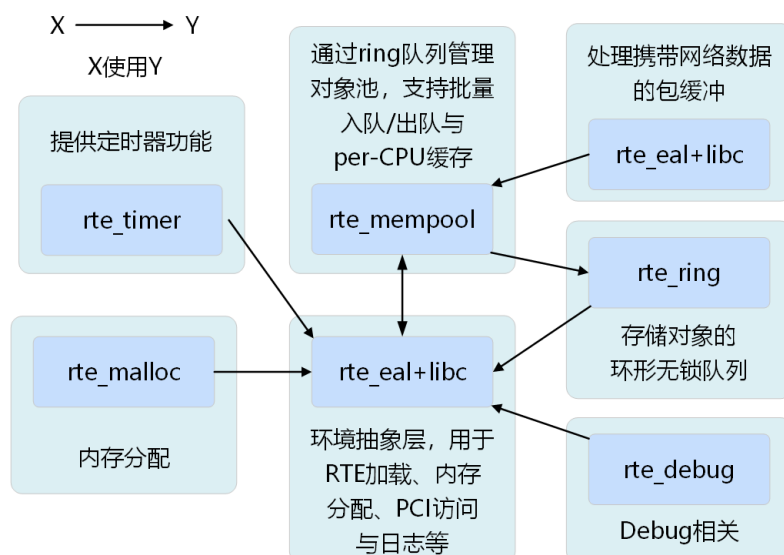


图 2-1 DPDK 核心组件图

2.1.1 UIO 技术

Linux 内核在处理网络流量时,需要频繁地在用户态与内核态之间进行切换,这极大地影响了 Linux 内核的数据包处理性能。UIO 是一种在用户空间执行 I/O 操作的方法。在传统的 Linux 系统中,设备驱动主要运行在内核空间,应用程序则在用户空间进行调用;UIO 则将大部分设备驱动的功能操作转移至用户空间,只留下很小一部分在内核空间运行。DPDK 利用 UIO 技术,将 Linux 内核态的数据包处理流程转移至用户态,避免多次拷贝与上下文切换。

2.1.2 轮询模式驱动技术

在传统网络通信机制中,当网卡接收报文时,会向 CPU 发出中断请求,CPU 在接收到中断信号后,会先保存当前寄存器状态,随后依据中断服务例程(Interrupt Service Routine, ISR)

执行相应处理，最后恢复堆栈中保存的寄存器状态并继续之前的操作。然而，此机制在高速网络环境中会导致显著的中断开销，严重影响数据包处理的效率。

为了减少中断带来的性能损耗，DPDK 基于轮询来处理网络数据包，当网卡接收到数据包时，不发出中断信号，而是直接将数据包暂存到缓存区或内存中，并设置标志位。应用程序通过周期性的轮询这些标志位来检测是否存在待处理的数据包。轮询机制大幅减轻了系统的中断负荷，提高了数据包处理性能，让应用程序能够快速高效地处理数据报文。

2.1.3 无锁队列技术

与传统的基于锁的同步机制不同，无锁队列通过 CAS（Compare And Swap）等原子操作确保对共享数据的修改是原子性的，保证了同一时刻只允许单个线程修改数据，其他线程在检测到冲突时会根据设计的策略进行重试或退让，直到操作成功为止，无锁队列的结构如图 2-2 所示。无锁队列依赖于硬件的支持，绕过了操作系统层面的同步机制，从而显著降低了因操作系统层面的上下文切换和调度延迟所引起的开销。通过无锁队列，DPDK 保证了数据包处理的高吞吐与低延迟，使得网络数据包在多核处理器之间的传递更为高效。

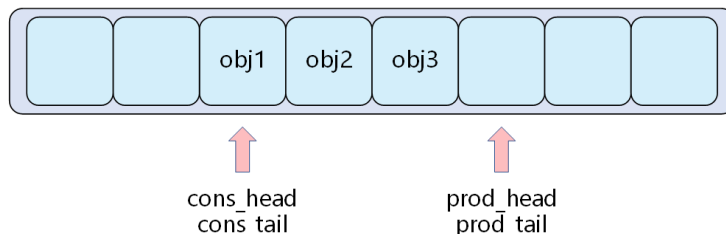


图 2-2 DPKK 无锁队列结构图

2.1.4 RSS 技术

Receive-Side Scaling（RSS）是利用网卡的基础硬件特性实现的多队列机制，通过对数据包的特定字段（如 IP 地址和端口号）进行哈希计算，进而将流量分散到多个接收队列中，以实现数据包在多个队列间的负载均衡，如图 2-3 所示。RSS 允许多个 CPU 核心并发访问不同的网卡队列，有效避免了访问网卡队列时锁与同步的开销。同时，RSS 哈希是基于网卡硬件实现的，所以整个过程并没有额外的 CPU 周期消耗，能够有效提高通用计算平台对数据包的处理能力。

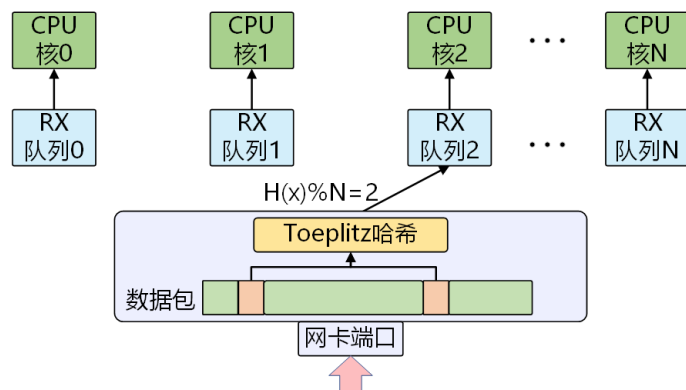


图 2-3 RSS 示意图

2.2 数据流技术

数据流技术主要用于测量高速网络流量的统计信息，常用的数据结构有 Bitmap、Bloom Filter、Sketch 与 Cuckoo 哈希表等。

2.2.1 Bitmap

在 Bitmap 中所有位初始化为 0，当元素到达时，Bitmap 通过哈希函数将其映射至数组特定的位上，并将该位设置为 1，如图 2-4 所示。但是由于哈希冲突，Bitmap 的存在较大的误差，且当数据集范围很广时，Bitmap 需要大量的存储空间。

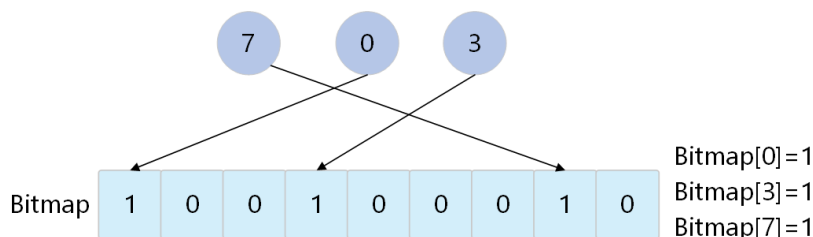


图 2-4 Bitmap 示意图

2.2.2 Bloom Filter

布隆过滤器 (Bloom Filter, BF) 通过一个长为 m 的位数组与 k 个哈希函数来实现，在初始状态，位数组所有位都设为 0，在元素插入时，使用 k 个哈希函数将元素 x_i 分别映射到位数组的 k 个位置，并将这些位都赋值为 1。查询时，同样将元素映射到 k 个位置，并检查这 k 位是否全为 1，如果全为 1 则可能存在，如果任一位为 0，则必然不存在，如图 2-5 所示。然而，布隆过滤器并不支持对元素计数，因此产生了 CBF (Counting Bloom Filter)、SBF (Spectral Bloom Filter) 等^[66]变体，为元素的频数估计提供了基础。

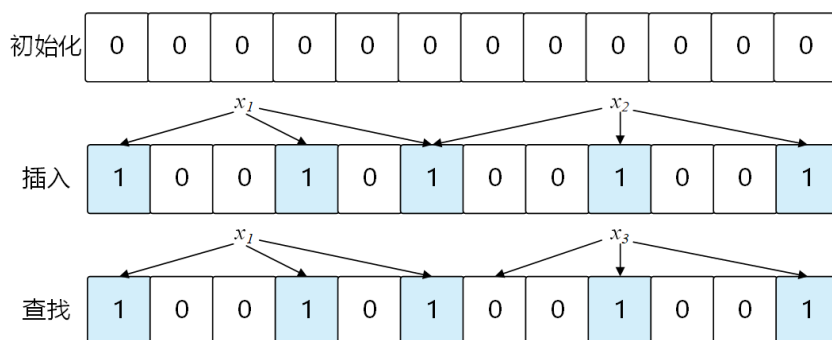


图 2-5 布隆过滤器示意图

2.2.3 Sketch

Sketch 通过哈希将元素映射到一个紧凑的哈希表中，实现高效的数据计数和特征统计，可以在有限的存储空间内保留大体量数据信息的统计特性。在网络测量中，以 Sketch 为基础的数据流算法已经被广泛应用，通常一种 Sketch 结构是专门针对一种或几种测量任务而设计，常见的 Sketch 有 Count-Min Sketch (CMS)^[67]、Count Sketch (CS)^[68]与 Reversible Sketch (RS)^[69]等，图 2-6 描述了 CMS 的插入与查询操作：

图 2-8 展示了三种不同场景下元素 x 插入哈希表的过程，元素 x 通过哈希运算被映射到两个潜在的位置。在图 2-8(a)中，两个位置均空闲，则 x 可被插入任一位置；在图 2-8(b)中， T_1 位置已经被元素 x 占用，而 T_2 位置空闲，所以将 y 插入 T_2 ；在图 2-8(c)中， T_1 和 T_2 均被不同元素占用，则会将 T_1 位置中的元素 x 踢出，将 z 插入该位置，并尝试将移除的元素 x 重新插入。如果连续踢出操作的次数达到指定的阈值，则触发哈希表的重构，清空当前表并扩容至原容量的两倍，随后重新插入所有元素。

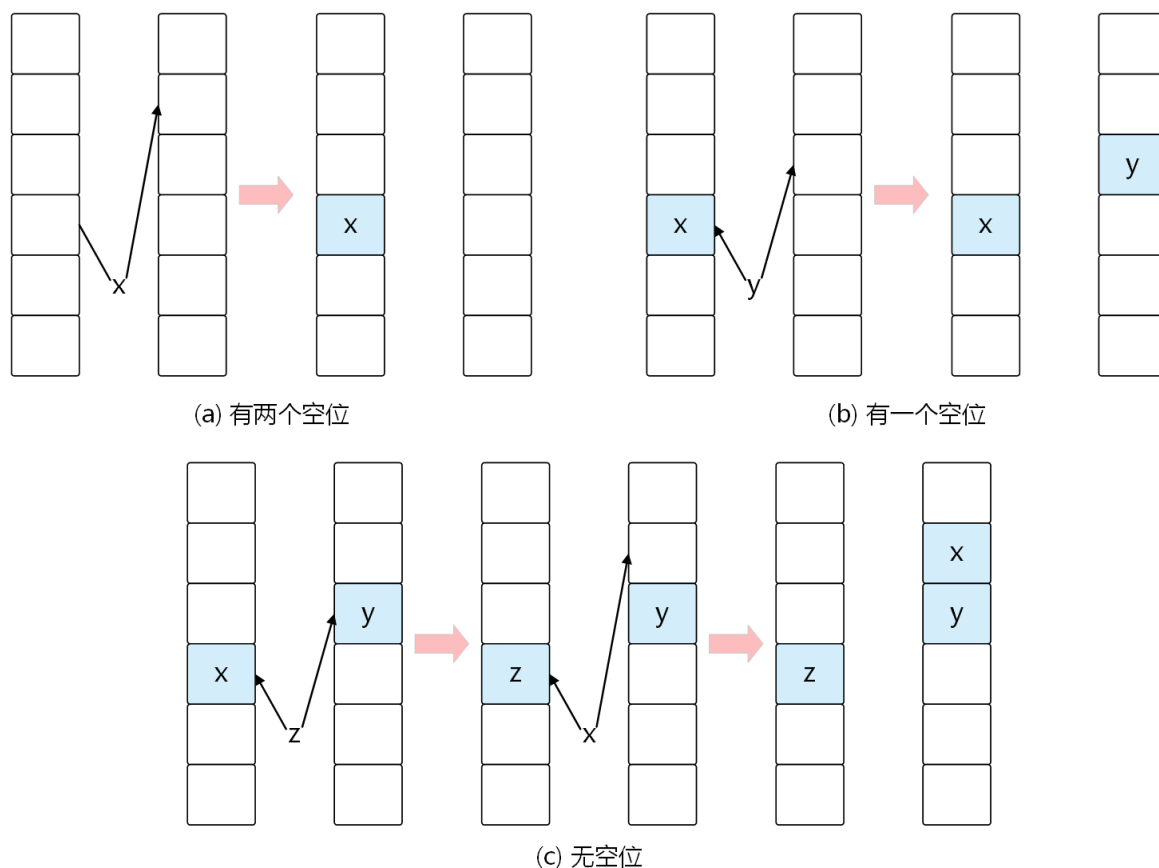


图 2-8 Cuckoo 哈希插入示意图

随着 Cuckoo 哈希表中元素的增加，表的负载因子逐渐增大，探测到不为空的候选桶的几率增加，哈希冲突的概率也会随之增长，而当踢出次数达到预设的阈值后，Cuckoo 哈希表需要使用新的哈希函数将所有元素再重新插入到更大的哈希表中，在此过程中哈希表处于完全不可用状态，导致时间与空间资源的巨大浪费。

桶式 Cuckoo 哈希表 (Bucketize Cuckoo Hash Table, BCHT) 是 Cuckoo 哈希技术的一种变体，它在解决哈希冲突与提高空间利用率方面做出了进一步的优化。与传统的 Cuckoo 哈希每个位置仅能存储一个元素不同，桶式 Cuckoo 哈希表将哈希表的每个位置扩展为一个“桶”，每个桶可以存储多个元素。当发生哈希冲突时，只要所在桶未满，新的元素就可以直接插入到这个桶中，减少了元素踢出与重定位的次数。当所有目标桶均已满导致新元素无处安放时，将尝试移动现有元素或采用备用哈希函数为新元素腾出空间，这可能涉及到一个链式重定位过程，可能会影响到一个或多个桶中的多个元素。如果仍无法成功插入新元素，哈希表则可能需要进行扩容或重哈希操作。

然而，在采用 d 个哈希函数的桶式 Cuckoo 哈希表中，查询一个元素最多需要探测 d 个桶，且还需要进一步探测桶中的所有槽。假设一个键值对存在于哈希表中，当按顺序进行探查时，平均需要探查约 $d/2$ 个桶，这既增大了查询延迟，也占据了更多不必要的内存带宽。

2.3 本章小结

本章聚焦于基于通用计算平台的高速网络测量的核心概念及相关技术，首先本章介绍了 DPDK 的关键组件与技术，包括 UIO、轮询模式驱动、无锁队列与 RSS 技术。随后，本章介绍了常见的数据流技术，描述了 Bitmap、Bloom Filter、Sketch 与 Cuckoo 哈希的工作机制与基本操作流程，并重点分析了 Cuckoo 哈希在冲突管理中的应用与效率，为高速网络环境下的细粒度测量提供了理论基础。

第三章 基于自适应包间休眠的间歇式轮询方法

随着数据传输速率的快速提升，DPDK 等高速数据包处理框架应运而生，基于零拷贝、批处理和轮询等技术，有效提升了数据包处理效率。然而，轮询机制导致了 CPU 即使在网络流量规模较低的情况下也始终保持满负载运行。为优化 DPDK 轮询过程中的 CPU 使用率，本章在轮询中引入精细的休眠调节，以在确保低丢包率的同时减轻 CPU 的工作负荷。具体而言，当网络流量速率较低时，通过在轮询循环中加入休眠指令，避免 CPU 无用的轮询循环，实现了更高效的资源利用。然而，如何精准确定休眠时长，则需要对 DPDK 轮询收包的处理流程及其时间分布进行深入分析。

因此，本章提出一种基于自适应包间休眠的间歇式轮询方法，旨在通过实时预测数据包速率，并结合 PID 控制算法，动态调整轮询过程中的休眠时长，以此实现处理性能与功耗消耗之间的优化均衡，整体方法架构如图 3-1 所示。首先，本章对 DPDK 轮询收包过程进行分析，计算当前数据包速率的理想休眠时长期望。紧接着，本章提出一种基于一维卡尔曼滤波的数据包速率预测方法，该方法使用一维卡尔曼滤波算法预测数据包速率，并对预测进行实时更新以响应网络的动态变化，为网络流量的变化趋势提供可靠的预测。随后，本章提出一种基于 PID 控制器的自适应轮询调度方法，根据当前轮询-休眠周期下 RX 接收队列的期望负载长度与实际负载长度之间的差距反馈，动态调整休眠时长，通过精细调控轮询周期，有效平衡了 DPDK 轮询收包过程中的性能与功耗。最后，本章通过实验验证了本方法在高速网络测量上的可行性与有效性。

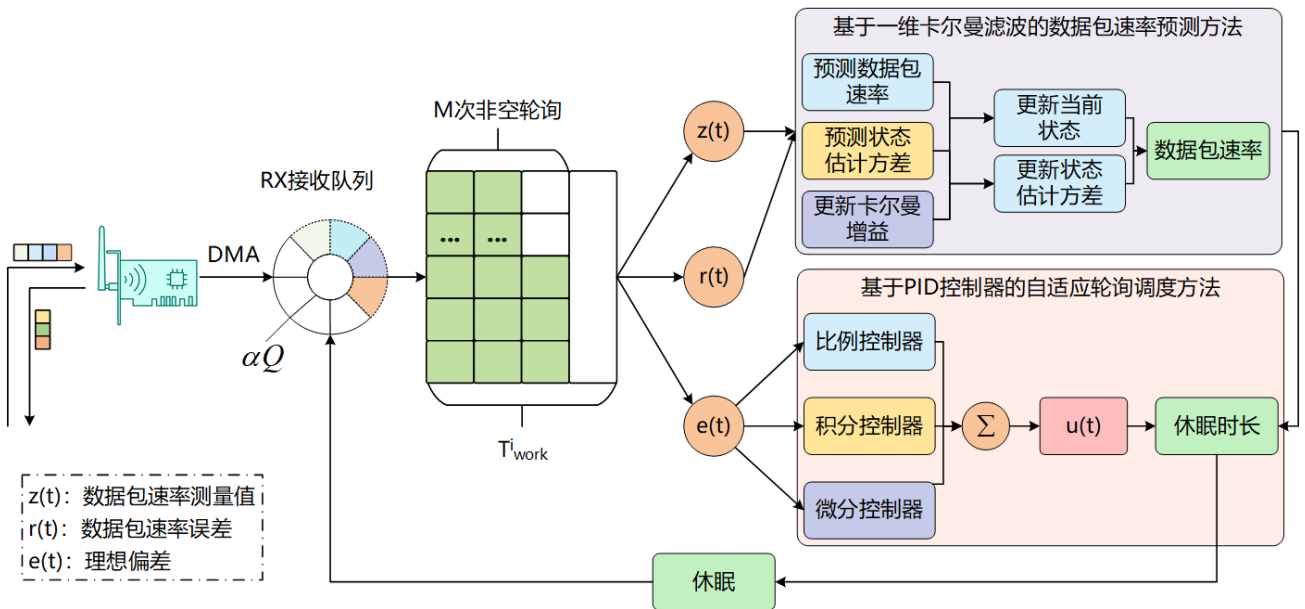


图 3-1 基于自适应包间休眠的间歇式轮询方法架构图

3.1 问题分析

传统数据包处理基于 Linux 内核协议栈实现，当数据包到达网卡时，网卡通过 DMA (Direct Memory Access) 技术将其复制到可用的 sk_buff 中，并将相应的环形缓冲区描述符标记为已

用。随后，网卡向 CPU 发送硬件中断请求，让 CPU 处理新来的数据包，CPU 通过执行注册的中断服务程序来获取数据包，然后将其交由内核协议栈以进行进一步的协议处理，并最后发送给用户空间应用程序。然而，内核网络协议栈的处理机制会导致大量的软硬件中断、多次内存拷贝、系统调用以及处理器大量频繁的进程上下文切换，这些产生大量的资源开销，导致无法及时处理数据包，引起网卡数据堆积，造成服务器网卡在收包时产生大量丢包，难以适用于高速网络流量的数据包处理。

近年来，10Gbps 网络链路广泛应用于数据中心和运营商网络中，用于连接服务器、交换机与路由器，由于操作系统内核网络协议栈带来的开销（数据包拷贝、中断等），CPU 在处理此类高速链路的数据包时很快就成为瓶颈。为此，一些新型高性能数据包处理框架被提出，如 Netmap、DPDK 和 PF RING，这些框架绕过内核网络协议栈并利用零拷贝、批处理和轮询等技术来实现高吞吐量和低延迟的数据包处理。

尽管 DPDK 等在吞吐量和延迟方面具有显著优势，但是，繁忙的轮询不仅会浪费大量的 CPU 周期，导致功耗的增加，同时 CPU 的高使用率也会对服务器中其它应用程序的性能产生严重的影响。因为轮询会导致 CPU 核心处于活跃状态并始终以最大频率运行，这样，当网络链路空闲或流量速率较小时，大部分 CPU 周期将浪费在无意义的空轮询上，从而导致 DPDK 的功耗效率低下。而且，网络流量往往不是持续的高负载，网络的平均链路利用率仅在 20% 到 40% 之间，且网络流量还具有“潮汐效应”的特征。网络流量的“潮汐效应”和较低的平均链路利用率意味着 DPDK 的轮询将浪费大量的 CPU 功耗。

为了解决轮询机制中 CPU 持续的高利用率和能源浪费的问题，DPDK 允许用户空间应用程序通过利用 DVFS（Dynamic Voltage and Frequency Scaling）和 LPI（Low Power Idle）等电源管理技术来降低功耗。具体来说，DVFS 通过根据工作负载调整各个 CPU 内核的频率和电压来减少不必要的功耗，然而 DVFS 虽然降低了轮询线程的 CPU 核心的时钟频率，但是 CPU 的使用率仍然维持在 100%，因此该 CPU 核心依然无法用于其他任务。此外，降低主机 CPU 频率可能会影响服务器上运行的其他业务应用的性能，而且调整 CPU 频率的转换等待时间较长，无法适应网络流量的快速变化。而 LPI 则在 CPU 内核空闲时通过关闭某些支持电路的电源来节省功耗，但状态之间切换的等待时间较长，不适合实时切换的场景。

因此，本文提出基于一种基于自适应包间休眠的间歇式轮询方法，旨在根据实际的网络流量速率动态调整 CPU 的休眠与唤醒参数，通过使用一维卡尔曼滤波对网络流量的数据包速率进行实时预测，并基于 PID 控制器的反馈，动态调整休眠时长，从而在保持轮询收包性能的同时最小化功耗损失。

3.2 DPDK 轮询批处理分析

为提高网络数据包处理的效率，DPDK 采用了轮询与批处理技术，一般来说，DPDK 中的每个线程都会被固定到一个专用的 CPU 核心来循环执行数据包处理任务。在 DPDK 中数据包处理循环包括以下步骤：

- 1) 首先，在数据包的接收阶段，CPU 通过轮询 RX 接收队列来检索一批数据包；

2) 接着, 在数据包的处理阶段, CPU 处理批次中的每个数据包;

3) 最后, 在数据包的发送阶段, CPU 将处理后的数据包发送到 TX 发送队列。

而当 RX 环形缓冲区为空时, CPU 核心仍会对 RX 接收队列执行忙检索 (即空轮询), 直到有数据包到达, 然后再次进入上述处理循环, 如图 3-2 所示。

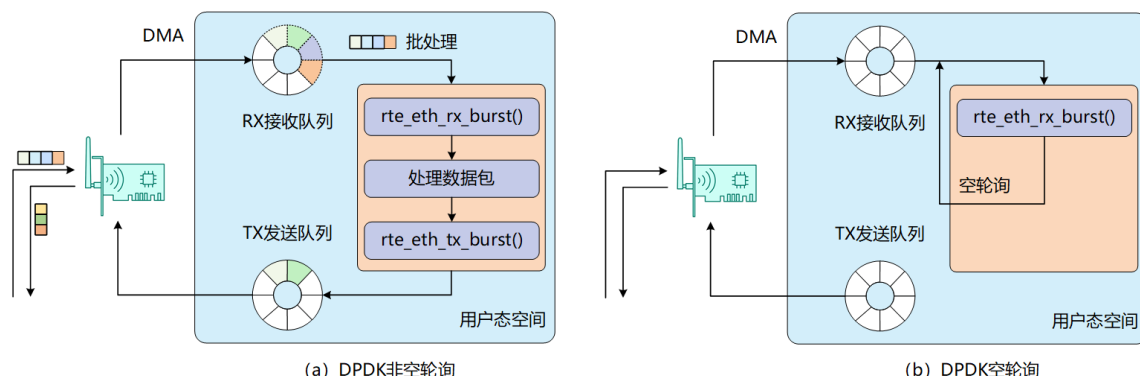


图 3-2 DPK 轮询收包与空轮询对比图

具体来说, 在数据包的接收阶段, DPK 基于网卡的 DMA 功能, 采用零拷贝技术, 将网络数据包直接传输至预先配置的 RX 环形接收队列中, 这一过程并不依赖 CPU 的直接参与, 有效地减少了传统数据包处理过程中数据拷贝和内存分配等的开销。随后, DPK 会周期性地调用 `rte_eth_rx_burst()` 函数, 一次性从 RX 环形接收队列中提取出多个数据包, 这种批处理方式显著减少了因频繁的函数调用而产生的额外开销。DPK 通过采用轮询批处理技术, 不仅减轻了 CPU 的中断处理压力, 而且充分利用了现代 CPU 的矢量处理指令集, 显著提升了数据包的接收效率。

在数据包的处理阶段, DPK 提供了高度可定制化的数据包处理流程, 用户可以根据具体需求编写自定义的数据包处理逻辑, 并且由于批处理的优势, 大批量数据包可被集中高效地处理, 充分发挥了现代多核 CPU 的并行计算能力。在处理完成后, DPK 通过 `rte_eth_tx_burst()` 函数以批量形式再将数据包发送出去, 同样遵循了批处理的思路, 降低了发送函数的调用开销。值得一提的是, DPK 在整个收包、处理、发包的流程全部在用户态空间内完成, 无需操作系统内核的参与, 避免了内核态和用户态之间的切换开销。

在 DPK 的轮询批处理过程中, CPU 会不断检查 RX 环形接收队列, 以检测是否有新的数据包到达, 与中断模式相比, 轮询模式极大地降低了处理中断所导致的延迟, 对于高吞吐量、低延迟的网络数据包处理具有显著优势, 但是轮询也大幅提高了 CPU 的使用率, 造成了功耗的损失。

为了降低 DPK 轮询过程中的 CPU 使用率, 当网络流量规模较小时, 可以通过在 DPK 轮询中加入休眠函数来降低 CPU 负载, 如图 3-3 所示。为了进一步探寻 DPK 数据包处理过程中的最佳休眠时间, 本文构建了用于评估分析 DPK 轮询收包的分析模型, 通过评估和解析 DPK 在数据传输过程中的时间分配, 量化分析 CPU 在数据包接收与处理所需的时间分布情况, 从而科学地确定休眠时长, 保障 DPK 轮询收包过程中, 数据包处理效率与 CPU 功耗损失之间达成最优平衡。

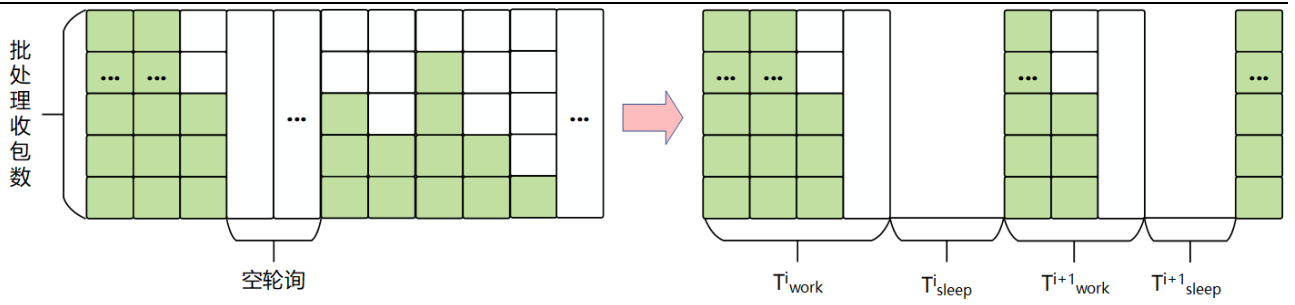


图 3-3 DPDK 轮询与间歇式轮询对比图

本文定义 DPDK 在整个循环收包过程中的线程执行总时间为 T_{total} ，则 T_{total} 可以分为两个部分：

$$T_{total} = \sum_{i=1}^N T_{cycle}^i = \sum_{i=1}^N (T_{work}^i + T_{sleep}^i) \quad (3-1)$$

其中， T_{cycle} 表示一次轮询收包-休眠循环， N 表示在本次 DPDK 程序运行过程中共有 N 次轮询收包-休眠循环， T_{work}^i 表示在第 i 次循环中 CPU 轮询收包的工作时间， T_{sleep}^i 表示在第 i 次循环中休眠的时间。

更进一步地，本文将 DPDK 收包工作进行分解：

$$T_{work}^i = \sum_{j=1}^{M_i} (T_{rx}^j + \sum_{k=1}^{B_j} T_{handle}^k) \quad (3-2)$$

其中， M_i 表示在第 i 次轮询收包-休眠循环中的轮询批处理次数， T_{rx}^j 表示在第 j 次轮询批处理过程中调用 `rte_eth_rx_burst()` 函数所消耗的时间， B_j 表示在第 j 次轮询批处理过程中 `rte_eth_rx_burst()` 的收包数， T_{handle}^k 表示在第 j 次轮询批处理过程中对第 k 个数据包进行处理所需要的时间。

同时，考虑到不同的休眠函数都有相应的延时，所以：

$$T_{sleep}^i = T_{delay}^i + T_{overhead} \quad (3-3)$$

综上，对于第 i 次轮询收包-休眠循环有：

$$T_{cycle}^i = T_{work}^i + T_{sleep}^i = \sum_{j=1}^{M_i} (T_{rx}^j + \sum_{k=1}^{B_j} T_{handle}^k) + T_{delay}^i + T_{overhead} \quad (3-4)$$

假设 t 时刻的数据包速率为 P_t ，DPDK 的 RX 接收队列长度为 Q ，为了满足不丢包的条件，则需要满足公式 3-5：

$$\int_0^{T_{cycle}^i} P_t dt \leq Q \quad (3-5)$$

不妨设 T_{cycle}^i 时间内的数据包平均速率为 $\bar{P}^i$ ，即：

$$\bar{P}^i = \frac{\int_0^{T_{cycle}^i} P_t dt}{T_{cycle}^i} \leq \frac{Q}{T_{cycle}^i} \quad (3-6)$$

因此, 在平均数据包速率为 $\bar{P}^i$ 的情况下, 第 i 次轮询收包-休眠循环中的休眠时长需要满足公式 3-7:

$$T_{cycle}^i \leq \frac{Q}{\bar{P}^i} \quad (3-7)$$

更进一步的, 可以得到在第 i 次轮询收包-休眠循环过程中的 T_{delay}^i , 如公式 3-8 所示:

$$T_{delay}^i \leq \frac{Q}{\bar{P}^i} - \sum_{j=1}^{M_i} (T_{rx}^j + \sum_{k=1}^{B_j} T_{handle}^k) - T_{overhead} \quad (3-8)$$

在 DPDK 系统中, 通常可以认为调用 `rte_eth_rx_burst()` 的时间开销为一个固定的值, 即 $T_{rx}^j = T_{rx}$ 。同时, 为了简化分析模型, 不如假设处理每个数据包所需要的时间开销也是常数, 即 $T_{handle}^k = T_{handle}$, 因此简化后的 T_{delay}^i 如公式 3-9 所示:

$$T_{delay}^i \leq \frac{Q}{\bar{P}^i} - \sum_{j=1}^{M_i} (T_{rx} + T_{handle} \times B_j) - T_{overhead} \quad (3-9)$$

最后, 考虑到流量的实时性与动态性, 本文使用一维卡尔曼滤波对数据包速率进行实时预测, 同时根据当前轮询收包-休眠循环中, RX 接收队列的期望负载长度与实际负载长度之间的差距反馈, 结合 PID 反馈控制器, 对休眠时长进行动态调整。

3.3 基于一维卡尔曼滤波的数据包速率预测方法

在现代高速网络环境中, 网络流量具有显著的时间相关性, 通过建立预测模型来捕获这些特征, 并对未来一段时间内的数据包速率进行预测, 对于网络资源的合理分配、流量控制以及拥塞管理等方面起着至关重要的作用。数据包速率预测为网络设备提供了关键的流量趋势信息, 使其能够提前进行资源规划, 进而优化网络性能, 提高用户体验。然而, 现代网络中的流量模式呈现出高度的动态性、突发性和多变性, 给速率预测带来了巨大的挑战。在网络流量传输过程中, 数据包的传输速率受到诸多动态变化因素的影响, 例如网络拥堵、路由变化以及链路带宽波动等。

当前, 常用的数据包速率预测方法包括基于时间序列分析的自回归模型 (如自回归积分滑动平均模型 ARIMA)、基于机器学习的算法 (如支持向量机 SVM 和神经网络) 等, 尽管机器学习方法在预测准确性方面表现出色, 但它们在实时性和资源消耗方面存在挑战, 特别是在 DPDK 轮询收包过程中, 这些方法由于计算和存储资源的高开销, 往往难以直接应用。

卡尔曼滤波是一种在动态不确定性系统中做出高效预测的方法, 它结合系统状态的预测与实时观测数据的更新, 对预测结果进行优化, 适用于需要连续调整的场景, 以应对输入数据变化的影响。本章提出一种基于一维卡尔曼滤波的数据包速率预测方法, 探讨如何利用卡尔曼滤波算法对实时网络数据包速率进行有效的预测, 本文首先构建表征 DPDK 轮询收包过程的动态系统模型, 并使用一维卡尔曼滤波算法对网络状态进行快速且准确的预测, 为进一步的自适应轮询调度方法奠定基础。

针对网络流量的时间相关性和动态性特征，本章基于一维卡尔曼滤波算法预测数据包速率，首先定义数据包速率状态空间，并通过一系列的迭代计算过程，结合预测数据和实时观测值，对预测结果进行实时优化，能够有效捕捉网络流量的短期相关性和长期趋势，实现对数据包速率的实时预测。在本模型中，卡尔曼滤波主要包含两个阶段：预测阶段和更新阶段。

在预测阶段中，模型结合过去的状态和系统的动态模型，计算当前时间的预测状态以及估计误差协方差。随后，在更新阶段中，模型结合最新的观测数据和预测状态，通过卡尔曼增益对状态估计值进行修正，并更新误差协方差，从而校正由模型或外界环境噪声导致的预测偏差。

通过连续的观测和预测更新，一维卡尔曼滤波能够适应网络状态的变化，提供对数据包速率变化的准确估计。由于算法的递归性质，一维卡尔曼滤波能够实时更新其状态参数，而无需存储全部的历史数据，这样显著降低了系统计算与存储的负担。即使在网络环境中数据包速率呈现非线性变化或受到拥塞和噪声的影响，一维卡尔曼滤波模型也能保证预测结果的鲁棒性和准确性。

本章基于一维卡尔曼滤波算法，为 DPDK 轮询批处理收包建立数学模型，并实时预测数据包速率，如图 3-4 所示。

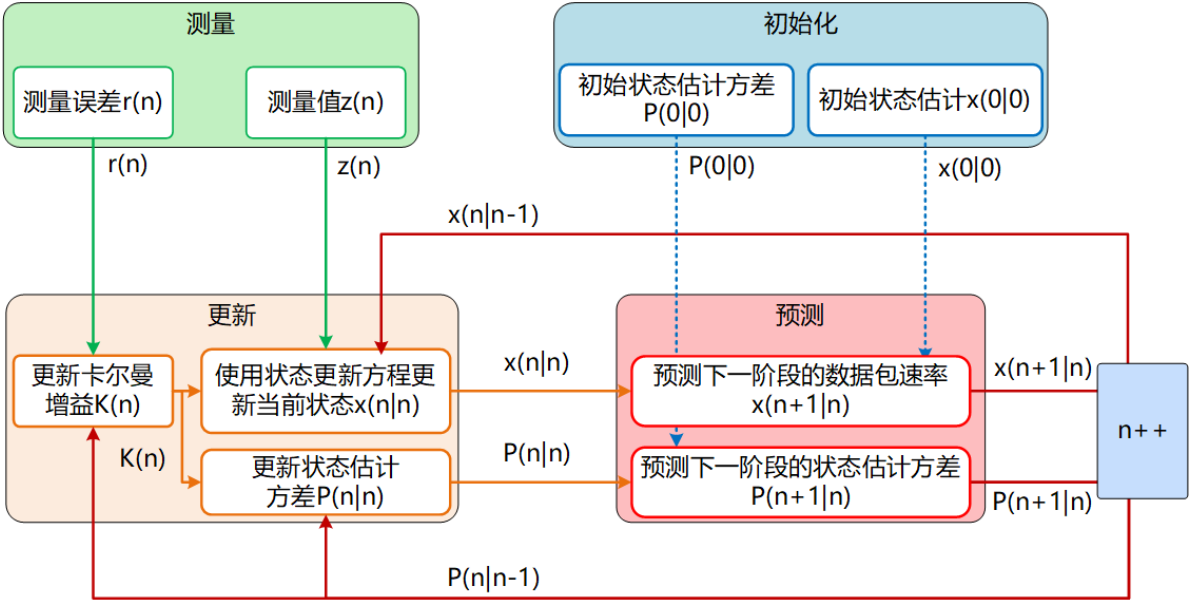


图 3-4 基于一维卡尔曼滤波的数据包速率预测方法图

首先，本文定义一维卡尔曼滤波的相关变量，如表 3-1 所示：

表 3-1 基于一维卡尔曼滤波的数据包速率预测方法变量定义

变量	说明
$x(n)$	在第 n 次收包-休眠循环中的状态估计，即预测的数据包速率
$z(n)$	在第 n 次收包-休眠循环中的测量值，即测量的数据包速率
$P(n)$	状态估计误差的协方差
Q	过程噪声，反映了模型的不确定性
R	测量噪声，反映了测量过程中的误差
$K(n)$	卡尔曼增益，用于权衡预测值和测量值

在 DPDK 轮询批处理收包的场景下，我们无法直接观测到精确的数据包速率 $x(n)$ ，而是通过计算每次轮询收包-休眠循环中的收包数作为观测值 $z(n)$ ：

$$z(n) = \frac{\sum_{i=1}^{N_n} B_i}{T_n} \quad (3-10)$$

其中， N_n 表示在 DPDK 轮询收包过程中，第 $n-1$ 次到第 n 次轮询收包-休眠循环之间的轮询次数； B_i 表示第 i 次轮询批处理过程中的收包数，且 B_i 满足：

$$0 \leq B_i \leq B \quad (3-11)$$

B 为 DPDK 的最大批处理大小，表示 DPDK 在一次轮询过程中所能处理的最大包数。

T_n 为第 $n-1$ 次休眠到第 n 次休眠之间的时间间隔，更进一步地可以表示为：

$$T_n = \frac{Cycles_n}{F} \quad (3-12)$$

其中， $Cycles_n$ 表示两次休眠之间的 CPU 运行周期数， F 表示 CPU 的频率。因此，有：

$$z(n) = \frac{\sum_{i=1}^{N_n} B_i}{Cycles_n} \times F \quad (3-13)$$

在预测阶段，定义 $x(n|n-1)$ 为第 $n-1$ 时刻对第 n 时刻的预测状态估计， $P(n|n-1)$ 为 $x(n|n-1)$ 的状态估计误差协方差，则：

$$x(n|n-1) = x(n-1|n-1) \quad (3-14)$$

$$P(n|n-1) = P(n-1|n-1) + Q$$

其中， $x(n-1|n-1)$ 表示第 $n-1$ 时刻的最优状态估计， $P(n-1|n-1)$ 表示 $x(n-1|n-1)$ 的状态估计误差协方差， Q 表示过程噪声。

最后，在更新阶段，计算卡尔曼增益 $K(n)$ ，并更新状态估计 $x(n|n)$ 与状态估计误差协方差 $P(n|n)$ ：

$$K(n) = \frac{P(n|n-1)}{P(n|n-1) + R}$$

$$x(n|n) = x(n|n-1) + K(n) * (z(n) - x(n|n-1)) \quad (3-15)$$

$$P(n|n) = P(n|n-1) * (1 - K(n))$$

其中， $K(n)$ 表示第 n 时刻的卡尔曼增益， $x(n|n)$ 表示第 n 时刻的最优状态估计， $P(n|n)$ 表示 $x(n|n)$ 的状态估计误差协方差， R 表示测量噪声。

因此，最后得到的最优状态估计即为 $x(n|n)$ ：

$$x(n|n) = x(n|n-1) + K(n) * \left(\frac{\sum_{i=1}^{N_n} B_i}{Cycles_n} \times F - x(n|n-1) \right) \quad (3-16)$$

在 DPDK 轮询批处理过程中，本章通过监测每次轮询收包-休眠循环周期内接收到的数据包数量作为观测值 $z(n)$ ，然后应用上述一维卡尔曼滤波算法来预测下一次循环周期内的数据

包速率，这种方法可以动态地调整预测，以适应网络流量的变化，从而优化资源分配和性能表现。基于一维卡尔曼滤波的数据包速率预测方法的整体流程如算法 3-1 所示：

算法 3-1 基于一维卡尔曼滤波的数据包速率预测方法

输入：数据包流

输出：预测的数据包速率 x

```

1:  initialize  $x, z, P, Q, R, K$ 
2:   $F \leftarrow \text{rte\_get\_tsc\_hz}()$ 
3:   $\text{last\_time} \leftarrow \text{rte\_get\_tsc\_cycles}()$ 
4:  while true do
5:       $\text{nb\_pkts} \leftarrow \text{rte\_eth\_rx\_burst}(\text{port\_id}, \text{queue\_id}, \text{mbufs}, \text{MAX\_PKT\_BURST})$ 
6:      if  $\text{nb\_pkts} > 0$  then
7:           $\text{sum\_pkts} \leftarrow \text{sum\_pkts} + \text{nb\_pkts}$ 
8:      else then
9:           $\text{current\_time} \leftarrow \text{rte\_get\_tsc\_cycles}()$ 
10:          $z \leftarrow \text{sum\_pkts} / (\text{current\_time} - \text{last\_time}) \times F$ 
11:          $x \leftarrow x$ 
12:          $P \leftarrow P + Q$ 
13:          $K \leftarrow P / (P + R)$ 
14:          $x \leftarrow x + K \times (z - x)$ 
15:          $P \leftarrow P * (1 - K)$ 
16:          $\text{last\_time} \leftarrow \text{rte\_get\_tsc\_cycles}()$ 
17:          $\text{sum\_pkts} \leftarrow 0$ 
18:     end if
19: end while
20: return  $x$ 

```

在算法 3-1 中，首先对一维卡尔曼滤波的相关变量进行初始化（第 1 行），紧接着获取 CPU 频率与开始时间（第 2-3 行），随后算法进入 DPDK 轮询收包流程中，通过调用 `rte_eth_rx_burst()`，CPU 可以从 RX 接收队列获取一批数据包（第 5 行），并判断是否含有数据包。如果存在数据包，则将本次轮询收包-休眠循环中的收包数进行累计（第 6-7 行）；如果不存在数据包，则需要对数据包速率进行预测。算法首先计算本次轮询收包-休眠循环中的数据包速率测量值（第 9-10 行），紧接着计算预测状态估计以及状态估计误差协方差（第 11-12 行），随后更新卡尔曼增益、最优状态估计以及状态估计误差协方差（13-15），最后算法更新当前 CPU 周期数并将累计包数归零（第 16-17 行）。在基于一维卡尔曼滤波的数据包速率预测方法中，算法仅需在每次空轮询后对数据包速率进行预测与更新，因此本算法的时间复杂度为 $O(1)$ ，同时由于一维卡尔曼滤波只需维持状态变量与卡尔曼增益系数，因此空间复杂度也为 $O(1)$ 。

基于一维卡尔曼滤波的数据包速率预测方法综合考虑了数据包速率的随机波动性与 DPDK 轮询批处理收包机制的特点，能够较好地捕捉网络流量的动态变化趋势。此外，通过调整过程噪声方差 Q 和观测噪声方差 R ，本方法还可以平衡模型对流量的突变性变化和缓慢变化的响应能力。

3.4 基于 PID 控制器的自适应轮询调度方法

为了解决 DPDK 轮询模式下, CPU 持续的高负载问题, 本章提出了基于 PID 控制器的自适应轮询调度方法。首先介绍分析了 Linux 与 DPDK 中常用的休眠函数, 然后对 DPDK 自适应轮询调度流程进行建模分析, 最后通过 PID 控制器, 根据 RX 接收队列的期望负载长度与实际负载长度之间的差距反馈, 对休眠时长进行动态调整。

3.4.1 Linux 与 DPDK 的休眠函数

在 Linux 内核及 DPDK 中, 部分休眠函数实现之间的调用关系如图 3-5 所示。

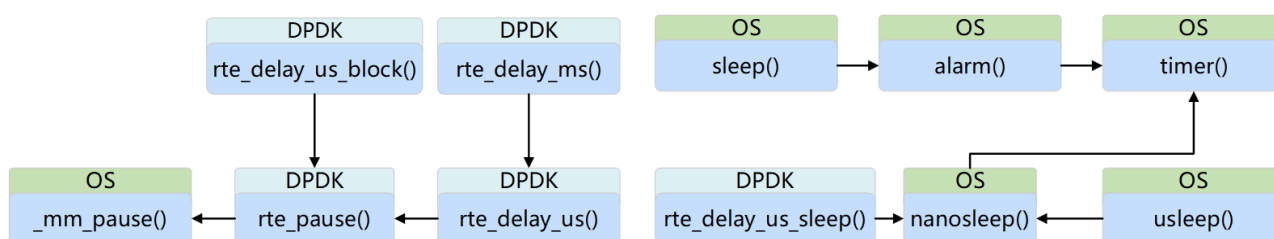


图 3-5 Linux 与 DPDK 的休眠函数关系图

Linux 操作系统为 `pthread` 线程模型提供了多种休眠函数, 不仅广泛应用于程序执行的时间控制中, 更在多任务处理和系统资源管理中起到了至关重要的作用, 主要的延迟函数包括: `sleep()`、`usleep()`与 `nanosleep()`等。

`sleep()`函数用于实现秒级精度的延迟, 通常用在对时间精确度要求不高的场合, 例如使用 `sleep()`函数来等待长时间任务的完成。然而, 这个函数的缺点是精度受限于系统的时钟频率, 且在延迟期间无法响应信号, 这在某些需要快速响应的应用场景中是不可接受的。

`usleep()`函数提供了更高的时间精度, 可以实现微秒级的延迟, 适合于需要较短暂等待的场合。相较于 `sleep()`, `usleep()`允许程序执行非常小的休眠周期, 不过由于该函数在休眠较长时间时会干扰信号处理, 它在最新的 POSIX 标准中已不推荐使用, 被 `nanosleep()`替代。

`nanosleep()`函数是一种高精度的休眠调用, 提供了比传统的休眠函数更高的时间精度, 它允许程序在纳秒级别上暂停执行, 可以用于需要精确时间控制的场合, 如实时系统、高性能计算任务或者其他需要精确控制程序休眠时间的应用。这个函数的原理基于系统调用, 它通过指定一个时间间隔来挂起当前线程的执行, 直到所设定的时间间隔消逝或线程接收到一个信号并触发信号处理函数或进程结束。`nanosleep()`可以确保程序在指定的时间内被挂起, 而不会受到系统调度的影响, 这对于需要保证操作精确同步的应用程序来说至关重要。即使如此, 由于 Linux 内核时钟的实现方式, `nanosleep()`在实际中通常无法达到理论的纳秒级精度。

同时, DPDK 作为专门针对数据包处理优化的开发工具包, 也提供了一系列用于线程休眠与延迟的函数, 如 `rte_pause()`、`rte_delay_us()`、`rte_delay_ms()`、`rte_delay_us_block()`和 `rte_delay_us_sleep()`等。

*rte_pause()*函数通过 CPU 的 *pause* 指令以极小的性能开销实现了忙等待，避免了线程状态切换的开销，从而降低了延迟，适用于那些需要快速响应的场景。然而，由于该函数并不会让出 CPU 时间片，所以 *rte_pause()*并不会降低 CPU 使用率。

rte_delay_us()、*rte_delay_us_block()*和 *rte_delay_ms()*则采用了微调系统定时器的方法，在用户态提供微秒级和毫秒级的延迟。这三个函数通过精确计算经过的 CPU 时钟周期数来实现延迟，同样它们也并不让出 CPU 时间片，这使得它们在延迟精度上相对较高，但在 CPU 资源利用率上不如其余休眠函数高效。

*rte_delay_us_sleep()*函数对 Linux 中的 *nanosleep()*函数进行了封装，这意味着它依赖于操作系统的调度器，因此可能会涉及上下文切换，其优点是能更有效地让出 CPU 时间片以降低 CPU 使用率，但可能会因操作系统的调度策略而使得延迟精度受到影响。

最后，Faltelli 等^[36]提出 *hr_sleep()*函数，该函数利用高精度计时器（*hrtimer*）机制来精确控制休眠时间长度，确保内核线程能够在预设时间内准确地进入和退出休眠状态。*hr_sleep()*函数通过修改系统调用表来添加新的系统调用接口，使用内核探针技术（*kprobe*）精确定位并访问系统调用表的位置。在修改过程中，它会暂时移除内存写保护以便写入新内容，并在操作完成后迅速恢复保护状态，以此确保系统的安全性不被破坏。*hr_sleep()*函数实现了纳秒级的时间控制精度，并对调用过程中可能遇到的错误和系统中断进行了全面的考虑与处理，从而确保了 *hr_sleep()*函数的稳定性与健壮性。各个休眠函数的对比如表 3-2 所示。

表 3-2 Linux 与 DPDK 的休眠函数

函数来源	函数名称	精确度	是否释放 CPU 资源
Linux	<i>sleep()</i>	秒级	是
	<i>usleep()</i>	微妙级	是
	<i>nanosleep()</i>	纳秒级（理论）	是
DPDK	<i>rte_pause()</i>	纳秒级	否
	<i>rte_delay_ms()</i>	毫秒级	否
	<i>rte_delay_us()</i>	微秒级	否
	<i>rte_delay_us_block()</i>	微秒级	否
	<i>rte_delay_us_sleep()</i>	微秒级	是
Faltelli 等	<i>hr_sleep()</i>	纳秒级	是

3.4.2 基于 PID 控制器的自适应轮询调度方法

PID 控制器是一种经典而有效的控制算法，因其结构简单、鲁棒性强以及易于实现而被广泛应用于闭环控制系统中，在控制器中扮演着核心角色。PID 控制器的原理基于三种基本的控制策略：比例（Proportional）、积分（Integral）和微分（Derivative），它们分别对应目前误差、过去累计误差及未来误差，共同作用于控制系统以实现期望的输出响应。

比例控制是 PID 控制器的基础，其工作原理是产生一个与当前误差成比例的控制信号。这种控制方式简单直接，能够快速减小系统的稳态误差，但单独使用时无法完全消除稳态误差。积分控制通过累积误差随时间的积分来调整控制效果，有效地解决了长期累积误差的问

题。通过引入积分控制，能够消除系统的稳态误差，使得系统的输出最终能达到我们的设定值，一个简单的比例控制系统会导致系统在设定值周围振荡，因为它无法消除过度的矫正，通过添加了一个负的平均误差值，系统的平均误差会逐渐降低，并最终使 PID 闭环系统在设定值附近趋于稳定。微分控制则关注误差的变化率，即误差的微分，微分控制有助于预测系统的未来行为，通过减少或消除系统的超调和振荡，提高系统的动态性能。

在 PID 控制器中，首先，需要获取对象当前的实际输出值 $y(t)$ 与预期的理想输出值 $r(t)$ ，并计算出二者的偏差 $e(t)$ 。

$$e(t) = r(t) - y(t) \quad (3-17)$$

偏差 $e(t)$ 便是作为 PID 控制器的输入信号。随后，PID 控制器根据输入的偏差 $e(t)$ 分别计算比例、积分与微分三个控制量，并对这三个量进行加权叠加，即：

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{d}{dt} e(t) \quad (3-18)$$

其中， K_p 为比例增益， K_i 为积分增益， K_d 为微分增益，这三个参数的值直接影响了 PID 控制器的动态性能与稳态性能。

PID 控制器通过实时收集系统输出与期望设定值之间的误差，依次对其进行比例、积分与微分进行计算，并将这三个作用的加权和作为控制输入，以实现系统动态行为的精确调节。

但是在 DPDK 轮询收包过程中使用 PID 控制器，需要对其进行离散化，一阶微分可以使用后向有限差分表示：

$$\frac{de(t_k)}{dt} = \frac{e(t_k) - e(t_{k-1})}{\Delta t} \quad (3-19)$$

积分项也可以离散化，若取样时间为 Δt ，则积分项的离散化可以使用公式 3-20 来近似：

$$\int_0^{t_k} e(\tau) d\tau = \sum_{i=1}^k e(t_i) \Delta t \quad (3-20)$$

因此，离散化的 PID 控制器算法可以表达为：

$$u(t_k) = K_p e(t_k) + K_i \sum_{i=1}^k e(t_i) \Delta t + K_d \frac{e(t_k) - e(t_{k-1})}{\Delta t} \quad (3-21)$$

在 DPDK 轮询收包过程中，本章定义 t_k 表示第 k 次收包-休眠循环，并用 Δt 表示连续两次休眠之间的时间间隔，则基于 PID 控制器的自适应轮询调度方法如图 3-6 所示，本方法旨在精准调节轮询收包-休眠周期以优化轮询调度效率，平衡 CPU 功耗与性能。

假设在 DPDK 中，配置的 RX 接收队列长度为 Q ，考虑到丢包率与包处理延时，本文定义 RX 接收队列负载因子 $\alpha (0 < \alpha \leq 1)$ ，表示在 Δt 时间段内 RX 接收队列的期望负载比例，具体来说，它反映了在给定的时间间隔内，RX 接收队列期望达到的最大长度占比。因此，基于 PID 控制器的自适应轮询调度方法的核心在于维持 RX 接收队列长度在理想的状态，为此本文定义预期的理想值 $r(t_k)$ 为 αQ ，即在第 k 次收包-休眠周期内，RX 接收队列应持有的理想数

据包数，同时本文定义当前的实际值 $y(t_k)$ ，表示第 k 次轮询收包-休眠周期内的实际收包数，如公式 3-22 所示：

$$y(t_k) = \sum_{i=1}^{M_k} B_i \quad (3-22)$$

其中， M_k 表示为第 k 次轮询收包-休眠循环中的轮询次数，即在此期间调用 `rte_eth_rx_burst()` 函数的次数。 B_i 表示第 i 次轮询批处理的收包数。

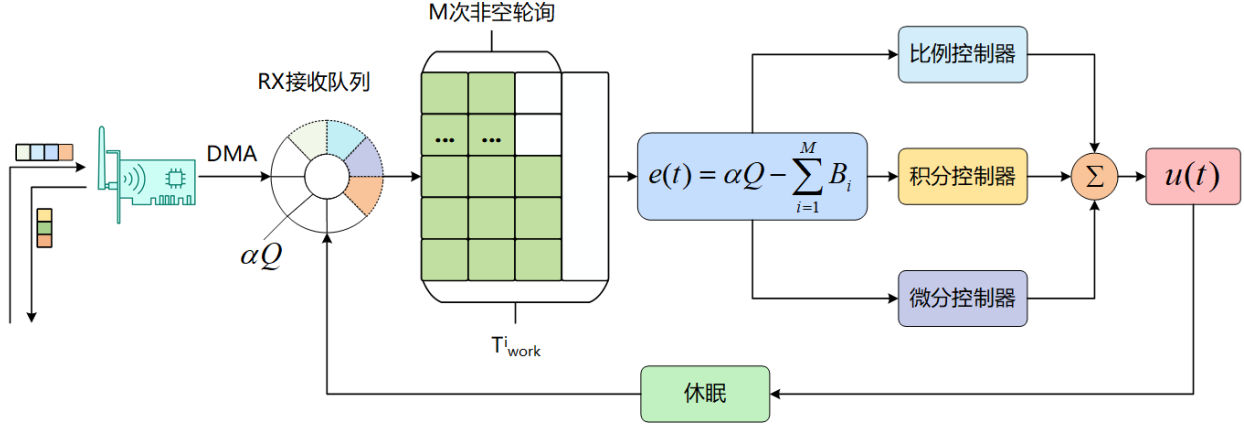


图 3-6 基于 PID 控制器的自适应轮询调度方法图

随后，计算理想值 $r(t_k)$ 与实际值 $y(t_k)$ 之间的偏差 $e(t_k)$ ：

$$e(t_k) = r(t_k) - y(t_k) = \alpha Q - \sum_{i=1}^{M_k} B_i \quad (3-23)$$

因此，结合离散化的 PID 控制器算法， $u(t_k)$ 可以表示为：

$$u(t_k) = K_p \left(\alpha Q - \sum_{i=1}^{M_k} B_i \right) + K_i \left(\alpha Q - \sum_{i=1}^{M_k} B_i \right) \Delta t + K_d \frac{-\sum_{i=1}^{M_k} B_i + \sum_{i=1}^{M_{k-1}} B_i}{\Delta t} \quad (3-24)$$

综上，考虑网络流量的动态性，本文在公式 3-9 的基础上结合基于一维卡尔曼滤波算法对流量数据包速率 P^k 进行实时预测，并使用 PID 控制器对休眠时间 T_{delay}^k 进行动态调整：

$$T_{delay}^k = \frac{\alpha Q + u(t_k)}{P^k} - \sum_{i=1}^{M_k} (T_{rx} + T_{handle} \times B_i) - T_{overhead} \quad (3-25)$$

本文结合了一维卡尔曼滤波算法和 PID 控制器，在保证 CPU 数据包处理效率的同时，对网络流量的实时变化进行动态调整，算法的整体流程如算法 3-2 所示，首先初始化 PID 控制器与 RX 接收队列的相关参数并获取 CPU 频率（第 1-3 行），紧接着进入 DPDK 轮询收包循环中，通过 `rte_eth_rx_burst()` 函数获取一批数据包并更新轮询次数，随后判断数据包数量是否大于 0，如果大于 0 则累计本次收包-休眠循环中的收包数（第 5-8 行），否则基于 PID 控制器计算队列长度控制量 u ，并更新本次的休眠时长，最后使用休眠函数进行休眠，调节 CPU 的

使用率与功耗。在本方法中, 仅需维护偏差 e 与控制量 u 等少量参数变量, 因此本算法的时间复杂度与空间复杂度均为 $O(1)$ 。

算法 3-2 基于 PID 控制器的自适应轮询调度方法

输入: 数据流, 预测的数据包速率 p^k

输出: 休眠时长

```

1:  initialize  $K_p, K_i, K_d, e, e_{sum}, e_{pre}, u$ 
2:  initialize  $\alpha, Q$ 
3:   $F \leftarrow \text{rte\_get\_tsc\_hz}()$ 
4:  while true do
5:       $nb\_pkts \leftarrow \text{rte\_eth\_rx\_burst}(\text{port\_id}, \text{queue\_id}, \text{mbufs}, \text{MAX\_PKT\_BURST})$ 
6:       $sum\_burst \leftarrow sum\_burst + 1$ 
7:      if  $nb\_pkts > 0$  then
8:           $sum\_pkts \leftarrow sum\_pkts + nb\_pkts$ 
9:      else then
10:         // 基于一维卡尔曼滤波算法的数据包速率预测, 见算法 3-1
11:          $e \leftarrow \alpha Q - sum\_pkts$ 
12:          $e_{sum} \leftarrow e_{sum} + e$ 
13:          $u \leftarrow K_p \times e + K_i \times e_{sum} + K_d \times (e - e_{pre})$ 
14:          $e_{pre} \leftarrow e$ 
15:          $delay \leftarrow \frac{\alpha Q + u}{p^k} - sum\_burst \times T_{rx} - sum\_pkts \times T_{handle} - T_{overhead}$ 
16:          $\text{sleep}(delay)$ 
17:     end if
18: end while
19: return  $delay$ 

```

3.5 实验与分析

3.5.1 实验与环境

本实验在两台通用服务器上分别部署了一套基于 DPDK 的收包、发包系统, 两台服务器的硬件配置与软件配置与表 3-3 所示。

表 3-3 基于 DPDK 的收发包系统的软硬件配置

配置名称	收包机	发包机
CPU	Intel(R) Xeon(R) Silver 4210R @ 2.40GHz 40 核	Intel(R) Xeon(R) Silver 4210R @ 2.40GHz 40 核
NUMA 节点	2	2
网卡	Ethernet 10G 2P X520 Adapter	Ethernet 10G 2P X520 Adapter
内存大小	251GB	251GB
操作系统	Ubuntu 20.04.6 LTS	Ubuntu 20.04.4 LTS
内核版本	5.4.0-169-generic	5.4.0-100-generic
DPDK 版本	23.11	23.11
GCC 版本	9.4.0	11.2.0

同时，在 DPDK 程序中的队列长度、批处理大小等参数设置如表 3-4 所示：

表 3-4 DPDK 程序队列长度与批处理大小等参数配置

参数名	参数说明	参数值
RX_QUEUE	收包队列的长度	4096
NB_MBUF	内存池中的 mbuf 数量	16384
MBUF_CACHE_SIZE	mbuf 缓存大小，用于快速分配	512
BATCH_SIZE	批处理的大小，表示 DPDK 单次最大收包数	32

3.5.2 数据集构建

本实验使用光纤将发包端与收包端相连。在发包端，本实验基于 Pktgen 在不同发包速率下对来自 CERNET 的流量数据集进行重放，并发送到收包端的网卡端口；在收包端测试本实验提出的方法，如图 3-7 所示。

CERNET 是中国教育和科研网络 CERNET 的骨干节点之一，位于南京。它负责管理与运营该地区的 CERNET 网络，为高校、科研机构等提供互联网接入服务。作为国内重要的网络节点，CERNET 不仅承担着为教育和科研单位提供网络服务的重任，同时也积累了大量的网络流量数据，这些数据对于研究网络行为、优化网络性能、发现网络异常等方面具有重要的价值。

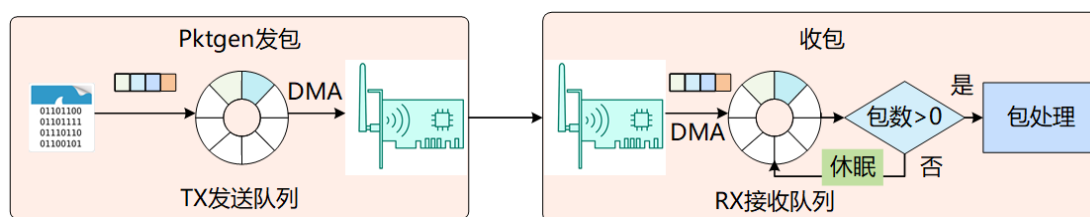


图 3-7 基于 Pktgen 的收发包实验架构图

3.5.3 对比实验设置

为了验证方法的有效性，本文首先比较 *nanosleep()*、*rte_delay_us_sleep()*、*rte_delay_us()*、与 *hr_sleep()* 函数在不同休眠时长下的延时；随后，比较基于时间滑动窗口、基于指数加权移动平均（Exponentially Weighted Moving Average, EWMA）与基于一维卡尔曼滤波的数据包速率预测方法在预测数据包速率时的 MASE（Mean Absolute Scaled Error）；最后，比较原生 DPDK、Nap^[34]与本方法的 CPU 使用率与丢包率等指标。

3.5.4 评价指标

在评估数据包速率预测模型时，本文采用 MASE 指标来评价预测的准确性。MASE 即平均绝对尺度误差，是评估时间序列预测模型性能的一种常用指标，其计算公式如公式 3-26 所示：

$$MASE = \frac{\frac{1}{n} \sum_{i=1}^n |F_t - A_t|}{\frac{1}{n-1} \sum_{t=2}^n |A_t - A_{t-1}|} \quad (3-26)$$

其中， F_t 是第 t 个时间点的预测值， A_t 是第 t 个时间点的实际值， n 是预测周期的数量。

在评估 DPDK 自适应休眠策略时, 本文使用 CPU 占用率、功耗、丢包率等指标来评估, 同时定义轮询非空流 (Polling Non-Empty Ratio, PNER), 表示在 DPDK 轮询收包过程中, `rte_eth_rx_burst()` 返回值大于 0 的轮询次数的占比, 其计算公式如公式 3-27 所示:

$$PNER = 1 - \frac{P_{empty}}{P_{total}} \quad (3-27)$$

其中, P_{empty} 表示返回包数为 0 的轮询次数, P_{total} 表示轮询总次数。

3.5.5 实验结果

首先, 我们比较了 `nanosleep()`、`clock_nanosleep()`、`rte_delay_us_sleep()`、`hr_sleep()`、`rte_delay_us_block()` 与 `rte_delay_us()` 函数在不同休眠时长下的延迟, 结果如图 3-8 所示:

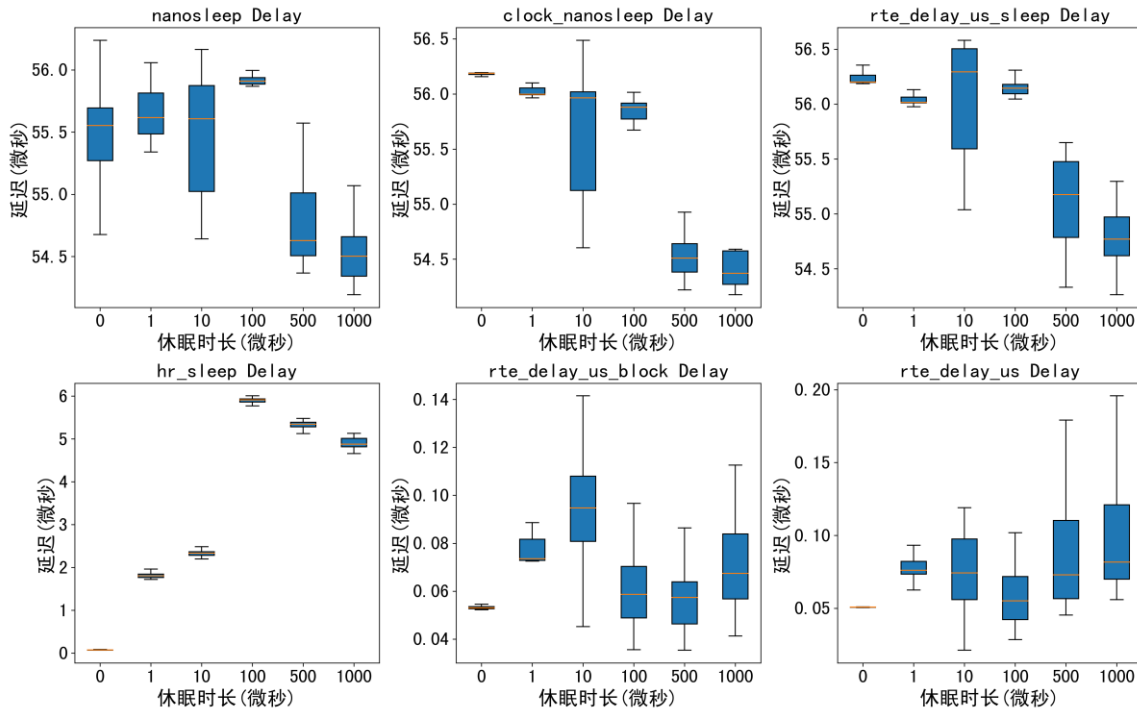


图 3-8 六种休眠函数延迟图

同时, 本文也分别测量了上述休眠函数在不同休眠时长下的 CPU 利用率, 如表 3-5 所示:

表 3-5 六种休眠函数的 CPU 利用率

	0 μs	1 μs	10 μs	100 μs	500 μs	1000 μs
<code>nanosleep()</code>	8.9	8.6	7.6	3.3	1.0	0.7
<code>clock_nanosleep()</code>	8.9	8.6	7.5	3.3	1.0	0.7
<code>rte_delay_us_sleep()</code>	8.6	8.5	7.3	3.2	1.0	0.7
<code>hr_sleep()</code>	100.0	55.3	19.5	4.6	1.7	1.3
<code>rte_delay_us_block()</code>	100.0	100.0	100.0	100.0	100.0	100.0
<code>rte_delay_us()</code>	100.0	100.0	100.0	100.0	100.0	100.0

基于表 3-5, 可以看到 `rte_delay_us_block` 与 `rte_delay_us` 虽然在休眠时有着最短的延迟, 平均只有 0.08 微秒, 但是由于使用忙等待策略, 程序并未让出 CPU 时间片, 所以 CPU 的负载仍然是 100%; 而 `nanosleep`、`clock_nanosleep` 与 `rte_delay_us_sleep` 函数能够有效降低 CPU

的负载,但是有着平均 56 微秒的延迟;*hr_sleep* 则是较好地均衡了延时与 CPU 负载,在最高 6 微秒的延迟下,可以有效降低 CPU 的负载。

为了验证一维卡尔曼滤波对数据包速率预测的有效性,本文对比了基于滑动窗口、基于 EWMA 与基于一维卡尔曼滤波的数据包速率预测方法。在数据包大小为 64B 下,每当进入休眠态时使用上述三种方法对苏醒后的数据包速率进行预测,并计算其的 MASE 值,实验结果如图 3-9 所示。

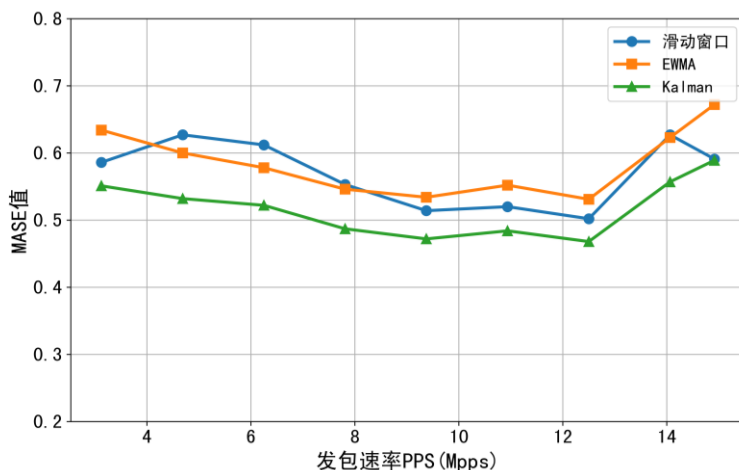


图 3-9 数据包速率预测的 MASE 对比图

从图 3-9 可以看到,在 1.56Mpps 至 14.86Mpps 的网络数据包速率预测中,基于一维卡尔曼滤波的数据包速率预测均有着最低的 MASE,证明了本方法在数据包速率预测上的有效性。

本文分别比较了原生 DPDK、基于 PID 控制器的自适应轮询调度方法、基于混合中断轮询的方法与 Nap 方法的 CPU 利用率、丢包率、轮询非空率与 CPU 功耗,如图 3-10 所示。

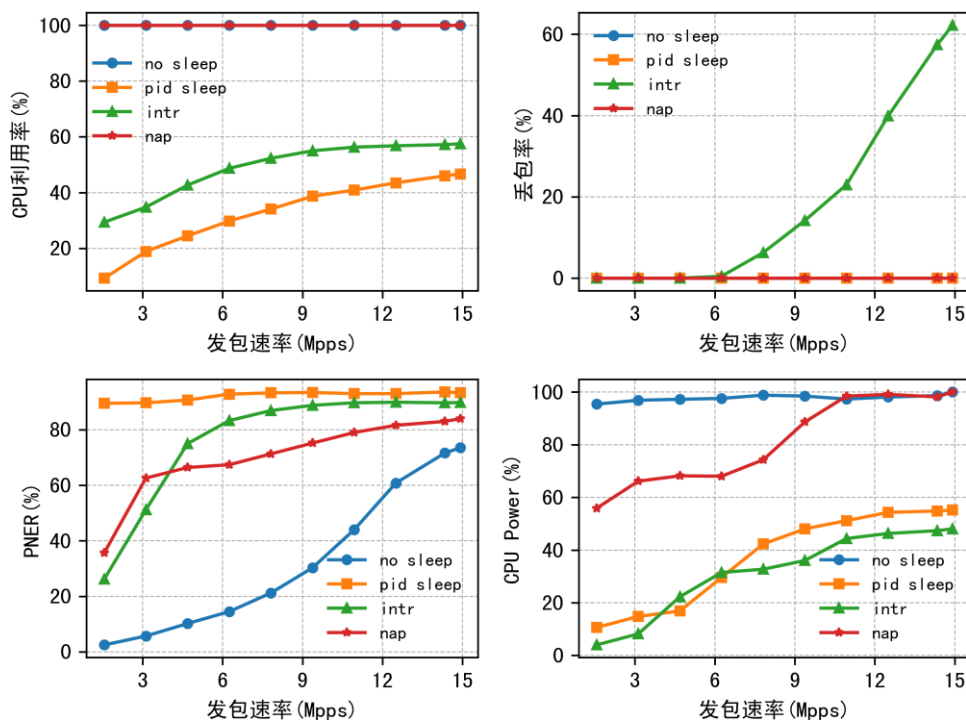


图 3-10 基于自适应包间休眠的间歇式轮询方法性能对比图

本文以 CPU 核心的满载功耗为 100%，从图 3-10 可以看到，在原生 DPDK 程序中，由于采用了轮询收包机制，CPU 的利用率与功耗都处于 100%，且有着最低的轮询非空率。而在基于混合轮询中断的方法中，虽然可以有效降低 CPU 负载，但是在数据包速率超过 6Mpps 后，其丢包率大幅增加。Nap 采用了 pause 指令，可以有效降低 CPU 的功耗，提高轮询非空率，但是其 CPU 利用率却仍然是保持在 100%。最后，本文提出的方法有着最低的 CPU 利用率与最高的轮询非空率，兼顾了 CPU 负载与收包性能。

值得注意的是，在构建非一致性存储访问（Non-Uniform Memory Access, NUMA）架构的服务器系统时，需要确保分配的 CPU 与网卡属于同一 NUMA 节点中，以防止在 CPU 与 NIC 间通信过程中穿越 NUMA 节点边界时，所可能引发的性能衰减，如图 3-11 所示。

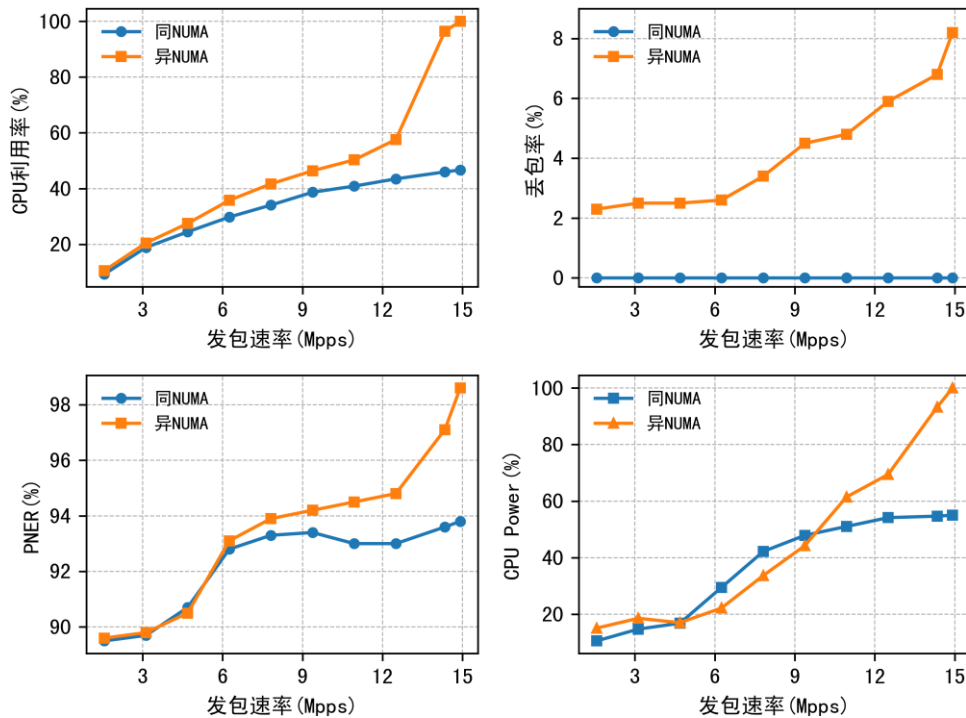


图 3-11 NUMA 性能对比图

从图 3-11 中可以看到，当发包速率到达 14Mpps 后，异 NUMA 节点的 CPU 利用率大幅增加，且存在一定的丢包率，同时有着更高的 CPU 功耗。

本文设置 DPDK 的收包队列为 4096，考虑在不同队列负载因子 α 时，CPU 利用率、丢包率与轮询非空率的变化如图 3-12 所示。

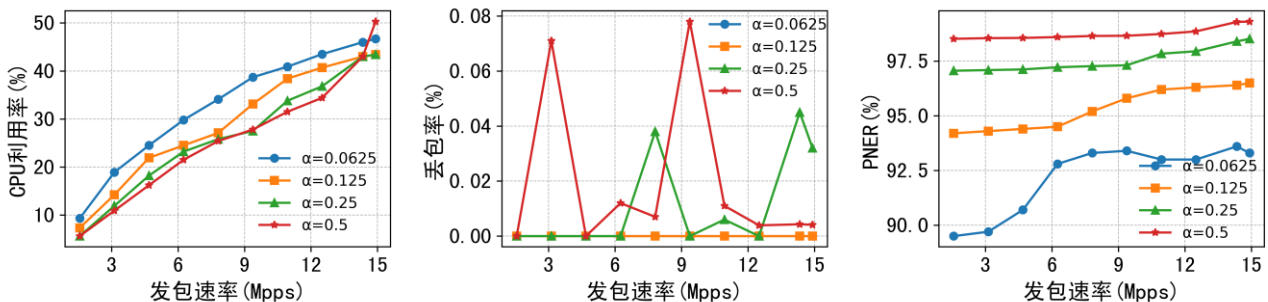


图 3-12 不同队列负载因子 α 下的性能对比图

从图 3-12 中可以看出，当队列负载因子 α 逐渐增大时，轮询非空率也会相应提升，同时 CPU 利用率也会在一定程度上有所降低，但是由于队列负载因子的增大会直接导致休眠时长的增加，因此丢包率也会增大，会产生波动。

3.6 本章小结

针对 DPDK 轮询机制下的高 CPU 负载问题，本章提出了一种基于自适应包间休眠的间歇式轮询方法，该方法利用间歇性休眠调整机制和精准的数据包速率预测相结合，有效减少了 CPU 在数据包接收过程中的能耗。在本章节中，我们首先针对 DPDK 在轮询收包过程中 CPU 时间分布进行了建模分析。随后，我们设计并实现了一种基于一维卡尔曼滤波的数据包速率预测算法，能够在每次轮询休眠前准确预测接下来的数据包速率。在此基础上，结合预测得到的数据包速率，本章进一步提出了一种基于 PID 控制理论的自适应轮询调度方法，通过 PID 控制器，根据 RX 接收队列的期望负载长度与实际负载长度之间的差距反馈，以动态调整休眠时长，实现功耗与性能之间的最优平衡。与此同时，本章还深入探讨了休眠函数选择对于延迟和 CPU 资源利用率的影响，通过实验对比分析，验证了所提出方法相较于传统 DPDK、基于混合中断轮询的 DPDK 以及 Nap 方法在 CPU 利用率、丢包率、轮询非空率及 CPU 能耗等关键性能指标上的显著优势。此外，本章还探讨了在不同队列负载系数 α 情况下，所提出方法的性能表现。

第四章 基于改进桶式 Cuckoo 哈希的高速网络测量方法

在网络测量领域，Cuckoo 哈希凭借其高效的查找性能被广泛采用，并且 DPDK 为该算法的应用提供了专门的软件开发实现。尽管如此，传统的 Cuckoo 哈希表在高速网络测量时仍面临若干性能挑战：包括多次哈希运算复杂度高、多个备选桶查询开销大、以及高负载时可能出现循环冲突等问题。为了改善这些问题，本章提出一种基于改进桶式 Cuckoo 哈希的高速网络测量方法，如图 4-1 所示。本章首先对传统 Cuckoo 哈希应用于通用计算平台上的高速细粒度网络测量时的不足进行细致剖析。紧接着，为了解决多次哈希运算造成的性能瓶颈，本章提出了一种基于对称 RSS 的软硬协同哈希方法。同时，为了提高桶式 Cuckoo 哈希的查询效率，避免不必要的内存访问，本章提出了一个基于计数布隆过滤器查找优化方法，而对于 Cuckoo 哈希在高负载率下的循环哈希冲突问题，本章设计了一个基于惰性时间衰减成本的流逐出方法。此外，本章还基于 AVX 与 SSE 指令集对元素查找与最小成本查找进行加速。最后，本章对主干网的流量数据集进行重放，表明本方法显著降低了查询与插入的操作时间，有效提升了 Cuckoo 哈希算法的整体性能表现。

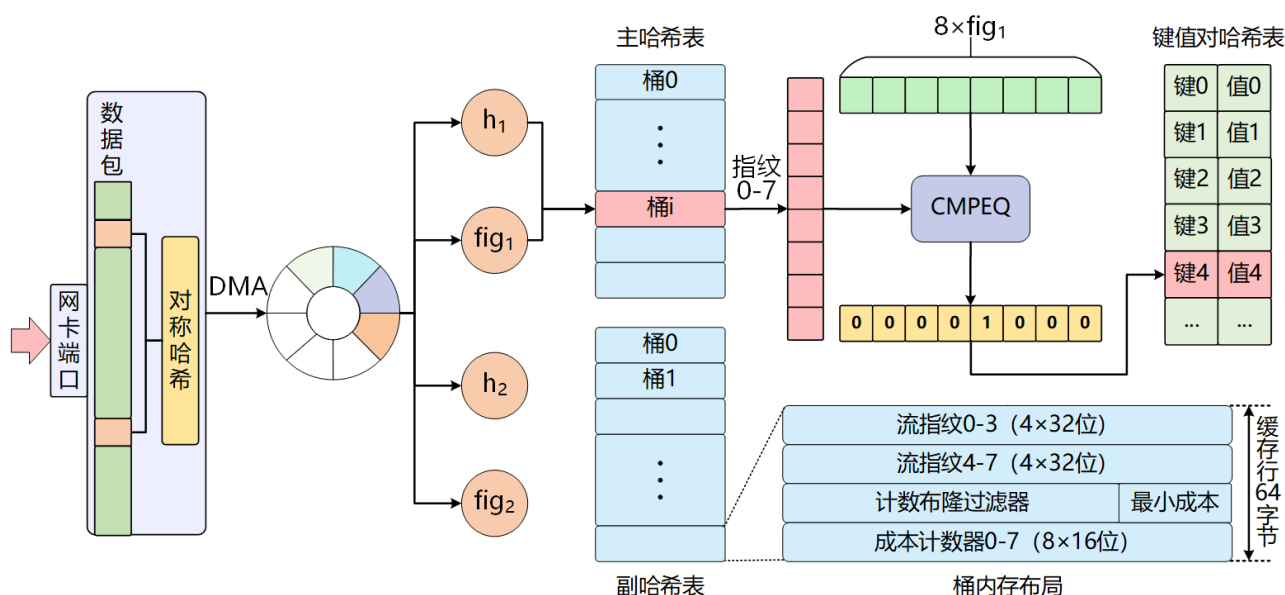


图 4-1 基于改进桶式布谷鸟哈希的高速网络测量方法架构图

4.1 问题分析

随着互联网的高速发展，设备的大规模互联导致了网络流量的激增，相应地，为了追踪测量网络连接所需要的键值对存储数量也随之快速攀升。然而，计算设备上可用于哈希表存储的内存资源往往是有限的，这使得在插入新的键值对时，哈希冲突的发生变得不可避免。Cuckoo 哈希因其有效缓解哈希冲突并优化操作性能而被广泛用于网络测量，它使用两个或多个独立的哈希函数来确定元素的多个备选存储位置，即使一个位置发生了冲突，元素仍可存储于另一位置。Cuckoo 哈希的查找操作时间复杂度可稳定维持在 $O(1)$ 的常数级别，即便在负载因子达到最大的情况下，也能保证常数级的查找与更新时间。

然而, 在 10Gbps 的高速网络环境下, 若以最小包大小 64B 进行数据传输, 理论上的数据包传输速率最高可达 14.88Mpps, 这意味着每个数据包的平均处理时间仅有 67 纳秒。在这样的高速场景下, 传统 Cuckoo 哈希的多次哈希计算往往会消耗大量 CPU 周期, 成为通用计算平台上处理高速网络流量的性能瓶颈。同时, 网络流量呈现出典型的长尾分布, 海量的小规模网络流只占据网络流量很小的一部分, 而少数的大流则占据了网络流量的绝大部分。在一定时间窗口内, 同一条大流中的数据包很可能会被多次接收, 而传统 Cuckoo 哈希并未根据网络流量的这一特性进行优化。

此外, Cuckoo 哈希虽然保证了最坏情况下常数级的查找时间和更新时间, 但对于新元素插入过程中的路径长度却无法保证。常用的踢出策略有随机漫游 (Random Walk, RW)^[56]与广度优先搜索 (Breadth First Search, BFS)。随机漫游方法以较高的概率保证了插入时间的期望值, 但无法避免在高负载率下陷入循环冲突状态, 且随机决策会在一定程度上影响踢出决策的准确性。广度优先搜索通过并行检查所有候选位置并搜索最佳路径, 可以在一定程度上避免循环, 但是随着负载率的增加, 需要搜索的空间呈指数级增长, 而且该算法实现的复杂度也过高。

因此, 亟需结合网络流量的实际特性, 针对上述问题对传统 Cuckoo 哈希算法进行改进, 以更好地服务于高速网络流量测量的应用场景。

4.2 基于对称 RSS 的软硬协同哈希方法

随着网络的快速发展, 对大数据的快速存储、查询的需求不断增加。因此, 基于哈希的数据结构在社交网络、数据库索引、数据包转发等方面得到了广泛的应用。在经典哈希表的基础上, Cuckoo 哈希通过多个哈希函数, 为每个元素提供多个候选位置, 保证在最坏情况下仍然能提供常数级的元素查询时间。但是, Cuckoo 哈希因为需要多个哈希函数, 所以有着较高的哈希计算开销, 而具有良好性能的哈希函数通常计算复杂度会较高, 这导致哈希函数的计算会消耗大量的 CPU 周期, 成为通用计算平台的性能瓶颈。

哈希函数通过将不定长的输入进行哈希运算, 输出定长的整数, 通常是 32 位或 64 位。但是因为哈希表的长度往往远小于 2^{32} 或 2^{64} , 这意味着当使用取模运算以将哈希函数的结果映射到哈希表中的地址时, 哈希结果所携带的大量位信息被丢弃。基于这一观察, 本章认为通过充分利用哈希函数产生的信息来降低 Cuckoo 哈希中的哈希计算成本是可行的, 并基于网卡硬件的对称 RSS 计算, 最大程度降低 CPU 在哈希运算上的开销。

4.2.1 Toeplitz 哈希算法

Toeplitz 哈希算法是一种通过矩阵乘法计算哈希值的方法, 它使用一个特定的 Toeplitz 矩阵来生成哈希值。Toeplitz 矩阵中主对角线上的元素相等, 平行于主对角线的线上的元素也相等。具体来说, 对于一个 m 行 n 列的矩阵 $T = [t_{ij}]$, 其中, $i \in [0, m]$, $j \in [0, n]$, 如果满足 $t_{ij} = t_{i-j}$, 则称矩阵 T 为 Toeplitz 矩阵, 如公式 4-1 所示:

$$T = \begin{bmatrix} t_0 & t_{-1} & t_{-2} & \cdots & \cdots & t_{-n+1} \\ t_1 & t_0 & t_{-1} & \ddots & & \vdots \\ t_2 & t_1 & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & t_{-1} & t_{-2} \\ \vdots & & \ddots & t_1 & t_0 & t_{-1} \\ t_{m-1} & \cdots & \cdots & t_2 & t_1 & t_0 \end{bmatrix} \quad (4-1)$$

因此, 对于一个长度为 n 的序列 x , Toeplitz 哈希将输入数据视作多项式的系数, 并与 Toeplitz 矩阵相乘, 最终得到哈希值 $hash(x)$:

$$hash(x) = T \times x = \begin{bmatrix} t_0 & t_{-1} & t_{-2} & \cdots & \cdots & t_{-n+1} \\ t_1 & t_0 & t_{-1} & \ddots & & \vdots \\ t_2 & t_1 & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & t_{-1} & t_{-2} \\ \vdots & & \ddots & t_1 & t_0 & t_{-1} \\ t_{m-1} & \cdots & \cdots & t_2 & t_1 & t_0 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{n-1} \end{bmatrix} \quad (4-2)$$

Toeplitz 矩阵的一个关键性质是, 只要确定了第一行与第一列的元素, 整个矩阵的其它元素便由此确定。因此, 存在一个长度为 $m+n-1$ 的序列 L 与 Toeplitz 矩阵 T 相对应, 即:

$$L = (t_0, t_1, t_2, \cdots, t_{m-1}, t_{-1}, t_{-2}, \cdots, t_{-n+1}) \quad (4-3)$$

因此, 在 Toeplitz 哈希算法中, 并不需要完整存储整个 Toeplitz 矩阵, 只需要将序列 L 与输入的每一位进行异或运算, 即可有效模拟 Toeplitz 矩阵乘法的过程, 并生成最终的哈希值, 整体流程如算法 4-1 所示。具体来说, 首先初始化哈希值 $hash$ 为 0 (第 1 行), 紧接着开始遍历输入数组的每一位比特, 如果当前遍历到的比特值为 1, 则将 $hash$ 与序列 L 的左 32 位比特进行异或运算, 并赋值更新 $hash$ (第 3-4 行), 随后将序列 L 左移一位 (第 6 行), 重复以上流程直至输入数组的所有比特均被遍历完成, 最终得到的哈希值 $hash$ 就是所有这些异或运算结果的累积。该算法的时间复杂度主要取决于输入数组 $input[]$ 的大小, 对于每一个比特, 都会执行一次检查, 如果比特为 1 则还需进行一次异或操作, 以及一次位移操作, 这些操作的时间复杂度都是 $O(1)$, 因此, 对于 n 比特的输入数组, 则该算法的时间复杂度为 $O(n)$ 。同时, 该算法只需要额外的常量阶空间来存储哈希值 $hash$ 与序列 L , 因此, 算法的空间复杂度为 $O(1)$ 。

算法 4-1 Toeplitz 哈希算法

输入: $input[]$, 序列 L

输出: 哈希值

```

1: initialize hash  $\leftarrow$  0
2: for each bit  $b$  in  $input[]$  do
3:     if  $b == 1$  then
4:         hash  $\wedge=$  (left-most 32 bits of  $L$ )
5:     end if
6:     shift  $L$  left 1 bit position
7: end for
8: return hash
```

在高速网络环境中，Toeplitz 哈希算法能够有效地处理大量的数据包，这对于现代网络应用中的负载均衡和流量管理至关重要。

4.2.2 对称 RSS 算法

RSS 是一种网络驱动程序技术，用于将数据包分发至不同的 CPU 核心以提升处理效率，如图 4-2 所示，网卡中含有多个 RX 接收队列，且每一个队列均被绑定至专用的 CPU 核心，当网络数据包到达网卡后，网卡首先会对数据包报文进行解析，获取数据包关键信息，如源 IP 地址、宿 IP 地址、源端口以及宿端口等信息，然后网卡基于 Toeplitz 哈希算法，计算 IP 地址、端口号等信息的哈希值，并根据哈希值与重定向表将数据包有效地分流至指定的 CPU 核心进行处理。

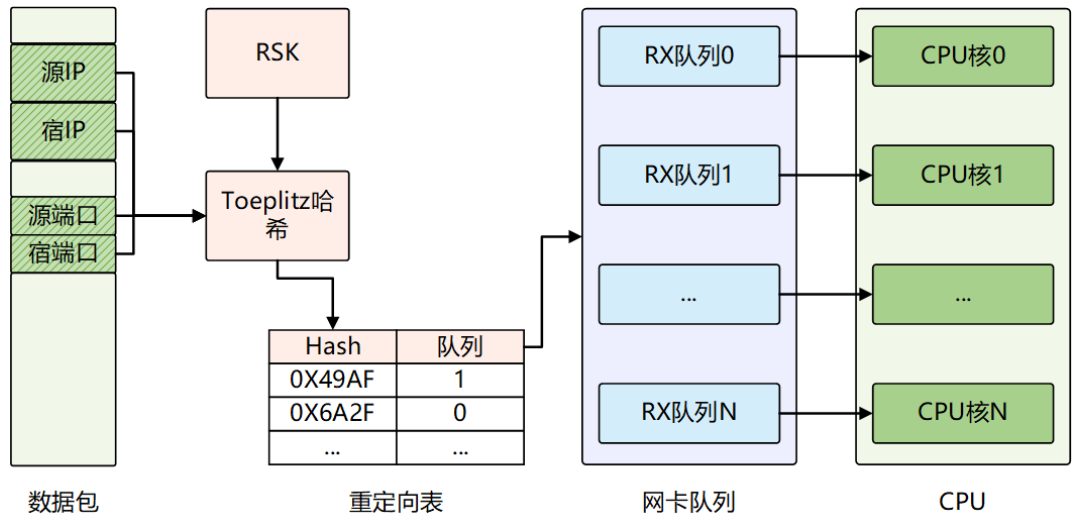


图 4-2 Toeplitz 与 RSS 图

RSS 通过针对数据包的特定字段进行哈希运算，生成对应的哈希值，并根据生成的哈希值将数据包分发至不同的 RX 接收队列中，最终交付给对应的 CPU 核心进行处理。值得注意的是，网卡一般会根据所接收数据包的类型，确定其需要参与哈希运算的报文字段，其中的主要数据包类型及其对应的哈希报文字段，如表 4-1 所示。

表 4-1 RSS 的哈希报文字段

数据包类型	哈希报文字段
IPv4 UDP	源 IP、宿 IP、源端口、宿端口
IPv4 TCP	源 IP、宿 IP、源端口、宿端口
IPv4 SCTP	源 IP、宿 IP、源端口、宿端口、验证标签
IPv4 OTHER	源 IP、宿 IP
IPv6 UDP	源 IP、宿 IP、源端口、宿端口
IPv6 TCP	源 IP、宿 IP、源端口、宿端口
IPv6 SCTP	源 IP、宿 IP、源端口、宿端口、验证标签
IPv6 OTHER	源 IP、宿 IP

RSS 采用 Toeplitz 哈希算法，算法的输入为数据包的特定字段与一个 Toeplitz 序列 RSK (Receive Side Key)。在 DPDK 中，存有一个 40 字节的默认 RSK，定义如下：

0x6d,	0x5a,	0x56,	0xda,	0x25,	0x5b,	0x0e,	0xc2,
0x41,	0x67,	0x25,	0x3d,	0x43,	0xa3,	0x8f,	0xb0,
0xd0,	0xca,	0x2b,	0xcb,	0xae,	0x7b,	0x30,	0xb4,
0x77,	0xcb,	0x2d,	0xa3,	0x80,	0x30,	0xf2,	0x0c,
0x6a,	0x42,	0xb7,	0x3b,	0xbe,	0xac,	0x01,	0xfa

RSS 通过对数据包的特定字段进行哈希来确保同一条流的数据包被分配到同一 RX 接收队列，并进一步由相同的 CPU 核心处理。但对于双向流，在基于 Toeplitz 的哈希运算中，相反方向的四元组会得到不同的哈希值，因此 RSS 会将同一会话中两条流的数据包分配至不同的 RX 队列。在网络测量中，这种分配方式会导致同一会话的数据需要在不同 CPU 核心间共享，增加了锁的使用和处理延迟。因此，Woo 等^[70]提出了一种对称 RSS 算法，通过修改原有 RSS 中的 RSK，将双向流映射到相同的 RX 队列，在对称 RSS 算法中，修改后的 RSK 如下：

0x6D,	0x5A,	0x6D,	0x5A,	0x6D,	0x5A,	0x6D,	0x5A,
0x6D,	0x5A,	0x6D,	0x5A,	0x6D,	0x5A,	0x6D,	0x5A,
0x6D,	0x5A,	0x6D,	0x5A,	0x6D,	0x5A,	0x6D,	0x5A,
0x6D,	0x5A,	0x6D,	0x5A,	0x6D,	0x5A,	0x6D,	0x5A,
0x6D,	0x5A,	0x6D,	0x5A,	0x6D,	0x5A,	0x6D,	0x5A,

采用对称 RSS 算法可以使得同一会话中的数据包在相同的 CPU 核心中进行统一处理，降低了同一会话中不同方向流的数据包在不同核上处理时出现的数据同步等额外的开销，可以有效提升基于会话的网络测量的效率。

4.2.3 基于对称 RSS 算法的软硬协同哈希方法

在对称 RSS 算法中所生成的哈希值为一个 32 位的整数，即该哈希值所能表征的范围为 $0 \sim 2^{32} - 1$ ，显然此范围远超常用哈希表的长度，假定通过 RSS 得到的哈希值为 h_{tp} ，同时设哈希表的长度为 n ，常用的处理方式是对 h_{tp} 执行取模运算，即计算 $h_{tp} \% n$ 。然而，这一处理不可避免地损失了大量的原始哈希值信息，导致 h_{tp} 的位信息未得到充分的利用。为了弥补这一不足，本章在对称 RSS 生成的哈希值 h_{tp} 之上，提出一种简单且高效的计算方法，来生成两个 t 位数 ($t < 32$) 的子哈希值。这些子哈希值将被应用于 Cuckoo 哈希表中，从而降低在高速网络测量中多次哈希运算的开销。

基于对称 RSS 算法的软硬协同哈希方法的基本思想就是通过移位与异或等运算，充分利用网卡硬件 RSS 算法生成的 32 位哈希值，进而生成 2 个 t ($t < 32$) 位的哈希值。假设基于网卡硬件 RSS 生成的哈希值为 h_{tp} ，则本章将基于公式 4-4 生成另外两个 t 位的哈希值：

$$\begin{cases} h_1 = (h_{tp} \gg 1) \& ((1 \ll t) - 1) \\ h_2 = (h_{tp} \gg 11) \wedge (h_{tp} \& (1 \ll (2t - 32))) \& ((1 \ll t) - 1) \end{cases} \quad (4-4)$$

其中， h_1 与 h_2 分别为基于 h_{tp} 生成的两个哈希值， n 为 Cuckoo 哈希表的长度。 h_1 与 h_2 本质上就是充分利用 h_{tp} 的位信息生成的另外两个哈希值。

基于对称 RSS 的软硬协同哈希方法如算法 4-2 所示，首先通过将 1 左移 t 位并减 1 来获取掩码 mask (第 2 行)，随后将 h_{tp} 向右移一位后与 mask 进行按位与操作来计算哈希值 h_1 (第 3 行)，将 h_{tp} 向右移 11 位并与原 h_{tp} 低 $2t-32$ 位进行异或运算并与 mask 进行按位与操作来计算

哈希值 h_2 。算法中只涉及按位与、左移、右移与异或操作，因此时间复杂度为 $O(1)$ ，同时算法只需要存储哈希值 h_1 、 h_2 与掩码 $mask$ ，因此算法的空间复杂度为 $O(1)$ 。

算法 4-2 基于对称 RSS 的软硬协同哈希方法

输入： 对称 RSS 的哈希值 h_{tp} ，生成的哈希位数 t ，哈希表长度 2^t

输出： h_1, h_2

```

1:  function srss_to_two_hashes( $h_{tp}, t$ )
2:       $mask \leftarrow (1 \ll t) - 1$ 
3:       $h_1 \leftarrow (h_{tp} \gg 1) \& mask$ 
4:       $h_2 \leftarrow (h_{tp} \gg 11) \wedge (h_{tp} \& (1 \ll (2t - 32))) \& mask$ 
5:      return  $h_1, h_2$ 
6:  end function

```

接下来本章将从均匀性与碰撞概率两个角度分析本方法的可行性。在均匀性上，均匀性的关键在于，对于任意不同的输入 x ，通过 $h_1(x)$ 与 $h_2(x)$ 生成的哈希值对能够均匀覆盖 $[0, n) \times [0, n)$ 的二维空间上。首先，考虑 $h_1(x)$ 与 $h_2(x)$ 各自的均匀性：由于 $h_{tp}(x)$ 本身时均匀分布的， $h_1(x)$ 与 $h_2(x)$ 分别是其低 t 位与高 t 位通过移位与异或得到的，并不会破坏其均匀性，所以 $h_1(x)$ 与 $h_2(x)$ 各自也是呈均匀分布的。在 $(h_1(x), h_2(x))$ 联合均匀性上，本章的方法通过错位异或，消除了 $h_1(x)$ 与 $h_2(x)$ 的相关性，因此 $(h_1(x), h_2(x))$ 的联合分布也是均匀的。在后续实验章节，将通过卡方检验验证其分布的均匀性。

在碰撞概率上，假设有一个包含 m 个元素的集合 S ，对于集合中的每个元素 x ，本章需要计算该元素的 k 个哈希值 $H_i(x) (1 \leq i \leq k)$ ，其中每个哈希值的范围在 0 到 $n-1$ 之间。给定一个不属于集合 S 的元素 z ，我们定义事件 A 为元素 z 与集合 S 中的元素发生碰撞，即在集合 S 中至少存在一个元素 x_j ，使得 $H_i(z) = H_i(x_j)$ 。

当使用 k 个独立的哈希函数来生成 k 个随机哈希值时，每个哈希函数都均匀地将元素映射到 0 到 $n-1$ 的范围，因此元素 z 与另一个元素在哈希函数 H_i 中发生冲突的概率为 $1/n$ ，则该元素与集合 S 中元素发生哈希冲突的概率 $P(A_i)$ 为：

$$P(A_i) = \left(1 - \left(1 - \frac{1}{n}\right)^m\right) \quad (4-5)$$

因为 k 个哈希函数都是独立的，所以元素 z 与集合 S 中的元素在所有 k 个哈希函数中都发生哈希冲突的概率为：

$$P(A) = \left(1 - \left(1 - \frac{1}{n}\right)^m\right)^k \quad (4-6)$$

$P(A)$ 是评估本章方法的重要指标，其衡量了在加入新元素 z 的过程中，该元素与集合 S 中已有成员在全部 k 个哈希函数下都发生冲突的概率。在插入元素 z 时，与集合 S 中的元素在所有 k 个哈希函数中都发生哈希冲突的概率越是接近 $P(A)$ ，则说明本方法的性能损失越低；反之则说明本方法在哈希上的性能损失越大。

4.3 基于改进桶式 Cuckoo 哈希的高速网络测量方法

在网络测量领域，经常需要追踪数以百万计的数据流，为了有效应对这一需求，数据流及其相应状态之间的映射关系需存储于哈希表中，以确保可以高效地查询数据流信息。在网络测量任务中，哈希表往往需要管理与每条连接相关的状态信息，这要求存储大量的数据条目，随之而来的是对空间的巨大需求，这远远超过了 TCAM 或 SRAM 的存储能力，因为这类芯片在容量方面通常有限制：当前市场上最大的 TCAM 仅有数十 MB，SRAM 的容量则被限制在 128MB 左右。因此，虽然 TCAM 与 SRAM 在性能方面表现卓越，但对于需要对网络流的多维特征进行细粒度测量的任务来说，运行于通用计算平台上的软件测量系统能够使用大量 DRAM，提供了一种既经济又高效的选择。

通用计算平台上的软件网络测量系统，在 DPDK 的助力下，通过内核旁路，绕过传统的内核网络协议栈，规避了成本高昂的上下文切换，并采用批处理机制分摊 I/O 代价、运用无锁队列消除同步负担并采取高效的数据结构来提升处理速度，极大提高了网络数据包的处理性能。

同时，在数据结构上，桶式 Cuckoo 哈希表，因其确保能在不到三次内存访问的情况下查询到特定数据流的状态，而被广泛应用于网络测量。在桶式 Cuckoo 哈希表中，通常将内存分割为两个固定大小的区域——主桶与副桶，每个桶内含有数个槽位，通常是 4 或 8 个，通过哈希函数将元素映射至这两个桶之中的一个。

本章的主要设计目标就是最大限度地减少访问的 CPU 缓存行的次数，因为内存访问是一种非常耗时的操作。本章将查找分为正向查找(Positive Lookup)与反向查找(Negative Lookup)。正向查找即在哈希表中找到了关键字，查找成功；负向查找是指因为关键字不在表中，从而导致查找失败。考虑一个桶式 Cuckoo 哈希表，它使用两个独立的散列函数将每个元素映射到两个候选存储桶之一，在正向查找时，因为一半的元素可以通过检查一个存储桶来检索，而另一半则需要依次访问这两个存储桶才能找到，因此平均将需要查询 1.5 个存储桶。而对于负向查找，则必然需要访问两个存储桶才能确定待查找的元素不存在表中。因此本章的主要优化思路就是尽量将常被访问的流放在主桶中，同时能尽快响应查找失败的情形，避免不必要的桶访问。

4.3.1 桶单元的内存布局

相比于已加载至 CPU 缓存行中的数据，访问未缓存的数据将产生更大的开销与延迟，在 Intel Xeon 处理器上，一级缓存的访问延迟是 4-5 个 CPU 周期，而当数据不在缓存中时，访问延迟甚至可以达到 150-300 个周期。因此，在网络测量时最大限度地减少访问的缓存行的次数是至关重要的。为适应现代 CPU 缓存结构，鉴于 CPU 缓存行大小普遍为 64 字节，而 IPv4 流的四元组关键字通常由两个 32 位 IP 地址与两个 16 位端口号构成，共占用 12 字节，这意味着当一个桶内含有 8 个槽位时，同一桶内的所有元素不可能被一次性完整地加载至 CPU 缓存行中，从而导致内存访问延迟的显著增长。因此，本章为了桶式 Cuckoo 哈希表的

每个桶可以被一次性加载至 CPU 缓存行内,将桶式 Cuckoo 哈希表作为键值对哈希表的索引,如图 4-3 所示,在流的查询过程中,首先对查询的键进行哈希,访问两个关联的桶,并将查询哈希与存储在桶中的指纹进行比较,如果存在匹配的指纹,则继续访问键值对哈希表中的目标流条目。由于哈希冲突的存在,匹配的指纹可能是假阳性,因此需要将流的四元组信息进一步与键值对哈希表中存储的四元组进行比较,如果匹配,则查找成功,并返回目标流的特征信息;如果没有匹配的键,则查找失败。因此,在桶式 Cuckoo 哈希的查询中,至多需要 3 次内存访问(即,主桶、副桶与键值对哈希表)来响应查找。

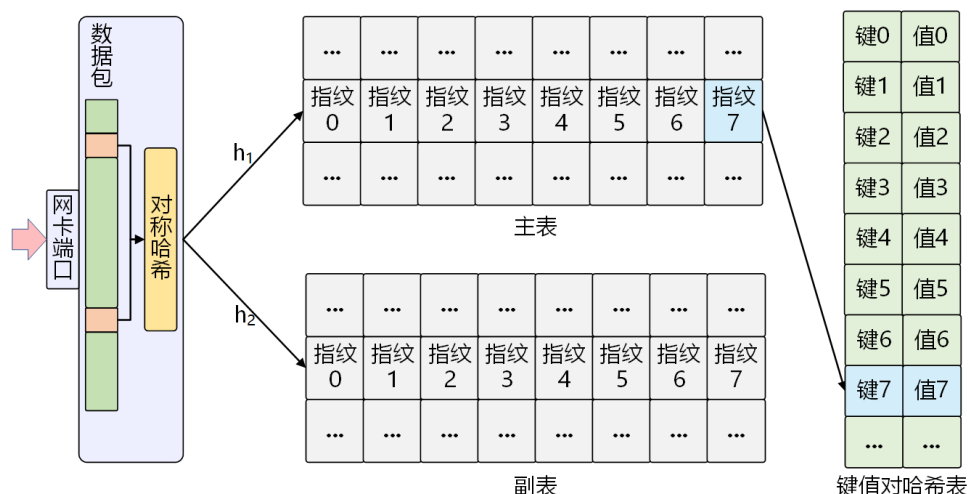


图 4-3 基于 Cuckoo 哈希表的键值对哈希表索引图

本文进一步对桶式 Cuckoo 哈希的桶单元结构进行了组织,以便在查找存储桶时访问的所有字段都位于同一缓存行中,如图 4-4 所示。图 4-4(a)显示了 DPDK 桶单元的内存布局,DPDK 在每个存储桶中存储了 8 个 32 位主哈希与 8 个 32 位索引,这些索引记录了流条目在键值对哈希表中的位置,此外,DPDK 还存储 8 个 32 位副哈希值。本章采用了如图 4-4(b)所示的内存布局,在每个存储桶中存储了 8 个 32 位流指纹,一个 96 位的计数布隆过滤器,其中每 3 位作为一个计数器,用于记录副桶中元素的个数,以及 8 个 16 位的流成本计数器与一个记录了当前最小成本流所在槽的位置。

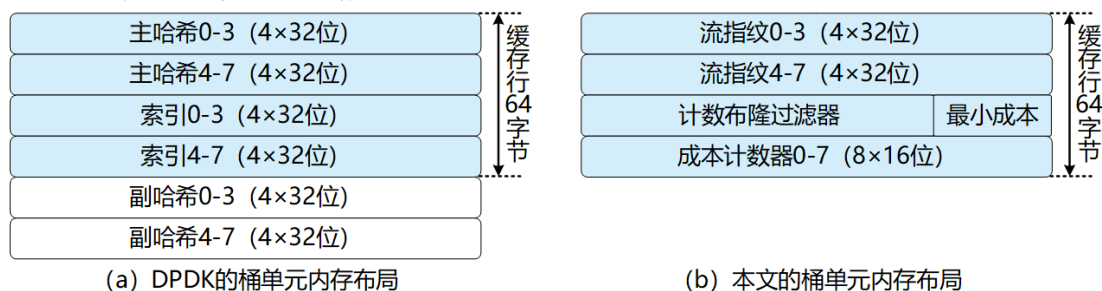


图 4-4 DPDK Cuckoo 与改进的 Cuckoo 的内存布局图

4.3.2 基于混合哈希编码的流指纹生成方法

在桶式 Cuckoo 哈希中,为了实现高效的流识别并有效降低哈希碰撞的概率,本章提出了一种基于混合哈希编码的流指纹生成方法。考虑到在桶式 Cuckoo 哈希中,基于 h_{tp} 生成的两个 t 位哈希值 h_1 、 h_2 已经用于定位流在桶式 Cuckoo 哈希表中的位置,因此 h_{tp} 中已经有 t 位信

息被隐式地包含在流当前的位置信息中。为了进一步减少哈希碰撞的概率，本章将 h_{tp} 中的其余 $(32-t)$ 位与流的四元组信息进行组合，形成一个混合式流指纹，存储于桶式 Cuckoo 哈希表中。具体来说，对于任一流 x ，本章设计了流指纹 f_1 和 f_2 ，分别存储于主桶和副桶，指纹的计算方法如公式 4-4 所示：

$$\begin{cases} f_1 = ((IP_{src} \oplus IP_{dst}) \ll (32 - t)) | (h_{tp} \gg t) \\ f_2 = (h_{tp} \ll t) | ((IP_{src} \oplus IP_{dst}) \gg (32 - t)) \end{cases} \quad (4-7)$$

通过将流的源 IP、宿 IP 信息与网卡生成的对称 RSS 哈希值 h_{tp} 进行混合编码，生成一个新的 32 位值作为流指纹存储于桶式 Cuckoo 哈希表中，再结合 t 位的位置信息，从而显著增强了流指纹的唯一性，并且有效降低了哈希碰撞的概率，进一步优化桶式 Cuckoo 哈希表的查询效率，确保网络流量测量的准确性和速度。

同时，为了优化内存访问延迟，本章采用首次适应的桶选择方法，保证元素尽可能地存储在 h_1 哈希函数映射的桶中，即主桶中。为了提高桶式 Cuckoo 哈希表的负载率，通常采用负载均衡的桶选择方法，通过为哈希表分配一个计数器，用于实时计算哈希表的负载率，当插入元素时，根据哈希函数 h_1 和 h_2 映射到哈希表的两个候选桶，并选择其中一个负载较低的候选桶，将元素插入到该桶中。然而，这样的代价是增加了元素插入与查找的复杂度，在这种方法下，每次元素的插入都至少需要访问两个桶。同时，由于元素被平均分配到两个桶中，因此元素的查找也平均需要访问 1.5 个桶。

因此，为了加速元素插入与查找的效率，本章采用首次适应的桶选择方法，具体来说，在添加新元素时，首先使用哈希值 h_1 将该元素映射到主桶中，如果主桶中尚有空闲的槽位，则将该元素直接插入；如果主桶已满，则继续使用哈希值 h_2 将该元素映射到副桶中，如果副桶中有空闲的槽，则直接插入，反之则选择副桶中已有的一个元素进行踢出，并将新元素插入到踢出的空位上，同时对踢出的元素继续执行插入操作。在基于首次适应的桶插入方法中，可以保证大部分的元素保留在主桶中，实验结果表明，即使在 Cuckoo 哈希表负载率达到了 80% 的情况下，副桶中的元素数量占比也不会超过 20%，因此在元素正向查找的过程中，访问桶的次数将更加接近于一次，平均而言，本方法可以显著减少内存访问次数，在大多数情况下，单次桶的访问便足以完成元素的插入与查询工作，从而优化了内存使用效率与查询性能。

4.3.3 基于计数布隆过滤器的查找优化方法

在传统的桶式 Cuckoo 哈希中，正向查找的过程（如图 4-5(a)所示）首先需要对键进行哈希运算并访问主桶，如果在桶中找到匹配的键，则进一步访问键值对哈希表，因此只需两次内存访问即可响应大多数查找。如果在主桶中并没有查找到匹配的键，则需要继续访问副桶，并继续对键值对哈希表进行访问，因此，在这个过程中需要进行三次内存访问才可完成查找。不过值得注意的是，在基于首次适应的桶选择方法中，若桶式 Cuckoo 哈希表的负载不高，则副桶中元素的数量占比将会非常小，因此在大多数场景下，只需要通过两次内存访问便可完成正向查找。

而在反向查找中，对于标准的桶式 Cuckoo 哈希（如图 4-5(b)所示），当在主桶中找不到匹配的键时，需要检查副桶，然后才能确定搜索的键是否存储在哈希表中。因此，对于负向查找，则必然需要进行 3 次内存访问。

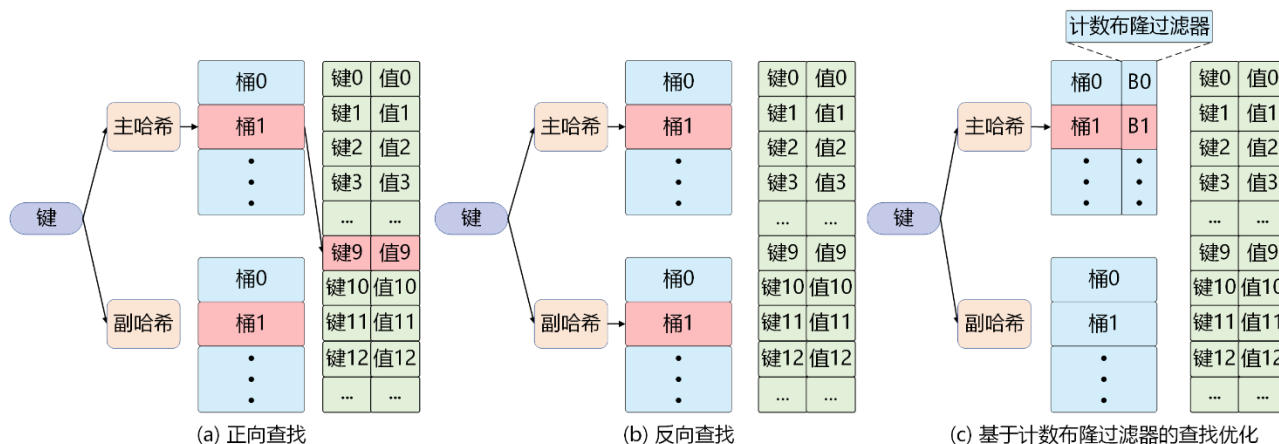


图 4-5 基于计数布隆过滤器的查找优化

为了提高反向查找效率，降低对副桶访问的频率，本章在 Cuckoo 哈希的基础上引入了一个计数布隆过滤器，如图 4-5(c)所示。这个计数布隆过滤器维护那些未能存放在主桶而被迁移到副桶中元素的记录，从而能够快速判断一个元素是否存在于副桶内。此外，相比于 Cuckoo++^[55]，本章使用的计数布隆过滤器可以支持更精确地跟踪元素的插入次数。

在查询过程中，如果主桶中没有匹配的键，系统将先检查计数布隆过滤器。如果计数布隆过滤器显示该元素对应的计数位存在 0，则可以断定副桶中也必然不包含所要查询的元素，因此可以直接返回查找失败，避免了不必要的副桶访问；如果计数布隆过滤器中相应位的计数器都不为 0，则需要继续访问副桶，进一步判断所查询的元素是否真实存在，如图 4-5(c)所示。考虑到在首次适应的插入方法下，大多数元素倾向于存储在主桶中，因此，在大多数情况下可以有效减少对副桶的访问需求，从而确保可以在一次内存访问中完成大多数元素的反向查找。

在元素的插入过程中，如果主桶中尚有空槽位，则插入过程与标准桶式 Cuckoo 哈希表相同；如果主桶中没有空闲的槽，则需要将该元素存储在副桶中，此时，除了标准桶式 Cuckoo 哈希表的插入流程之外，还需要更新相应主桶的计数布隆过滤器，将计数布隆过滤器中新元素映射到的位加一。

在元素的删除过程中，如果元素存储在主桶中，则删除过程与标准桶式 Cuckoo 哈希表的删除操作相同；如果元素存储在副桶中，则在每次删除时，除了执行标准删除流程外，还需要对计数布隆过滤器相应位置的计数器递减。

同时，为了加快元素查找操作，本章使用了 AVX2 指令集来加速查找桶中的元素，通过使用 AVX2 的加载指令将桶中的 8 个 32 位流指纹加载到 YMM 寄存器中（第 2-3 行），然后使用比较指令比较两个 YMM 寄存器中的数据，得到一个布尔掩码（第 4-5 行），如果掩码不等于零，则表示元素存在，查找成功，掩码中第一个为 1 的位就是对应的数组下标（第 6-9 行），反之则表示元素不存在，查找失败（第 10 行），整体流程如算法 4-3 所示。通过 AVX2

指令，将 8 个 32 位整数进行并行处理，大大降低了元素查找的时间复杂度，对于 8 个 32 位整数输入，本算法的时间复杂度仅为 $O(1)$ ，同时空间复杂度为 $O(1)$ 。

算法 4-3 基于 AVX2 指令集的元素查找

输入：桶 `bucket[i][8]`，键 `key`

输出：键 `key` 在桶中的下标 `index`

```

1:  function avx2_find_key_in_bucket(bucket, key)
2:      bucket_data  $\leftarrow$  _mm256_loadu_si256((_m256i*)&bucket[i][0])
3:      key_vector  $\leftarrow$  _mm256_set1_epi32(key)
4:      compare_result  $\leftarrow$  _mm256_cmpeq_epi32(bucket_data, key_vector)
5:      mask  $\leftarrow$  _mm256_movemask_epi8(compare_result)
6:      if mask  $\neq$  0 then
7:          index  $\leftarrow$  _builtin_ctz(mask)
8:          return index / 4
9:      end if
10:     return -1
11: end function
  
```

4.3.4 基于惰性时间衰减成本的流逐出方法

哈希冲突对哈希表性能的影响不容忽视，常见的处理哈希冲突的方法有开放定址法与链地址法等，与传统哈希表不同，Cuckoo 哈希的核心是用踢出并重新插入的概念来解决哈希冲突。Cuckoo 哈希通过使用 d ($d \geq 2$) 个哈希函数，为每个元素提供 d 个备选存储桶，当所有备选位置均被占用时，系统会选择一个位置，将其现有元素踢出，并把新元素插入，被踢出的元素则继续执行递归插入操作，如果递归插入的次数超过阈值，则需要对哈希表进行扩容与重哈希。

由于哈希函数的固有特性，Cuckoo 哈希同样无法彻底避免哈希冲突。在 Cuckoo 哈希中，当发生哈希冲突时，需要执行踢出操作，将所选哈希桶中的元素踢出到其他备选位置，并对被踢出的元素继续执行插入操作，直到所有元素都能被存储至 Cuckoo 哈希表中，而频繁的踢出操作会导致哈希表的多个哈希桶之间产生密集的数据迁移。

此外，在 Cuckoo 哈希插入的过程中，循环冲突也是一个常见问题，当一个元素插入 Cuckoo 哈希表时与某个已存在元素冲突，并将其踢出，而被踢出的元素在寻找备选位置时又遇到冲突，导致连锁反应不断发生，并最终回到最初的位置，形成闭环。如图 4-6 所示。

Cuckoo 哈希虽然易于实现，且保证了最差情况下的查找和更新时间，但它无法预测新元素插入时的迭代路径长度。传统处理哈希冲突的方法主要是随机漫游 (Random Walk, RW) 方法^[56]与广度优先搜索 (Breadth First Search, BFS)。当所有备选位置都已满时，随机漫游方法会从中随机选择一个元素执行踢出操作，并通过设置踢出阈值或超时状态来判定插入操作失败，但这消耗了大量的系统资源，同时随机决策会在一定程度上误导踢出决策。广度优先搜索 (Breadth First Search, BFS) 通过并行检查所有备选位置来搜索最优路径，在一定程度上避免了循环。然而，随着 Cuckoo 哈希表负载率的增加，搜索空间呈指数增长，严重消耗临

时缓存,此外广度优先搜索算法实现的复杂度也太高。Sun 等提出 MinCounter(MC)方法^[58],采用负载均衡的思想,在选择被踢元素时使用踢计数器,并选择踢次数较少的冷桶,这可以减少落入循环的概率,并减少平均插入时间。然而,这种方法只关注冲突踢出的次数,而忽略了网络流本身的状态。此外,这种方法的单元是桶,而不是元素,这导致决策信息的丢失。

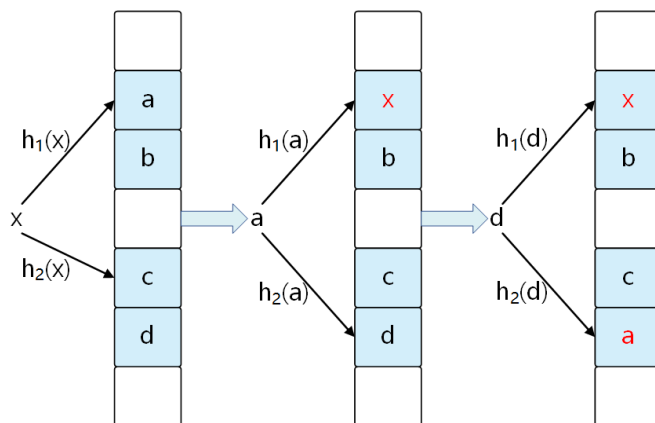


图 4-6 Cuckoo 哈希的循环冲突

为了解决上述问题,本章提出一种基于惰性时间衰减成本的流逐出方法,使其具有自老化能力,当新的流到达时,它会踢出具有最小时间衰减持久性的旧流。由于网络流呈现长尾分布的特征,极少数的大流占据了大多数的数据包,在桶式 Cuckoo 哈希发生冲突时,踢出大流的成本显然是要远大于小流的,因此,本章为桶式 Cuckoo 哈希表中的每个元素分配了一个计数器,来记录将其踢出所需要的成本,当发生哈希冲突时,选择成本最低的元素执行踢出,同时增加被踢出元素的成本,避免陷入循环踢出的局面。

具体来说,对于网络流 f ,则该流的成本计算如公式 4-8 所示:

$$NewCost = Cost \times e^{-\mu(t_i - t_{i-1})} + Bytes_i \quad (4-8)$$

其中, $NewCost$ 表示流 f 经过时间衰减更新后的成本; $Cost$ 表示流 f 在接收新数据包之前的成本值,即计数器记录的成本; μ 表示衰减率,决定了随时间衰减的速度; t_i 表示第 i 个数据包到达的时间; t_{i-1} 表示第 $i-1$ 个数据包到达的时间; $Bytes_i$ 表示第 i 个数据包的字节数。

如果两个数据包之间的时间间隔较大(即 $t_i - t_{i-1}$ 较大),则原有的成本会通过衰减因子 $e^{-\mu(t_i - t_{i-1})}$ 显著减少,表明流在这段时间内的活跃性较低,新到达的数据包的字节数 $Bytes_i$ 则会加到衰减后的成本上,更新流的活动情况,并将这个新活动的成本纳入总成本中。

因此,当桶式 Cuckoo 哈希表发生冲突时,其成本计数器将直接确定其被踢出的优先级,成本较低的流将被优先选择踢出。同时,为了防止某条流被反复踢出,本章将被踢出流的成本增加一个基础成本 $Cost_0$,通过这种方法可以有效避免陷入循环踢出的困局。此外,如果没有成本更低的流可供踢出,即为插入失败,若出现插入失败的情形,本文会将无法插入的元素临时存放在一个固定大小的缓冲区中,而不是立刻对整个哈希表进行扩容重构,可以有效降低由哈希表重构带来的昂贵开销。

基于惰性时间衰减成本的流逐出方法结合了时间间隔和新到达数据包的大小两个因素来动态更新每一条流的成本，时间间隔长的流的成本会逐渐减少，而新到达的数据包则会更新流的成本，本方法将流的历史活动模式和最新的数据活动相结合，用于在桶式 Cuckoo 哈希发生冲突时决定哪些流应该被踢出。

此外，为了快速判断桶中是否还有空位，本章使用了 AVX2 指令集来加速桶中的空位查找，通过将算法 4-3 元素查找的元素改为 0 即可实现空位查找。同时，为了快速定位最小成本的流，本章使用了 SSE 指令集来加速桶中 8 个 16 位成本计数器的最小成本查找，如算法 4-4 所示。

算法 4-4 基于 SSE 指令集的最小成本查找

输入：桶 *bucket[i][8]*

输出：桶中空位的下标 *index*

```

1: function sse_find_empty_in_bucket(bucket)
2:   bucket_data  $\leftarrow$  _mm_loadu_si128((__m128i*)&bucket[i][0])
3:   min_pos  $\leftarrow$  _mm_minpos_epu16(bucket);
4:   index  $\leftarrow$  _mm_extract_epi16(min_pos, 0);
5:   if index  $\geq$  0 then
6:     return index
7:   return -1
8: end function
  
```

在基于 SSE 的最小成本查找算法中，首先将桶数据加载到 SSE 寄存器中（第 2 行），随后使用 _mm_minpos_epu16 指令查找最小值的位置 *index*（第 3-4 行），最后判断 *index* 是否大于等于零，如果是则返回最小值的位置，否则返回 -1（第 5-7 行）。对于 8 个 16 位整数，本算法的时间复杂度为 $O(1)$ ，空间复杂度为 $O(1)$ 。

4.4 基于改进桶式 Cuckoo 哈希的高速网络测量方法操作

本节将详细介绍在基于桶式 Cuckoo 哈希的网络测量方法中，网络流的插入、查询更新与删除操作。

在基于 DPDK 的通用计算平台上进行网络测量时，DPDK 提供了两种的编程模型，即 Run-To-Completion 和 Pipeline。在基于 DPDK 的 Run-To-Completion 模型中，CPU 线程负责处理每个数据包的完整流程，包括从关联的 RX 接收队列持续性地轮询数据包，随后将它们批量地提取到用户空间，并对数据包进行逐一解析，最后针对网络流量进行细粒度的测量与分析，如图 4-7(a)所示。相比之下 Pipeline 架构采取了一种模块化方法，将数据包的接收和相关的处理流程解耦开来，进而分配到不同的 CPU 线程中去执行。在 Pipeline 架构下，一系列专门的线程按顺序协作，相继完成数据包的接收、预处理、测量分析等步骤，从而实现了任务的并行处理和资源优化，线程之间通过环形无锁队列进行通信，详细的处理流程如图 4-7(b)所示，这种分阶段的处理架构能够提高系统的扩展性和灵活性，允许动态地调整资源以适应网络流量的变化，同时确保了更高效率的负载平衡和性能增强。

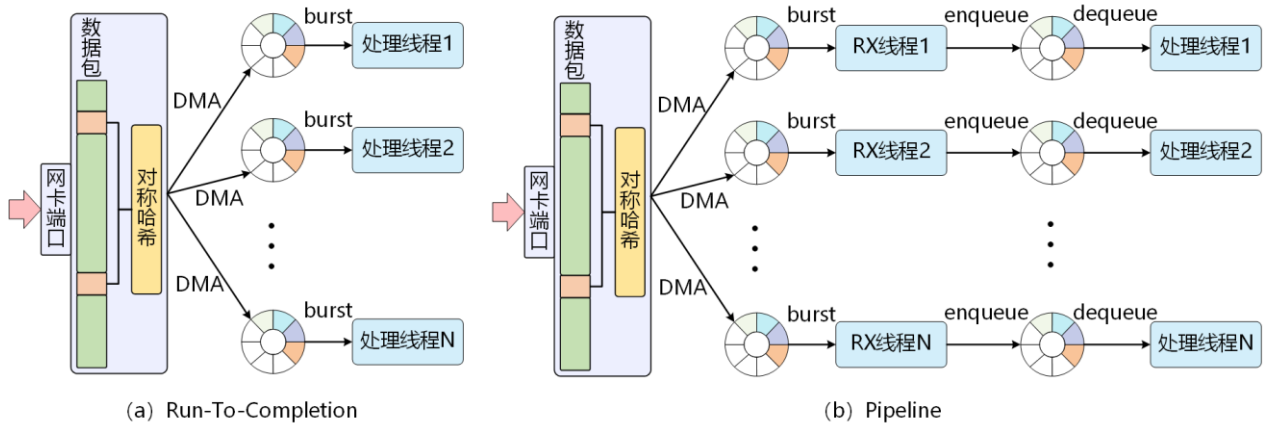


图 4-7 Run-To-Completion 与 Pipeline 模型架构图

本章将分别采用 Run-To-Completion 与 Pipeline 两种架构实现测量算法，并比较分析二者的性能。对于解析出的网络流，本章将对流的包数、字节数、持续时间等特征进行测量记录，作为哈希表的值，如表 4-2 所示：

表 4-2 网络测量的流特征

特征名	特征大小	说明
报文数	32 字节	流的报文数，包括双向
字节数	32 字节	流的字节数，包括双向
流开始时间	64 字节	流的第一个报文的时间戳
流结束时间	64 字节	流的最后一个报文的时间戳
最大包间隔	64 字节	两个数据包之间的最大时间间隔
最小包间隔	64 字节	两个数据包之间的最小时间间隔

4.4.1 插入操作

对于收到的数据包 P ，获取网卡生成的对称 RSS 哈希值 h_{tp} ，生成另外两个哈希值 h_1 、 h_2 ，分别用于定位流在桶式 Cuckoo 哈希表中主桶与副桶的位置，并生成流指纹 f_1 、 f_2 （第 2 行）。接着尝试将流插入，如果主桶中 h_1 行还有空闲的槽，则直接插入，并更新流的成本（第 3-9 行）；如果主桶中没有空闲的槽，则先预修改主桶中的计数布隆过滤器，将相应的位数加一，随后尝试将元素插入副桶，如果副桶中 h_2 行还有空闲的槽，则直接插入，并更新流的成本，反之则执行踢出操作（第 10-17 行）。在踢出操作中，将副桶中 h_2 行所有槽中成本最小的流踢出，然后将元素插入到踢出的空位中，最后增加踢出元素的成本，并对其继续执行插入操作（第 18-26 行）。如果达到了插入操作的阈值 MAXLOOP，则插入失败，将最后被踢出的元素放入一个额外的缓存空间中，并将之前预修改的布隆过滤器还原（第 27-30 行），整体流程如算法 4-5 所示。

在插入的算法过程中，由于算法基于 AVX 与 SSE 指令集实现，对于大多数元素的插入，其时间复杂度为 $O(1)$ ，而在最坏情况下，每次插入都需要达到最大插入阈值 MAXLOOP 才能找到插入位置或者插入失败，则此时的时间复杂度将为 $O(\text{MAXLOOP})$ 。考虑到在实际应用中，在表的负载并没有达到较高的阈值时，元素的递归插入往往并不会达到最大次数，因此本算

法的平均时间复杂度将更接近 $O(1)$ 。在空间复杂度的计算上, 算法主要的额外空间消耗来自递归的调用栈, 因此本算法的空间复杂度为 $O(\text{MAXLOOP})$ 。

算法 4-5 插入

输入: 数据包的 SRSS 哈希值 h_{tp} , 键值对 $(key, value)$

输出: 是否成功插入

```

1:  function Insert( $h_{tp}$ , kick_count, key, value)
2:      initialize  $h_1$ 、 $h_2$ 、 $f_1$ 、 $f_2$ 
3:       $index \leftarrow \text{avx2\_find\_empty\_in\_bucket}(\text{primary\_bucket}[f_1])$  //基于 AVX 的空位查找
4:      if  $index \neq -1$  then
5:           $\text{primary\_bucket}[h_1][index] \leftarrow f_1$ 
6:           $\text{costs}[index] \leftarrow \text{cal\_cost}(key)$ 
7:           $\text{update\_hash\_table}(key, value)$ 
8:          return true
9:      end if
10:      $\text{counting\_bloom\_filter}(h_2)++$  //预修改计数布隆过滤器
11:      $index \leftarrow \text{avx2\_find\_empty\_in\_bucket}(\text{second\_bucket}[f_2])$  //基于 AVX 的空位查找
12:     if  $index \neq -1$  then
13:          $\text{second\_bucket}[h_2][index] \leftarrow f_2$ 
14:          $\text{costs}[index] \leftarrow \text{cal\_cost}(key)$ 
15:          $\text{hash\_table}(key, value)$ 
16:         return true
17:     end if
18:     if  $\text{kick\_count} < \text{MAXLOOP}$  then //执行踢出操作
19:         if  $\text{cost} > \text{costs}[\text{index\_of\_min}]$  then
20:              $h_{tp\_kick}, key\_kick, value\_kick \leftarrow \text{hash\_table}[h_2 \times 8 + \text{index\_of\_min}]$ 
21:              $\text{second\_bucket}[h_2][\text{index\_of\_min}] \leftarrow f_2$ 
22:              $\text{costs}[\text{index\_of\_min}] \leftarrow \text{cost}$ 
23:              $\text{hash\_table}(key, value)$ 
24:              $\text{index\_of\_min} \leftarrow \text{sse\_find\_min\_cost\_in\_bucket}(\text{second\_bucket}[h_2])$ 
25:              $\text{Insert}(h_{tp\_kick}, \text{kick\_count} + 1, key\_kick, value\_kick)$ 
26:         end if
27:     else then
28:          $\text{stash}[] \leftarrow (key, value)$ 
29:          $\text{counting\_bloom\_filter}(h_2)--$  //未插入成功, 还原计数布隆过滤器
30:     end if
31:     return false
32: end function

```

4.4.2 查询更新操作

查询操作需要遍历待查询元素在哈希表中的所有备选哈希桶和额外空间, 桶式 Cuckoo 哈希以简单的方式支持查询请求, 通过首次适应策略以最小化正向查找的存储桶次数, 并基于计数布隆过滤器, 支持对反向查找的快速响应。具体来说, 在查询更新操作中, 首先初始化 h_1 、 h_2 与 f_1 、 f_2 (第 2 行), 紧接着查询主桶中 h_1 行中的所有槽, 如果找到该元素, 则更新其

成本并继续去键值对哈希表中执行查询或更新流的特征信息（第 3-9 行）；如果在主桶中未找到该元素，则先查询主桶中的计数布隆过滤器，如果计数布隆过滤器中相应的位数存在 0，则该元素也肯定不会存在于副桶中，直接响应该元素不存在，查找失败（第 19-20 行）；如果计数布隆过滤器中相应的位数均大于 0，则该元素可能存在于副桶，继续去查询副桶中 h_2 行中的所有槽，如果找到该元素，则更新其成本与键值对哈希表，反之则返回查找失败（第 10-18 行）。查询算法的整体时间复杂度与空间复杂度均为 $O(1)$ ，由于采用了首次适应的桶选择方法，大部分元素被放置在主桶中，因此对于大多数的元素查询更新操作只需要访问一次桶即可。

算法 4-6 查询

输入： 数据包的 SRSS 哈希值 h_{tp} ，键值对 $(key, value)$

输出： 查询结果 result

```

1:  function Query( $h_{tp}$ , key, value)
2:      initialize  $h_1$ 、 $h_2$ 、 $f_1$ 、 $f_2$ 
3:       $index \leftarrow avx2\_find\_key\_in\_bucket(primary\_bucket[h_1], f_1)$  //基于 AVX 的空位查找
4:      if  $index \neq -1$  then
5:           $result \leftarrow primary\_bucket[h_1][index]$ 
6:           $costs[index] \leftarrow cal\_cost(key)$ 
7:           $update\_hash\_table(key, value)$ 
8:          return result
9:      end if
10:     if  $check\_key\_in\_counting\_bloom\_filter(h_2)$  then //检查计数布隆过滤器
11:          $index \leftarrow avx2\_find\_key\_in\_bucket(second\_bucket[h_2], f_2)$ 
12:         if  $index \neq -1$  then
13:              $result \leftarrow primary\_bucket[h_2][index]$ 
14:              $costs[index] \leftarrow cal\_cost(key)$ 
15:              $update\_hash\_table(key, value)$ 
16:             return result
17:         end if
18:     end if
19:      $result \leftarrow null$ 
20:     return result
21: end function

```

4.4.3 删除操作

在删除操作中，首先初始化 h_1 、 h_2 与 f_1 、 f_2 （第 2 行），紧接着定位桶式 Cuckoo 哈希表中要删除的元素，如果在主桶中找到该元素则直接将其删除，将对应的成本也重置为 0，并持久化到本地文件中（第 3-9 行）；如果在副桶中找到该元素则还需要将主桶中的计数布隆过滤器减一（第 10-19 行）；如果无法找到该元素，则表示该元素并不存在于桶式 Cuckoo 哈希表中（第 20 行），整体流程如算法 4-7 所示。在删除操作中，主要耗时在查找键的位置，本算法利用 AVX2 指令的并行性，将算法的时间复杂度降低为 $O(1)$ ，空间复杂度也为 $O(1)$ 。

算法 4-7 删除**输入：**数据包的 SRSS 哈希值 h_{tp} ，键值对($key, value$)**输入：**是否成功删除

```

1:  function Delete( $h_{tp}, key, value$ )
2:      initialize  $h_1, h_2, f_1, f_2$ 
3:       $index \leftarrow avx2\_find\_key\_in\_bucket(primary\_bucket[h_1], f_1)$  //基于 AVX 的空位查找
4:      if  $index \neq -1$  then
5:           $primary\_bucket[h_1][index] \leftarrow 0$ 
6:           $costs[index] \leftarrow 0$ 
7:           $write\_to\_file(key, value)$ 
8:          return true
9:      end if
10:      $counting\_bloom\_filter(h_2)$ -- //预修改计数布隆过滤器
11:     if  $check\_key\_in\_bloom\_filter(h_2)$  then //检查布隆过滤器
12:          $index \leftarrow avx2\_find\_key\_in\_bucket(second\_bucket[h_2], h_{tp})$ 
13:         if  $index \neq -1$  then
14:              $primary\_bucket[h_2][index] \leftarrow 0$ 
15:              $costs[index] \leftarrow 0$ 
16:              $write\_to\_file(key, value)$ 
17:             return true
18:         end if
19:     end if
20:      $counting\_bloom\_filter(h_2)++$  //删除失败，还原 Bloom Filter 计数器
21:     return false
22: end function

```

4.5 实验与分析

4.5.1 实验与环境

本实验在两台通用服务器上分别部署了一套基于 DPDK 的收包、发包系统，两台服务器的硬件配置与软件配置与表 4-3 所示。

表 4-3 基于改进桶式 Cuckoo 哈希的高速网络测量方法的收发包软硬件配置

配置名称	收包机	发包机
CPU	Intel(R) Xeon(R) Silver 4210R @ 2.40GHz 40 核	Intel(R) Xeon(R) Silver 4210R @ 2.40GHz 40 核
NUMA 节点	2	2
网卡	Ethernet 10G 2P X520 Adapter	Ethernet 10G 2P X520 Adapter
内存大小	251GB	251GB
操作系统	Ubuntu 20.04.6 LTS	Ubuntu 20.04.4 LTS
内核版本	5.4.0-169-generic	5.4.0-100-generic
DPDK 版本	23.11	23.11
GCC 版本	9.4.0	11.2.0

在 DPDK 程序中的队列长度、批处理大小等参数设置如表 4-4 所示：

表 4-4 基于改进桶式 Cuckoo 哈希的高速网络测量方法的参数配置

参数名	参数说明	参数值
NB_RSS_QUEUE	RSS 收包队列的数量	2
NB_MEMPOOLS	内存池的数量	2
RX_QUEUE_SIZE	收包队列的长度	4096
NB_MBUF	内存池中的 mbuf 数量	16384
MBUF_CACHE_SIZE	mbuf 缓存大小，用于快速分配	512
BATCH_SIZE	批处理的大小，表示 DPDK 单次最大收包数	32

4.5.2 数据集构建

本章基于 PktGen-DPDK，对 CERNET 与美国 MAWI 流量数据集在不同的发包速率下进行重放。MAWI 数据集是一个基于 WIDE 骨干网 10Gbps 链路收集的公开数据集，本章选取了 MAWI-20200603 流量数据集，同时本章对 CERNET 流量进行了采集，两个流量数据集如表 4-5 所示。

表 4-5 MAWI 与 CERNET 流量数据集

数据集	流的数量
MAWI-20200603	2649345
CERNET	7358682

4.5.3 对比实验设置

首先，本章比较常用的哈希函数在包长 64 字节、发包速率 14.88Mpps 下所消耗的 CPU 周期数，并验证基于对称 RSS 的软硬件哈希方法生成的两个哈希函数的分布均匀性。然后，本章将对线性哈希表、HopScotch、基于 DPDK 的 Cuckoo、Cuckoo++ 与本方法在不同负载比例下的插入与查询的性能比较。最后，本章分别对 CERNET 与 MAWI 流量数据集进行重放，比较了基于 Run-To-Completion 与基于 Pipeline 架构下网络测量的性能差异。

4.5.4 评价指标

为了评估基于对称 RSS 扩展出的两个哈希值的分布均匀性，本章使用卡方检验作为评价指标。卡方检验是一种统计方法，它用于比较观察频数与期望频数之间的差异，从而判断两组数据的分布是否存在显著性差异，通过卡方检验，不仅能够客观地衡量哈希值的统计分布是否符合均匀分布的期待，而且还能为算法的均匀性提供量化的证据。假设在理想均匀分布时，每个哈希值的理论期望出现次数为 m_i ，观测到的每个哈希值出现次数为 x_i ，则卡方统计量 X^2 为：

$$X^2 = \sum_{i=1}^k \frac{(x_i - m_i)^2}{m_i} \quad (4-9)$$

其中， $[1, k]$ 表示哈希值可能的取值范围。如果计算出的卡方统计量小于临界值，则接受“哈希值服从均匀分布”的原假设如果大于，则拒绝原假设，认为分布存在偏差。

随后为了评估本方法在网络测量上的性能，本章以丢包率为核心指标，它直接反映了网络测量的可靠性和效率。同时，我们也衡量了元素查找与插入操作的平均耗时，以此来评价算法处理高速数据流的能力。

4.5.5 实验结果

本章选取了 FNV-1a(Fowler-Noll-Vo)、MurmurHash3、JHash、FastHash、Lookup3、CRC32、xxHash、Toeplitz（软件实现）、XOR32 共九种常见哈希算法与本章提出的方法进行对比。首先，为了评估各个哈希函数在 14.88Mpps 的最高包速下的性能表现，本章分别对每个哈希函数循环运行了 14880000 次，并记录各自所运行的 CPU 周期数与时间，实验结果如图 4-8 所示：

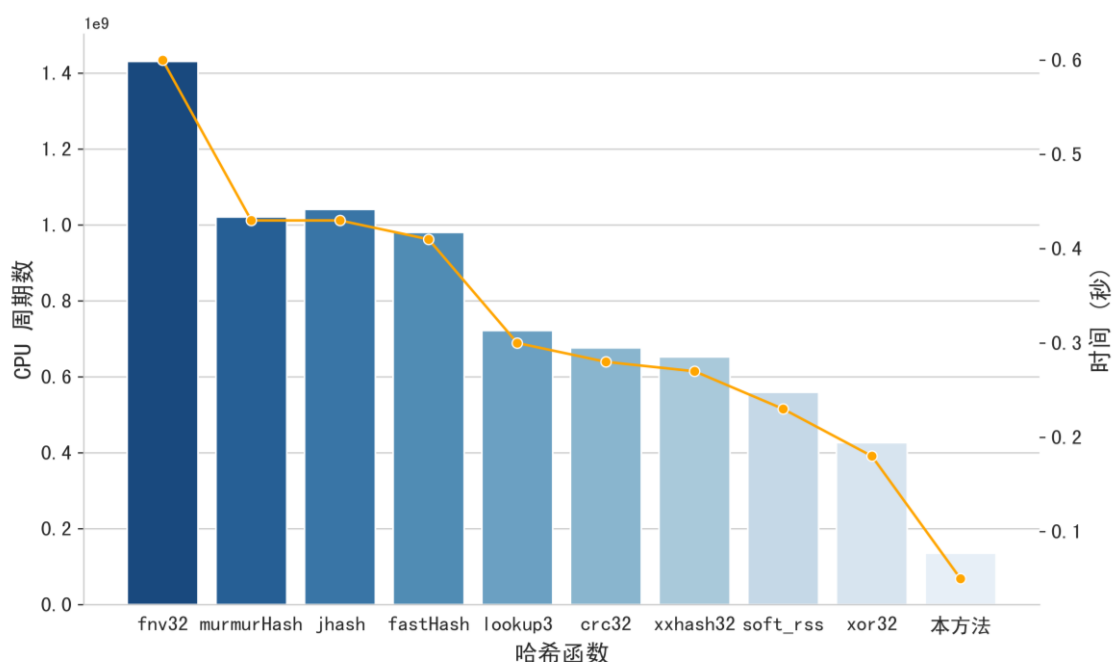


图 4-8 十种哈希函数计算开销对比图

由图 4-8 可见，在 10Gbps 的网络带宽下，若每个数据包的包长只有 64 字节，则理论上每秒能够发送的最大数据包数量高达 14.88M，面对如此高速的网络流量，在使用传统哈希函数进行处理时，则最少都需要消耗 0.2 秒以上的时间，最坏情况下甚至要消耗 0.6 秒以上的时间开销，这无疑对后续的网络数据包处理与测量分析带来了巨大的负担。而本章提出的方法，在网卡硬件对称 RSS 哈希的基础上，只通过简单的移位与异或运算，极大地减缓了对每个数据包再进行哈希运算的开销。

进一步地，为了验证本章提出方法的哈希值分布均匀性，本章仍然对每个哈希函数进行了 14880000 次运算，同时假设哈希表的长度设定为 1048575 (2^{20})，则各个哈希函数的卡方统计量如图 4-9 所示。在显著性水平不超过 0.05 的情况下，卡方分布临界值为 1050959，结合图 4-9 可知，基于本章方法生成的两个哈希值的卡方统计量虽然略高于性能最好的哈希函数，但是也仍然显著低于卡方分布临界值，因此可以认为本方法生成的哈希值服从均匀分布。

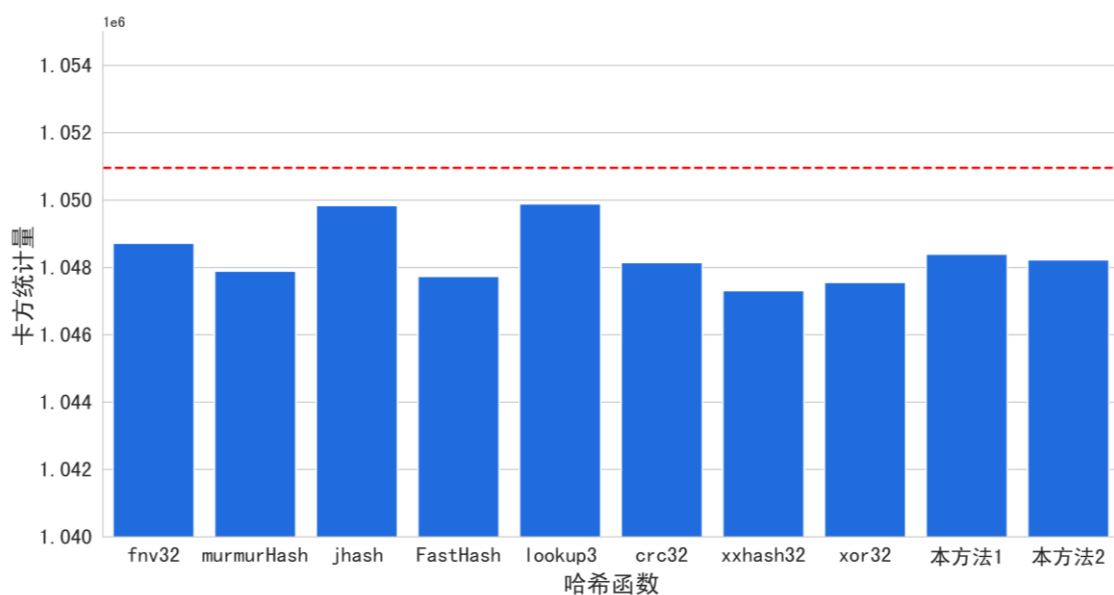


图 4-9 哈希函数卡方检验图

更进一步地, 为了验证本章方法生成的两个哈希函数的独立性, 本章对 1500 对 (h_1, h_2) 绘制了散点图, 如图 4-10 所示:

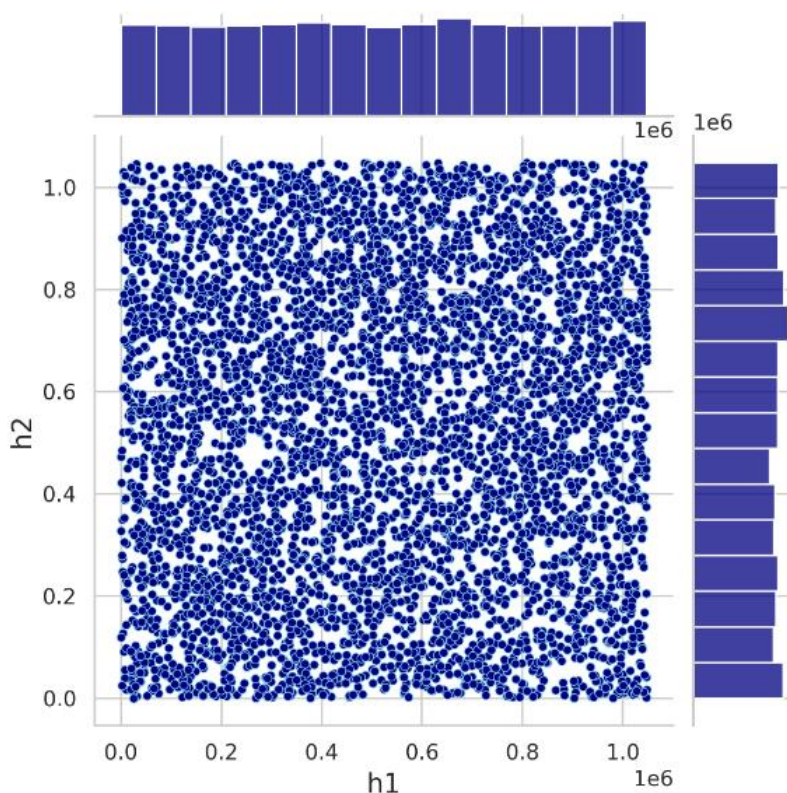


图 4-10 基于对称 RSS 的软硬协同哈希方法的分布散点图

从图 4-10 可以看出, 基于本章方法生成的哈希值 h_1 , h_2 在分布均匀的同时, 也具备良好的独立性。

同时,为了进一步说明本章基于 SSE 的最小成本查找与基于 AVX 的元素查找的高效性,本章与基于循环的元素查找、最小查找进行了对比,搜索次数从 1.48Mpps 到 14.88Mpps,实验结果分别如图 4-11、4-12 所示:

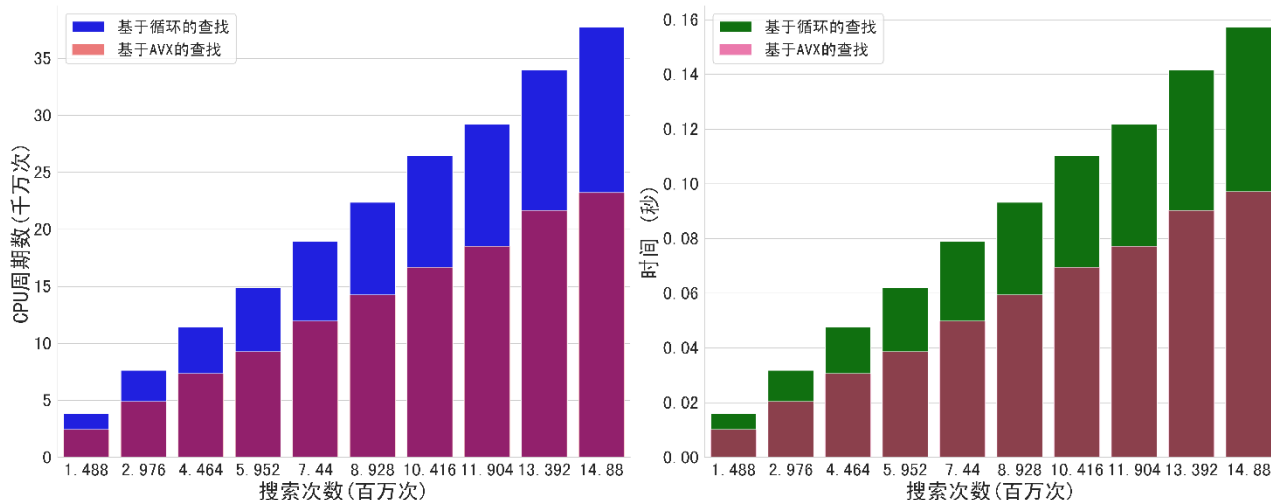


图 4-11 基于循环与基于 AVX 的元素查找对比图

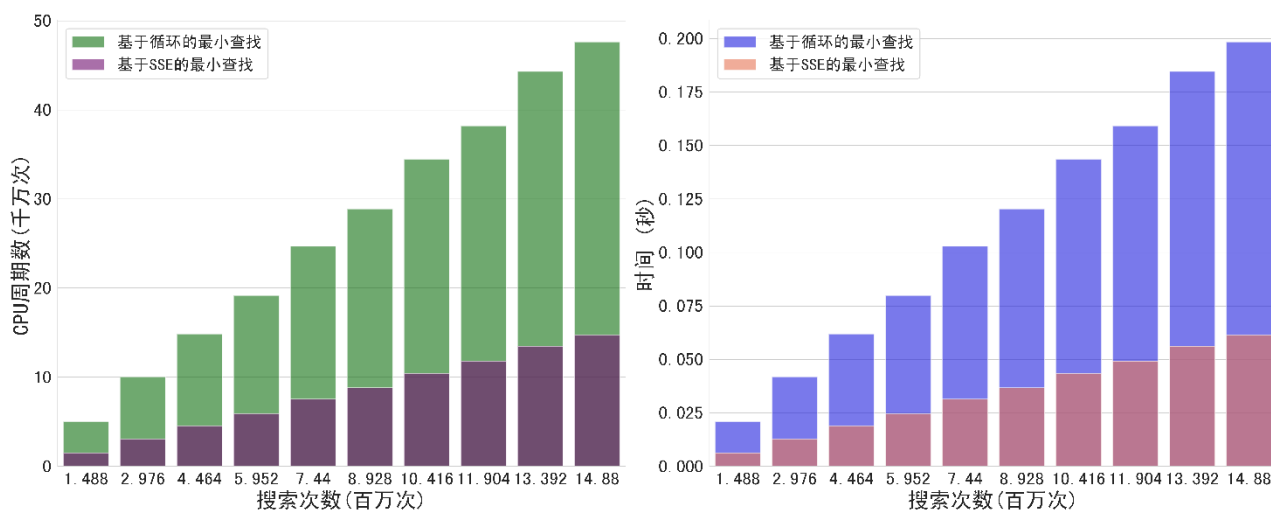


图 4-12 基于循环与基于 SSE 的最小查找对比图

从图 4-11 与图 4-12 中可以看出,在对 8 个 32 位整数进行查找时,相较于基于循环的查找,基于 AVX 指令所消耗的 CPU 周期数与时间都不足前者的三分之二;而在查找 8 个 16 位整数的最小值时,基于 SSE 指令所消耗的 CPU 周期数与时间甚至不足基于循环的一半。由此可得,在桶式 Cuckoo 哈希表中,AVX 与 SSE 指令可以显著提高程序的性能。

本章进一步测量了在不同哈希表负载情况下,基于 DPDK 实现的 Cuckoo、Cuckoo++、HopScotch、基于链地址法的哈希表与本章方法的平均插入和查询所消耗的 CPU 周期数,实验结果如图 4-13 所示。在图 4-13 中,在数据插入的过程中,随着哈希表负载的提升,插入所消耗的 CPU 周期也逐步增加,尤其当负载达到 90%以上时,插入所消耗的 CPU 周期数将大幅增加。相较于 Cuckoo 与 Cuckoo++,本章方法由于采用了基于惰性时间衰减成本的流逐出方法,在踢出流时会选择相对成本较低的元素,从而减少插入操作中的踢出次数,因此在较高负载时,本章方法在数据插入时所耗的 CPU 周期数的增长会相对较为平缓。在查询性能上,

尽管所有方法在哈希表接近满载时的查找所消耗的 CPU 周期数都缓慢增长, 但 HopScotch 在负载增加时查找所需的 CPU 周期数激增, 而 Cuckoo、Cuckoo++及本方法的查找时间则几乎保持不变。

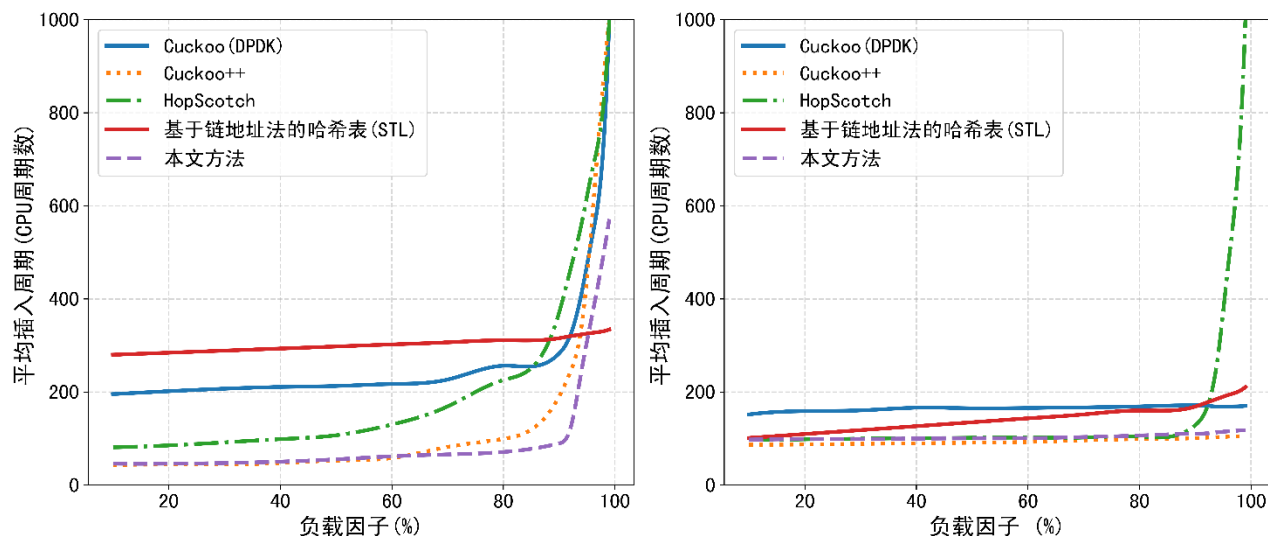


图 4-13 不同负载因子下的插入与查询周期对比图

同时, 本文比较了基于 Run-To-Completion (RTC) 与基于 Pipeline (PL) 架构, 不同 CPU 核心数目下, 进行网络测量时的丢包率, 实验结果如图 4-14 所示:

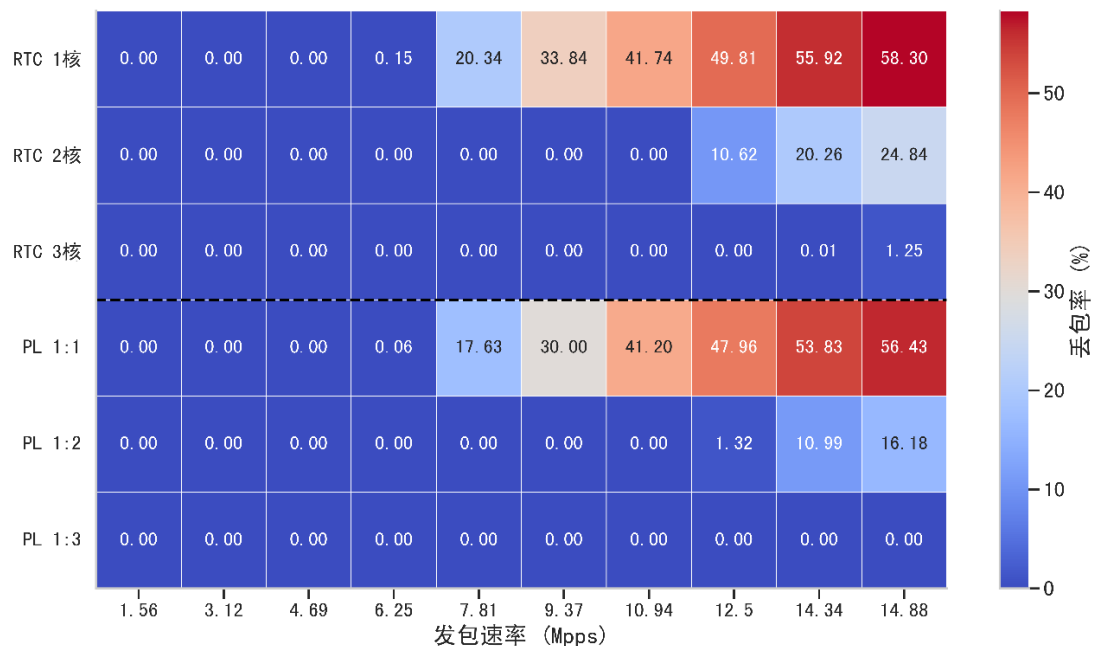


图 4-14 基于 Run-To-Completion 与 Pipeline 的丢包率图

从图 4-14 中的数据对比可以看到, 在拥有相同数量的测量核心的情况下, 采用 Pipeline 架构相比于 Run-To-Completion 架构, 其数据包的丢包率有着略微的降低, 这一差异凸显了 Pipeline 架构在处理高速网络流量时的优势。特别地, 当采用 Pipeline 架构时, 利用单个 CPU 核心对 RX 接收队列进行轮询, 并将数据包转发给另外三个核心进行测量分析时, 可以在高达 14.88Mbps 的发包速率下实现零丢包率的网络测量。

最后,为了评估本文提出的网络测量方法在实际应用中的可行性与效果,本章基于 Pktgen 分别对 CERNET 与美国 MAWI 的流量数据集进行重放实验,两个流量数据集的流分布如图 4-15 所示:

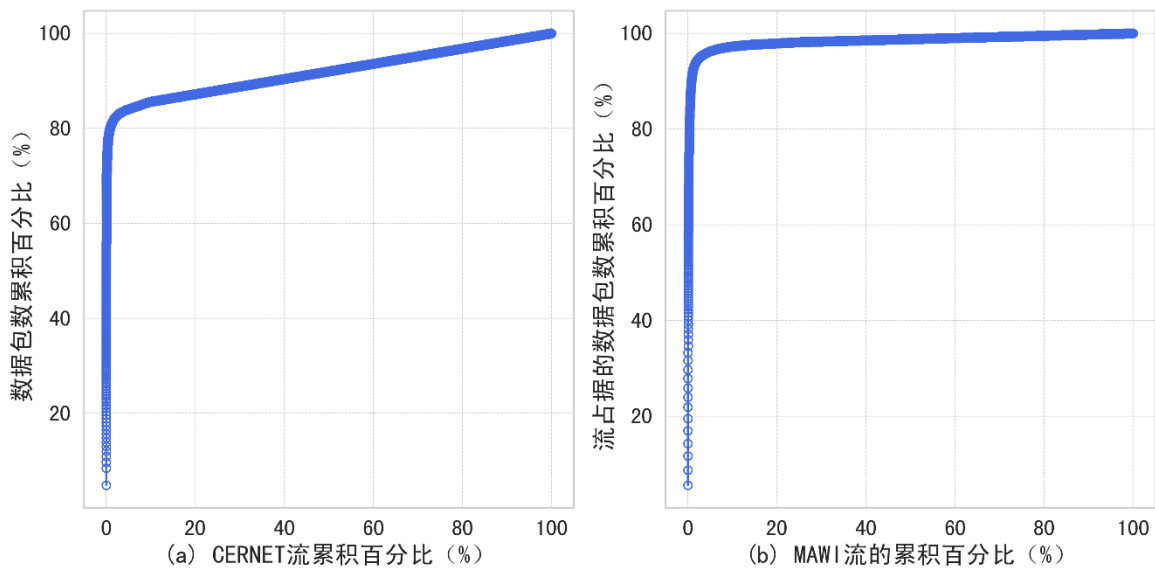


图 4-15 CERNET 与 MAWI 流累计百分比图

分别对两个流量数据集进行重放,测量丢包率,实验结果如图 4-16 所示:

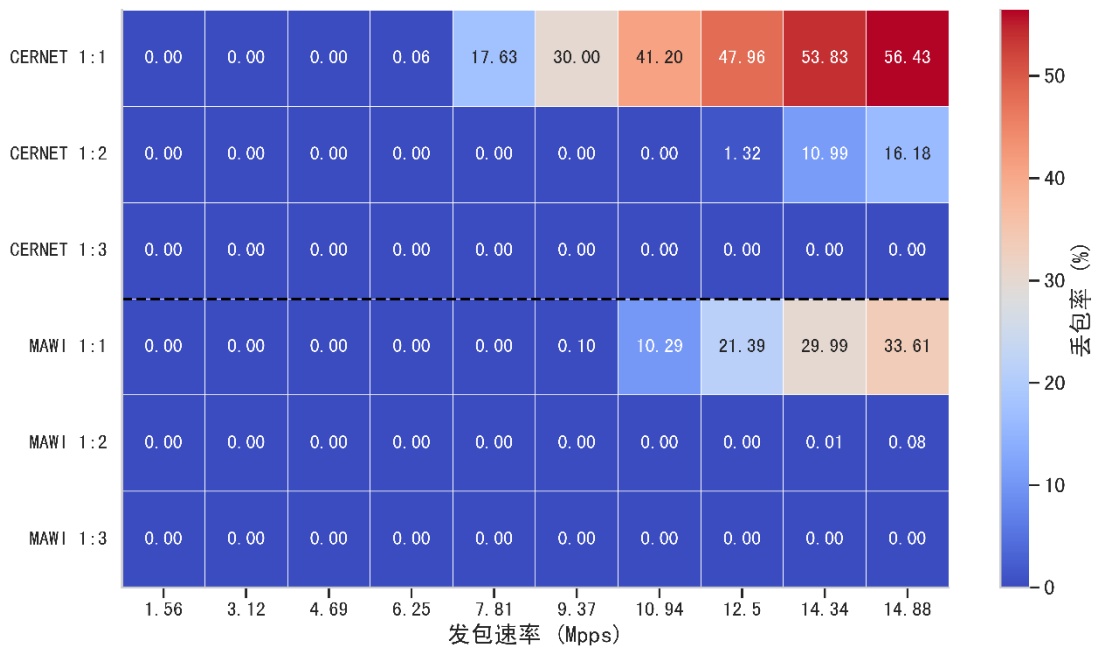


图 4-16 CERNET 与 MAWI 流量数据集重放的丢包率图

由于 CERNET 流量数据集中小流的占比较大,导致流数远多于 MAWI 的流数,因此对系统的压力更大,导致了单核 56%与双核 16%的丢包率;而在 MAWI 流量数据集中,数据包更集中地分布在几条大流上,流数较少,因此系统双核便可达到 0.08%的丢包率。

综上实验结果分析,相比于现有的 Cuckoo 与 Cuckoo++等方案,本章提出的方法更加适用于通用计算平台上的高速大规模网络测量场景,通过快速哈希与计数布隆过滤器,可以有

效提升查询与插入的速度，同时基于惰性时间衰减成本的流逐出方案避免了将活跃流踢出的局面，有效保证了在较高表负载率下的插入效率。

4.6 本章小结

针对通用计算平台上传统 Cuckoo 哈希中面临的多重哈希计算开销、多个备选桶查询开销大以及高负载条件下可能出现的循环冲突的问题，本章提出了一种基于改进桶式 Cuckoo 哈希的高速网络测量方法。本方法基于网卡的对称 RSS 迅速生成数据包的两个哈希值，并结合计数布隆过滤器降低副桶的查询频率，进而提升查询效率。此外，引入基于惰性时间衰减成本的流逐出机制，避免活跃流被踢出。本方法优化了现有算法对 CPU 缓存行的利用效率，在低负载时尽可能地保证在查询一次桶来响应操作，在高负载时尽可能避免活跃流被踢出。同时，本方法基于 AVX 与 SSE 指令集对计算进行加速，进一步提升了性能。最后，本章对 Cuckoo 与 Cuckoo++ 等哈希表在不同负载下查询与插入速度进行比较，并在 CERNET 与 MAWI 流量数据集上进行了丢包率测试，验证了本方法相比现有方法的优势。

第五章 系统设计与实现

本章基于第三章和第四章提出的方法,设计并实现了基于 DPDK 的高速自适应网络测量原型系统。本章首先分析了该原型系统的需求,接着介绍了系统整体架构和系统搭建原理,进而对各个模块进行详细介绍。该系统包括了资源分配模块、收包转发模块、网络测量模块与流持久化模块。最后对四个模块进行了具体实现与实际验证。实际验证结果表明,该系统可以有效实现 10Gbps 下网络流量的测量追踪,具有一定的实用意义。

5.1 系统需求与分析

本系统的核心需求是在高速网络流量中进行网络测量,提取流的包数、字节数等特征信息,涵盖了本文第三章与第四章的研究成果,从系统实现的角度分析需要包括软硬件资源分配、数据包接收解析转发、网络测量与流持久化等需求模块。在这些需求中,需要在 10Gbps 网络流量规模下进行网络测量,在包长 64 字节的情况下,数据包速度高达 14.88Mpps,系统需要快速完成检测,以缩短网络测量的时间,因此系统整体基于 DPDK 开发,并基于 Pipeline 多核架构,可以有效提升网络数据包处理效率。

5.2 系统整体架构设计

基于上述需求,本系统主要实现四个功能模块:资源分配模块、收包转发模块、网络测量模块与流持久化模块。资源分配模块需要指定监听的网卡端口号、分配合适数量的 CPU、配置网卡队列等;收包转发模块需要从网卡硬件队列中快速收包,并对数据包进行解析,提取流的四元组信息与特征信息,最后再通过无锁队列根据制定的转发规则发送到网络测量模块中的 CPU 中;随后在网络测量模块中,基于改进桶式布谷鸟哈希表进行网络测量,以流为单位更新网络流量特征,同时将老化的流通过无锁队列及时导出到流持久化模块;最后,在流持久化模块中,将流的四元组与特征信息写入文件中。

为了进一步实现上述功能,本文设计并实现了一个基于 DPDK 的高速自适应网络测量原型系统,其系统整体设计架构如图 5-1 所示。

由图 5-1 可知该系统主要分为资源分配模块、收包转发模块、网络测量模块与流持久化模块,四个模块的具体介绍如下:

(1) 资源配置模块

本模块负责综合管理和优化网络资源,其核心功能包括接收并解析输入的参数,根据参数初始化 EAL 环境,创建内存池,指定待监听的网卡端口号,并初始化网卡端口,为后续模块分配适当数量的 CPU 等资源,并根据实际需求配置网卡队列。这些功能的有效整合可以确保网络运行环境的最优化,从而提升整体的数据处理能力和效率。

(2) 收包转发模块

该模块从设备的硬件队列中迅速、高效地收集数据包,进而对这些数据包进行深入分析以提取流的四元组信息和关键特征信息。在该过程中,模块结合无锁队列的机制,根据预设

的转发规则，将数据包高效率地转发至网络测量模块的目标 CPU 处理。模块的灵活性和处理速度是确保数据流畅转发的关键所在。同时该模块还需要对网络流量速度进行预测，并动态调整收包线程的休眠时间，以平衡 DPDK 收包的功耗与性能。

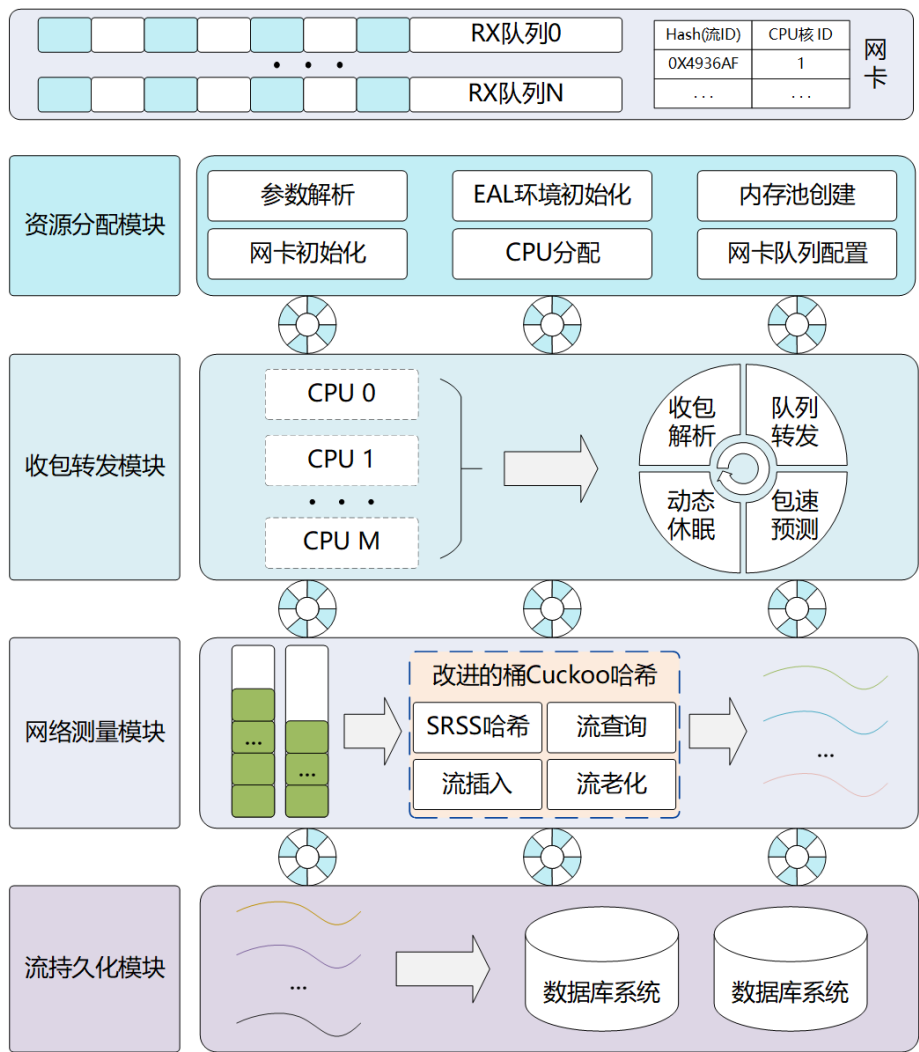


图 5-1 基于 DPDK 的高速自适应网络测量原型系统架构图

(3) 网络测量模块

网络测量模块是系统中对网络流量进行监测和分析的关键部分。它利用改进桶布谷鸟哈希算法来进行数据的存储和查询，能够有效地以流为单位跟踪和更新网络流量。同时，该模块包括一个直接导出老化数据流的机制，通过无锁队列技术确保了数据的及时导出与高效处理。

(4) 流持久化模块

最后，流持久化模块承担着将数据流的关键信息记录和存储的责任。它保存每个数据流的四元组和关联的特征信息，通过将这些数据写入持久存储文件中，为未来的数据分析和挖掘提供了可靠的基础。模块的设计旨在保障数据的完整性和可用性，支持更深入的网络分析和优化决策。

5.3 系统模块设计

基于上述的系统整体架构设计，本节将介绍各个模块的具体设计情况，详细说明模块设计的细节。

5.3.1 资源配置模块

本模块负责系统所需资源的初始化与分配，输入形式为用户指定的参数，对于本系统而言有两类参数，一类是 DPDK 环境初始化参数，包括分配 CPU 序列、启用大页内存、指定日志级别等，另一类是用户自定义参数，包括设置监听的网卡端口号、网卡队列数量、程序运行时间等。当该模块的流程开始时，接受一道参数，模块先解析 DPDK 环境初始化参数，来对 EAL 环境进行初始化，然后将 DPDK 环境初始化参数剥离，进一步解析用户自定义参数。随后，模块根据指定的参数创建内存池。紧接着，模块判断用户指定的网卡端口号是否有效，配置网卡的接收队列与发送队列，开启混杂模式，并尝试启动网卡端口。然后，模块利用 CPU 亲和性将收包解析线程、网络测量线程、流持久化线程与相应的 CPU 绑定，并让 CPU 开始运行相应的线程。完成上述配置后，资源分配模块陷入等待状态，最后在其他模块运行完成后，模块还需要负责回收并释放之前分配的资源。该模块的整体设计流程如图 5-2 所示。

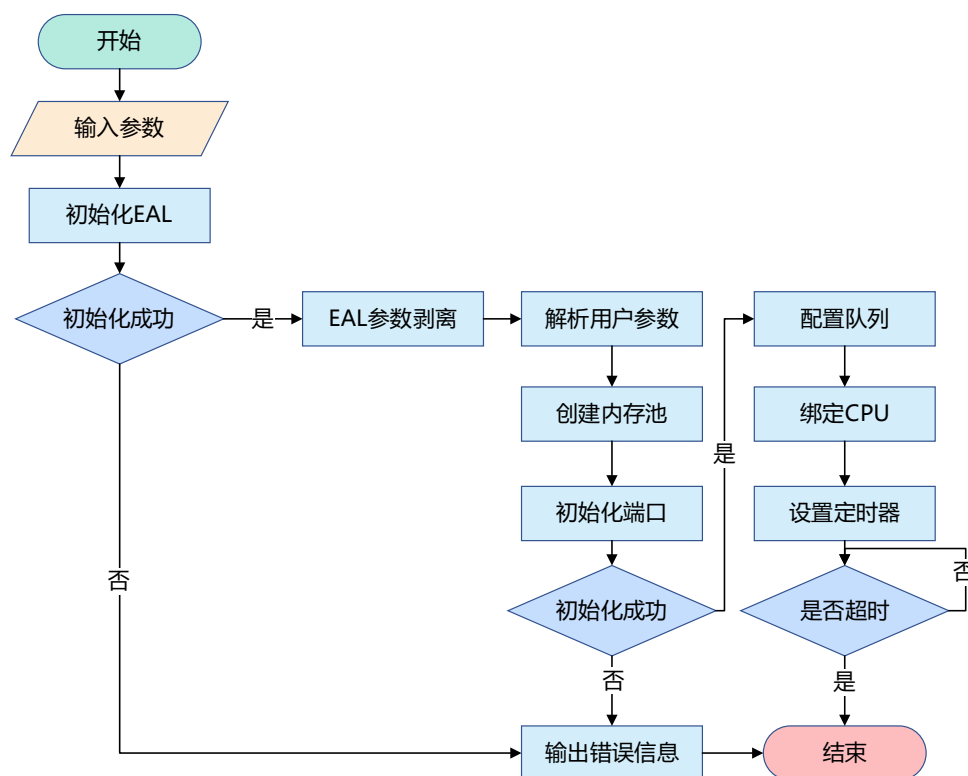


图 5-2 资源配置模块流程图

5.3.2 收包转发模块

本模块负责从网卡队列中轮询接收数据包，并对数据包进行解析，最后将流的四元组与流特征信息以键值对的形式通过环形无锁队列发送到网络测量模块，该模块的设计流程如图 5-3 所示。

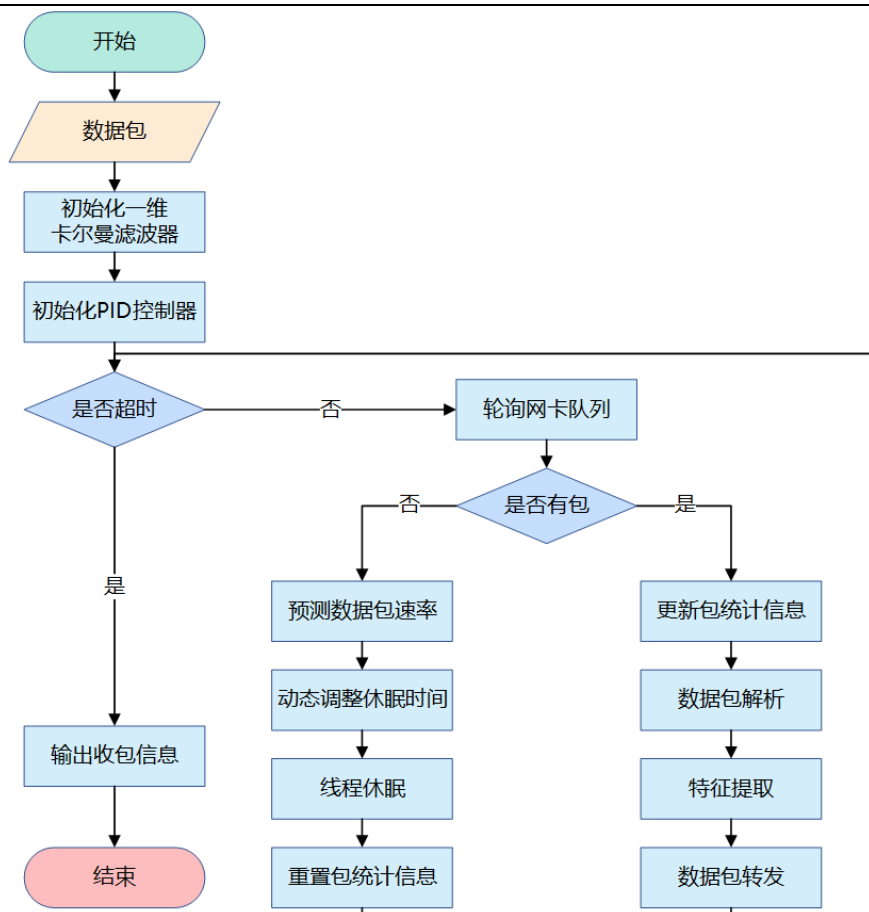


图 5-3 收包转发模块流程图

由于 DPDK 采用轮询收包机制，即使在流量规模较小的时候，CPU 也仍然保持在满负载状态，因此为了降低 CPU 功耗的损失，本模块还在每次空轮询之后增加了一段休眠，这也是本文第三章提出的方法。该模块首先基于一维卡尔曼滤波算法对数据包速率进行预测，随后根据预测的包速，基于 PID 控制器动态调整每次空轮询之后的休眠时长，以此平衡 CPU 收包的功耗与性能。

5.3.3 网络测量模块

本模块的主要功能是基于改进桶布谷鸟哈希表对网络流特征信息进行插入与更新，在该模块中，首先创建桶布谷鸟哈希表与键值对哈希表，随后轮询环形无锁队列，提取流的对称 RSS 哈希值，并基于次生成另外两个哈希值 h_1 与 h_2 ，并执行查找更新或插入操作，整体流程如图 5-4 所示。

在查找更新操作中，先查询主桶中 h_1 行中的所有槽中是否含有匹配的元素，如果存在则更新这条流的成本，并继续访问键值对哈希表，更新流的特征信息。如果不存在，则先查询布隆过滤器，如果布隆过滤器显示存在（即相应的位都为 1）则继续查询副桶 h_2 中是否确实存在该流，如果副桶中存在，则更新其成本与流特征信息，反之则返回查找更新失败，执行插入操作。如果布隆过滤器显示不存在（即相应的位存在 0）则立即返回查找更新失败，避免再去查询副桶，提高查询的效率。查找更新的流程如图 5-5 所示。

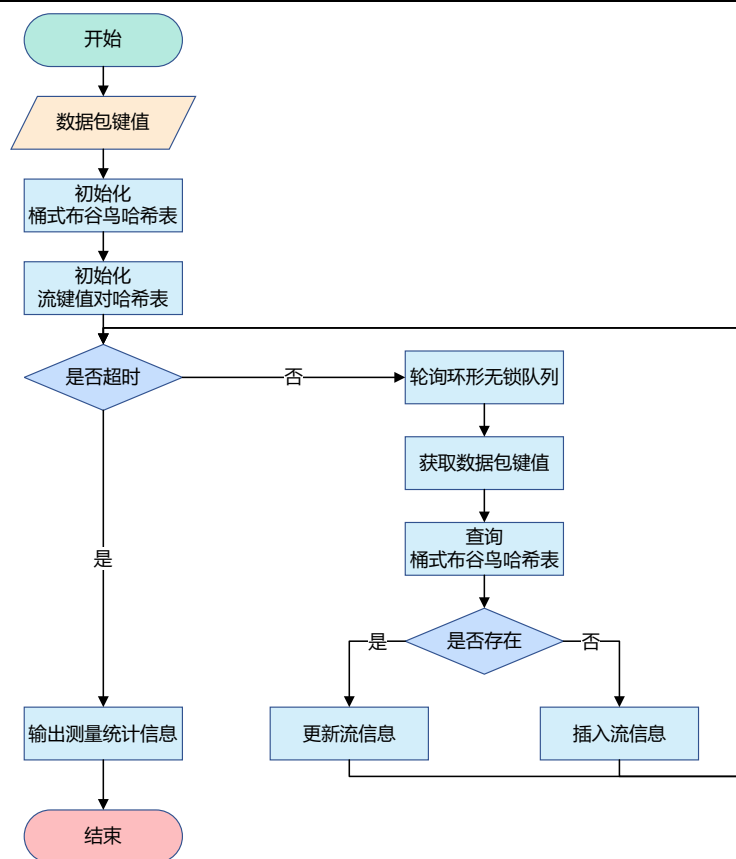


图 5-4 网络测量模块流程图

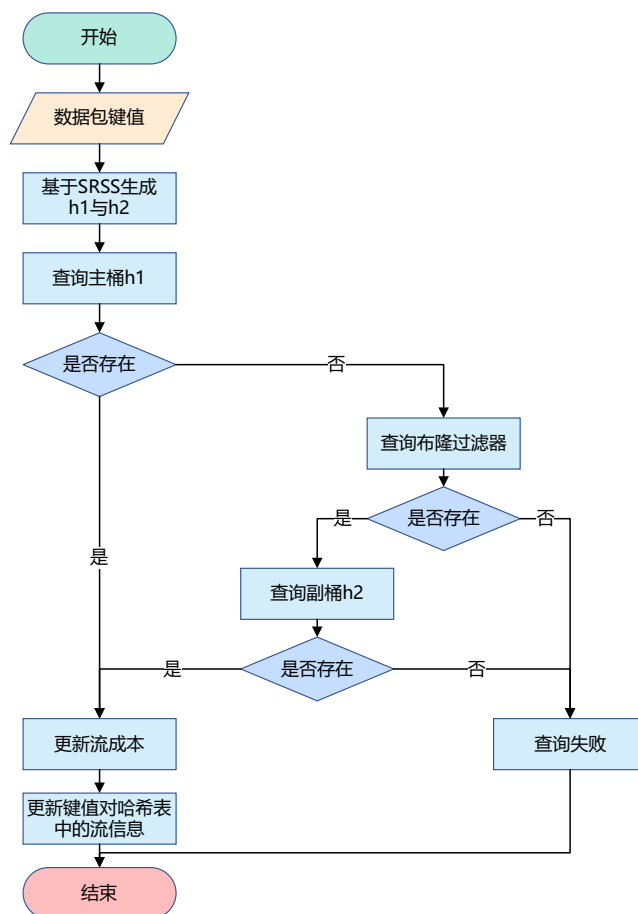


图 5-5 查找更新流程图

在插入操作中, 先查询主桶中 h_1 行中的所有槽中是否还有空位, 如果尚存在空位则直接插入桶布谷鸟哈希表中, 初始化成本, 并继续访问键值对哈希表, 插入该流的特征信息。如果不存在空位, 则访问副桶, 查询副桶中 h_2 行中的槽中是否还有空位。如果副桶中存在空位, 则将该流插入空位中, 初始化成本, 并在键值对哈希表中插入该流的特征信息, 同时还需要更新主桶中的布隆过滤器, 将相应位更新为 1, 并将布隆过滤器计数器加一。反之, 如果副桶中也不存在空位, 则选择副桶中 h_2 行槽保存的所有流中成本最小的流进行踢出, 然后插入到空位中, 并增加被踢出的元素成本, 对其继续执行递归插入操作。如果某个流递归插入的次数超过了指定的上限, 则返回插入失败。插入的流程如图 5-6 所示。

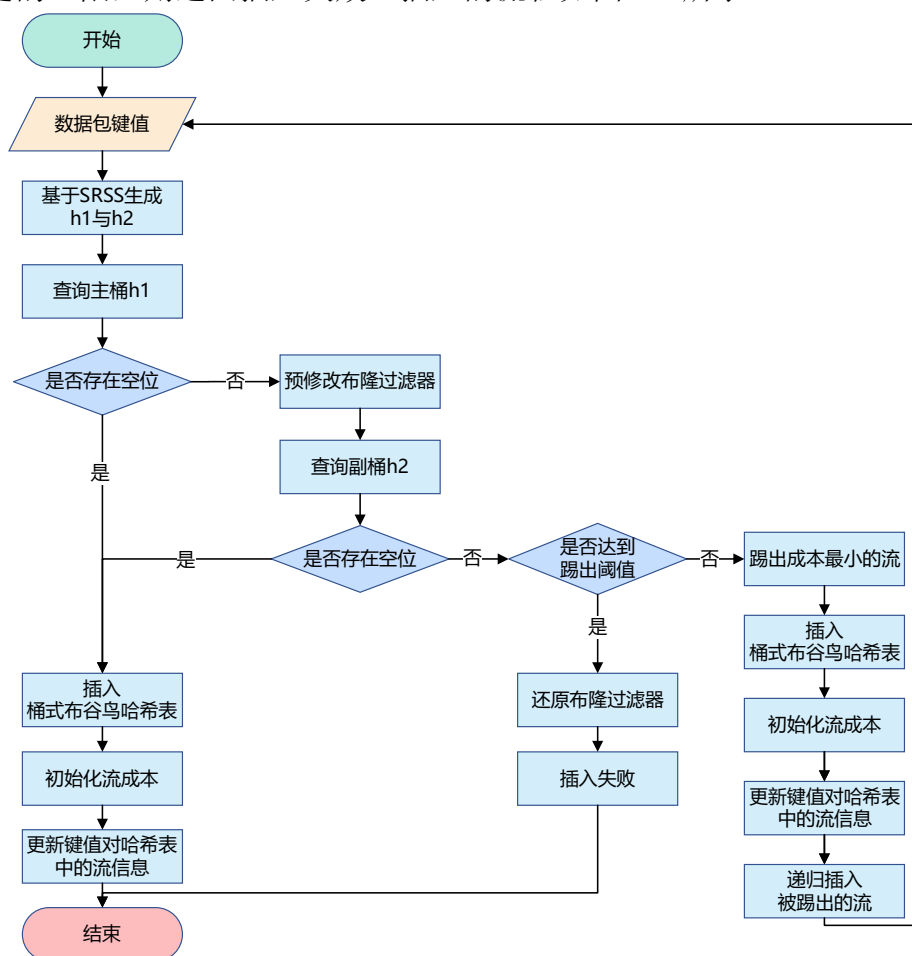


图 5-6 插入流程图

最后, 对于已经结束的流或插入失败的流, 则将其通过环形无锁队列发送至流持久化模块。同时, 为了提高操作中查询匹配元素、查询空位、查找最小成本的效率, 本模块使用了 AVX 与 SSE 指令集, 加速整体流程以适应 10Gbps 级的网络流量测量。

5.3.4 流持久化模块

本模块负责对网络流的特征数据进行存储, 为本系统的最后一个模块, 在该模块中, CPU 轮询环形无锁队列, 实时接收来自网络测量模块的老化流数据及其关键特征信息, 并将其写入到数据库文件中, 实现信息的持续性管理与保障。此外, 该模块还为用户提供了—个简便的接口, 用户可以通过这一接口轻松访问数据库, 以查询和分析被监测的流信息。

5.4 系统运行结果

本系统部署在中国教育和科研计算机网华东地区网络中心(CERNET)的网络环境下,通过交换机端口镜像,向收包服务器发送流量以进行测量。

现将本系统的各个运行结果展示如下:

(1) 系统主界面

本系统的主界面如图 5-7 所示,系统主界面的侧边栏为导航菜单,它作为用户在各个功能模块间切换的主要通道,涵盖了系统运行概览(即主界面)、网络测量以及测量结果等三个主要页面。此外,为了提供更多帮助与支持信息,导航栏还包含了直达 DPDK 官方网站和常见问题解答的链接。在主界面最上方是一系列的指标数据展示,它们包括当前系统测量的总流量、今日流量、本周流量以及相比上周流量增长率等指标,中间是系统每天测量到的流数,在下方是关于每次测量的流数统计与流类型的占比。在右侧是时间与 CPU、内存等系统资源的占用情况。

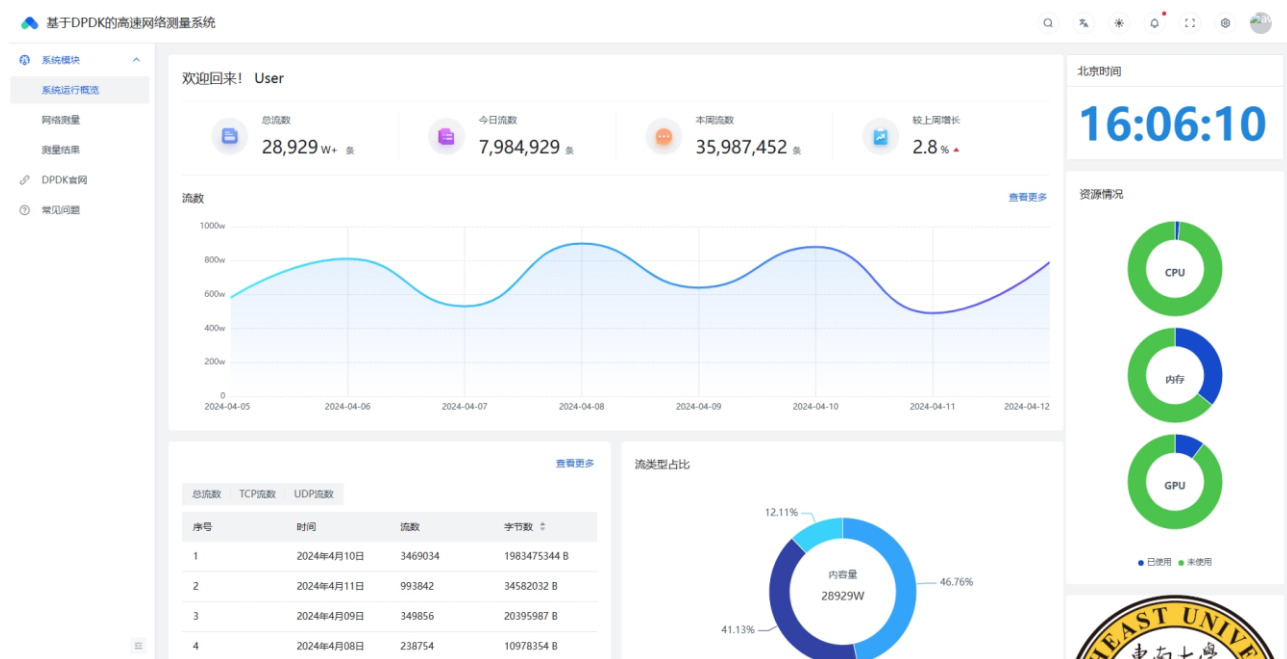


图 5-7 基于 DPDK 的高速自适应网络测量原型系统主界面图

(2) 网络测量界面

网络测量是系统的核心界面,包含了资源配置模块、收包转发模块与网络测量模块,如图 5-8 所示。在页面上方是用户输入的参数,用以配置系统运行环境,包括网卡端口号、CPU 核心数、网卡队列数、测量时间等参数,如果需要更详细的参数配置可以点击右上角的 More 选项。在输入参数后,点击开始测量按钮系统将开始收包测量。左下角是实时显示的每秒捕获到的报文数,右边是丢包率,右下角是其余指标,包括 CPU 利用率、轮询非空率、字节数、报文数、流数等。

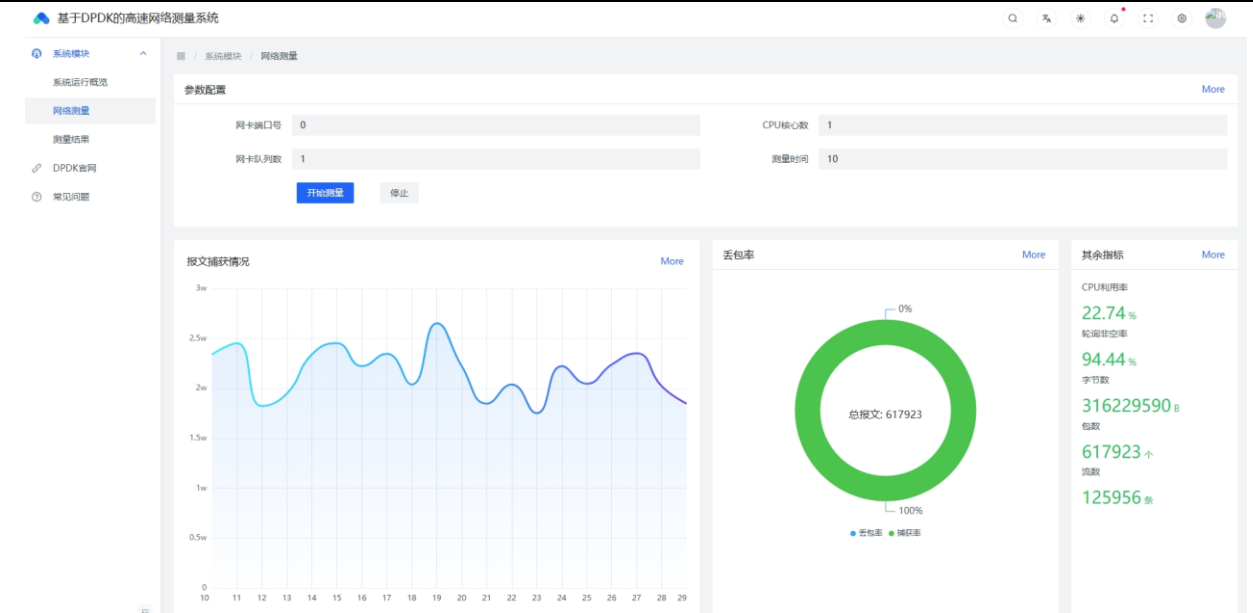


图 5-8 网络测量界面图

(3) 测量结果界面

在测量结果界面，系统将展示流的特征信息，如图 5-9 所示。在界面上方，提供了筛选流的选项，包括源 IP、宿 IP、源端口、宿端口、流开始时间等，用于自定义地选择查看流信息。下方则是具体显示了流的特征信息，除了流的四元组外，还包括报文数、字节数、流开始时间与持续时间等。

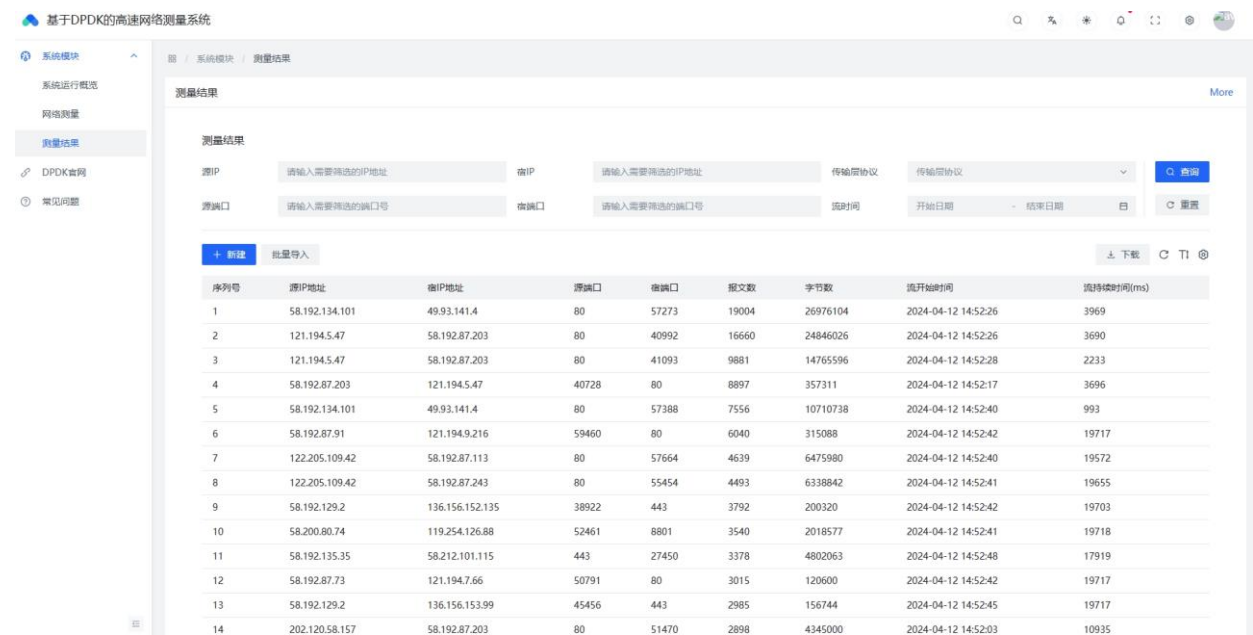


图 5-9 测量结果展示图

5.5 系统性能测试

为了进一步测试本系统的极限性能，本节使用 PktGen 对该系统以 14.88Mpps 的速率发送 64B 的小包以进行测试，考虑到尽量减少对系统性能的影响，本节测试将不使用网页结果展示。Pktgen 的发包界面如图 5-10 所示：

```

\ Ports 0-1 of 2 <Main Page> Copyright(c) <2010-2023>, Intel Corporation
Port:Flags : 1:P----- PCAP
Link State : <UP-10000-FD>
Pkts/s Rx : 00
Tx : 14,915,687
MBits/s Rx/Tx : 0/9,546
Pkts/s Rx Max : 00
Tx Max : 14,916,099
Broadcast : 0
Multicast : 0
Sizes 64 : 0
65-127 : 0
128-255 : 0
256-511 : 0
512-1023 : 0
1024-1518 : 0
Runts/Jumbos : 0/0
ARP/ICMP Pkts : 0/0
Errors Rx/Tx : 0/0
Total Rx Pkts : 0
Tx Pkts : 3,963,434,202
Rx/Tx MBs : 0/2,536,597
TCP Flags : .A....
TCP Seq/Ack : 74616/74640
Pattern Type : abcd...
Tx Count/% Rate : Forever /100%
Pkt Size/Rx:Tx Burst: 64 / 64: 64
TTL/Port Src/Dest : 64/ 1234/ 5678
Pkt Type:VLAN ID : IPv4 / TCP:0001.0
802.1p CoS/DSCP/IPP : K 23.11.0 Powered b 0/ 0/ 0-----
VxLAN Flg/Grp/vid : 0000/ 0/ 0
IP Destination : .0, Command Line Inte 192.168.0.1.0
Source : K 23.11.0 Powered b 192.168.1.1/24.0-----
MAC Destination : K 23.11.0 Powered b 00:00:00:00:00:0.0-----
Source : K 23.11.0 Powered b b4:96:91:e5:b1:9c.0-----
NUMA/Vend:ID/PCI : K 23.11.0 Powered b1/8086:154d/0000:87:00.0-----
Pktgen 23.10.2 (DPDK 23.11.0) Powered by DPDK (pid:397121) -----
Pktgen:/>

```

图 5-10 Pktgen 发包界面图

系统后端的测量界面显示如图 5-11 所示：

```

Port 1 ipackets: 4774472, ibytes: 286470240, missed: 0, ierrors: 0
Port 1 ipackets: 19690244, ibytes: 1181416320, missed: 0, ierrors: 0
Port 1 ipackets: 34571716, ibytes: 2074304880, missed: 0, ierrors: 0
Port 1 ipackets: 49487532, ibytes: 2969253600, missed: 0, ierrors: 0
Port 1 ipackets: 64403188, ibytes: 3864192960, missed: 0, ierrors: 0
Port 1 ipackets: 79318808, ibytes: 4759130160, missed: 0, ierrors: 0
Port 1 ipackets: 94234568, ibytes: 5654075760, missed: 0, ierrors: 0
Port 1 ipackets: 109150144, ibytes: 6549010560, missed: 0, ierrors: 0
Write Core 1: write 38878764 packets
Write thread 1: flow_count: 2897584, insert count: 2897584 lookup count: 38744308 insert fail count: 0 avg insert time: 49.008192(0.000000 s) avg lookup time: 73.846405(0.000000 s)
Write Core 2: write 34204491 packets
Write thread 2: flow_count: 1908422, insert count: 1908422 lookup count: 34185516 insert fail count: 0 avg insert time: 48.038892(0.000000 s) avg lookup time: 68.121389(0.000000 s)
Capture thread 0: work time: 0.742511 non empty bursts: 0.065218 capture pkts: 112132320
Write Core 0: write 37454247 packets
Write thread 0: flow_count: 2552676, insert count: 2552676 lookup count: 36632425 insert fail count: 0 avg insert time: 48.819310(0.000000 s) avg lookup time: 72.402190(0.000000 s)

```

图 5-11 基于 DPDK 的高速自适应网络测量原型系统性能测试图

由上图可见，在基于 Pipeline 架构下，使用 1 核来收包转发，另外 3 核用于测量，可以实现 14.88Mpps 速率下 0 丢包率的测量，共收到了 112132320 个数据包，3 个核心分别处理了 2552676、2897584 与 1908422 条流，总计 7358682 条流，证明了系统达到了 10Gbps 级的网络测量能力。

5.6 本章小结

本章基于本文第三章与第四章的研究产出，对基于 DPDK 的高速自适应网络流量测量原型系统进行了介绍与展示，首先进行了系统的需求分析，基于需求分析阐述了系统的整体架构和实现原理。接着详细介绍了系统的四个重要模块，并进行了系统界面展示，将各个模块的结果直观的展示在网页前端，最后对本系统进行了极限性能测试，测试结果表明，该系统完成了预期设计的功能，达到了预期需求的目标。

第六章 总结与展望

6.1 工作总结

随着网络流量规模和传输速率的不断提高，目前在数据中心与主干网中已广泛使用 10Gbps 网络链路来连接交换机和路由器，这给通用计算平台上的网络流量测量带来了巨大的挑战。为了提高报文处理效率，DPDK 采用了轮询模式驱动，然而繁忙的轮询也导致了高功耗问题，即使当前的流量规模很小，CPU 也仍然保持在满负载状态。同时，为了存储数百万条流的多维特征，片上内存极为有限的空间无法支持，因此只能存储在片外内存中，但是对片外内存的访问要远慢于对片上内存的访问，现有研究 Cuckoo 哈希是一种常被用于缓解哈希冲突的数据结构，但是也存在着多次哈希运算开销、多个备选桶查询开销与高负载时可能产生冲突循环而导致较高的插入时延。

针对上述问题，本文依次开展了基于自适应包间休眠的间歇式轮询方法研究与基于改进桶式 Cuckoo 哈希的高速网络测量方法研究，旨在基于通用计算平台，在最高 14.88Mpps 数据包速率下，实现对网络流量多维特征的细粒度测量。具体而言，本文的研究工作总结如下：

（1）基于自适应包间休眠的间歇式轮询方法

本方法首先对 DPDK 轮询收包过程进行建模分析，在允许的 RX 接收队列负载因子下，分析当前数据包速率下休眠时长的期望。此外，本方法进一步采用一维卡尔曼滤波算法对数据包速率进行实时在线预测，并基于 PID 控制器，处理 RX 接收队列的期望负载长度与实际负载长度之间的差距反馈，动态调整每次空轮询后的休眠时长。最后在实验测试中，本方法使用 Pktgen 进行发包重放测试，实验表明，仅考虑收包的情况下，在最高 14.88Mpps 的发包速率时，本方法仍然能保证零数据包丢失，同时将 CPU 使用率降低至 50% 以下，对比其他三种常见方法，本方法可以有效地降低 CPU 使用率与功耗，从而提高 DPDK 轮询收包的能效。

（2）基于改进桶式 Cuckoo 哈希的高速网络测量方法

本方法基于网卡硬件生成的对称 32 位 RSS 哈希值，使用简单的异或与移位运算将其扩展为 2 个低于 32 位的哈希值，以此减少 CPU 哈希运算的开销，同时利用桶式 Cuckoo 哈希作为网络测量键值对哈希表的索引，并基于计数布隆过滤器快速响应针对不存在元素的查询，最后基于惰性时间衰减计算流的成本，以优化 Cuckoo 哈希的踢出路径，避免陷入循环冲突。此外，本方法还基于 AVX 与 SSE 指令集进行加速。在实验测试中，本文分别对 CERNET 与 MAWI 的流量数据集进行重放测试，在使用 1 核收包转发与 3 核测量的 Pipeline 架构下，本方法实现了零丢包的网路测量，与 DPDK 提供的 Cuckoo 哈希等方法相比，本方法在不同的表负载下均有着更好的插入与查询性能。

（3）基于 DPDK 的高速自适应网络测量原型系统设计与实现

本文将上述研究成果进行整合，构建了一个基于 DPDK 的高速自适应网络测量原型系统。为了实现高速自适应的细粒度网络流量多维特征测量，本文首先对系统需求进行了分析，并在此基础上介绍了系统的整体架构，并进一步介绍了各模块的主要功能与流程，包括资源配

置模块、收包转发模块、网络测量模块与流持久化模块。最后本文对系统所有模块进行了展示与验证，并对系统极限性能进行了测试，结果表明该系统可以在通用计算平台上实现对 10Gbps 链路流量进行细粒度的多维特征测量，具有一定的实用意义。

6.2 问题和展望

本文在对基于 DPDK 的高速自适应网络测量方法进行探究和具体实现的过程中，受到一些限制因素仍然存在以下几点不足需要改进，同时也存在更多新的研究方向有待进一步研究：

（1）为了加速软件报文处理，本文提出的测量方法基于 DPDK 实现，因此本文提出自适应休眠方法也是针对于 DPDK 平台，这导致本文的自适应休眠方法在通用性方面仍有不足，在后续的研究中需要进一步研究本方法在 Netmap、PF_RING 与 PF_RING ZC 等高速 I/O 框架上的迁移与扩展。

（2）本文提出的方法均针对通用计算平台上基于软件实现，虽然使用了 DPDK 来加速报文处理，但纯软件的实现仍然存在性能瓶颈，因此本文提出的方法尚难以实现 100Gbps 的线速测量，后续研究中可以将 DPDK 与 FPGA 软硬件结合，将高计算开销的任务卸载到 FPGA 中，进一步加速网络测量。

（3）本文提出的方法采用 RSS 将网卡流量分发到不同的 CPU 核心上来处理，以此实现流量在多核之间的负载均衡，但是由于网络流量呈现长尾分布，少数的大流占据了网络流量的绝大部分，即使分配到各个核心的流在一定程度上是均衡的，但是由于流大小分布的差异，某些核心过载的情况也仍是很有可能发生的，因此后续需要进一步研究流量在多核之间的负载均衡方法，尤其是当流量规模来到 100Gbps 时，多个 CPU 核心之间的负载均衡策略将是影响系统吞吐量的重要因素。

参考文献

- [1] 中国互联网络信息中心. 第 53 次《中国互联网络发展状况统计报告》[EB/OL]. 北京: 中国互联网络信息中心, 2024.
- [2] Rizzo L. netmap: a novel framework for fast packet I/O[C]//21st USENIX Security Symposium (USENIX Security 12). 2012: 101-112.
- [3] Intel. Data Plane Development Kit[OL]. <https://www.dpdk.org/>, 2024.
- [4] Ye J, Li J, Jiang A E P, et al. PF_RING-TA: A traffic-adaptive method of packet capturing by enhancing PF_Ring[C]//2021 IEEE 6th International Conference on Smart Cloud (SmartCloud). IEEE, 2021: 51-56.
- [5] Marathe Y, Gulur N, Ryoo J H, et al. CSALT: Context switch aware large TLB[C]//Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture. 2017: 449-462.
- [6] Shah M, Yunus M, Vachhani P, et al. PowerDPDK: Software-Based Real-Time Power Measurement for DPDK Applications[C]//2020 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN). IEEE, 2020: 13-18.
- [7] Freitas E, de Oliveira Filho A T, do Carmo P R X, et al. A survey on accelerating technologies for fast network packet processing in Linux environments[J]. Computer Communications, 2022, 196: 148-166.
- [8] 赵鹏, 程光, 赵德宇. 基于 FPGA 的高性能可编程数据平面研究综述[J]. 软件学报, 2023, 34(11): 5330-5354.
- [9] Plakalovic M, Kaljic E, Mehic M. High-speed FPGA-based Ethernet traffic generator[C]//2022 XXVIII International Conference on Information, Communication and Automation Technologies (ICAT). IEEE, 2022: 1-6.
- [10] Lin W, Shan Y, Kosta R, et al. SuperNIC: An FPGA-Based, Cloud-Oriented SmartNIC[C]//Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays. 2024: 130-141.
- [11] Emmerich P, Pudelko M, Gallenmüller S, et al. FlowScope: Efficient packet capture and storage in 100 Gbit/s networks[C]//2017 IFIP Networking Conference (IFIP Networking) and Workshops. IEEE, 2017: 1-9.
- [12] 令瑞林, 李峻峰, 李丹. 基于多核平台的高速网络流量实时捕获方法[J]. 计算机研究与发展, 2017, 54(6): 1300-1313.
- [13] Zhang T, Linguaglossa L, Leonardo Linguaglossa, et al. FlowMon-DPDK: Parsimonious Per-Flow Software Monitoring at Line Rate[J]. Traffic Monitoring and Analysis, 2018: 1-8.
- [14] Zhang T, Linguaglossa L, Gallo M, et al. FloWatcher-DPDK: Lightweight Line-Rate Flow-Level Monitoring in Software[J]. IEEE Transactions on Network and Service Management, 2019, 16(3): 1143-1156.
- [15] Zhou Y, Alipourfard O, Yu M, et al. Accelerating network measurement in software[J]. ACM SIGCOMM Computer Communication Review, 2018, 48(3): 2-12.
- [16] Ren Q, Liang Zhou, Liang Zhou, et al. PacketUsher: Exploiting DPDK to accelerate compute-intensive packet processing[J]. Computer Communications, 2020, 161: 324 - 333.
- [17] Bhattacharjee R, Rajesh R, Prasanna Kumar K R, et al. Scalable flow probe architecture for 100 Gbps+ rates on commodity hardware: Design considerations and approach[J]. Journal of Parallel and Distributed Computing, 2021, 155: 87 - 100.
- [18] Ramani V, Chen J, Yates R D. Timely Mobile Routing: An Experimental Study[C]//IEEE INFOCOM 2023-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS). IEEE, 2023: 1-6.
- [19] Han L, Guo Z, Huang X, et al. A Multifunctional Full-Packet Capture and Network Measurement System Supporting Nanosecond Timestamp and Real-Time Analysis[J]. IEEE Transactions on Instrumentation and Measurement, 2021, 70: 1 - 12.
- [20] Huang X. Hardware nanosecond-precision timestamping for line-rate packet capture[J]. IET Networks, 2024: ntw2.12114.

- [21] Zhao Z, Liu Z, Chen H, et al. Effective DDoS Mitigation via ML-Driven In-network Traffic Shaping[J]. IEEE Transactions on Dependable and Secure Computing, 2024.
- [22] Zhang S, Suo L, Li W, et al. Anole: Scheduling Flows for Fast Datacenter Networks With Packet Re-Prioritization[J]. IEEE Transactions on Cloud Computing, 2024.
- [23] Hu S, Bai W, Zeng G, et al. Aeolus: A building block for proactive transport in datacenters[C]//Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication. 2020: 422-434.
- [24] Sun H, Huang Q, Lee P P C, et al. Distributed Network Telemetry With Resource Efficiency and Full Accuracy[J]. IEEE/ACM Transactions on Networking, 2024.
- [25] Zhang J, Wang Y, Zhong X, et al. PACC: A Proactive CNP Generation Scheme for Datacenter Networks[J]. IEEE/ACM Transactions on Networking, 2024.
- [26] Iqbal M F, Holt J, Ryoo J H, et al. Dynamic core allocation and packet scheduling in multicore network processors[J]. IEEE Transactions on Computers, 2016, 65(12): 3646-3660.
- [27] 王煜炜, 刘敏, 马诚, 等. 面向网络功能虚拟化的高性能负载均衡机制[J]. 计算机研究与发展, 2018, 55(4): 689-703.
- [28] Barbette T, Katsikas G P, Maguire Jr G Q, et al. RSS++ load and state-aware receive side scaling[C]//Proceedings of the 15th international conference on emerging networking experiments and technologies. 2019: 318-333.
- [29] 陈中卿, 李丹丹, 闪德胜, 等. 基于流特征的数据中心非对称流负载均衡方法[J]. 软件学报, 2022: 1-14.
- [30] Zhang Q, Liu Y, Liu T. IBalancer: load-aware in-server flow scheduling for sub-millisecond tail latency[J]. IEEE Transactions on Parallel and Distributed Systems, 2021, 33(8): 1761-1774.
- [31] Jeong E Y, Wood S, Jamshed M, et al. mTCP: a Highly Scalable User-level TCP Stack for Multicore Systems[C]//11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14). 2014: 489-502.
- [32] Ousterhout A, Fried J, Behrens J, et al. Shenango: Achieving high CPU efficiency for latency-sensitive datacenter workloads[C]//16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19). 2019: 361-378.
- [33] Belkhiri A, Pepin M, Bly M, et al. Performance analysis of DPDK-based applications through tracing[J]. Journal of Parallel and Distributed Computing, 2023, 173: 1-19.
- [34] Li X, Cheng W, Zhang T, et al. Towards power efficient high performance packet I/O[J]. IEEE Transactions on Parallel and Distributed Systems, 2019, 31(4): 981-996.
- [35] Wu M, Chen Q, Wang J. Toward low CPU usage and efficient DPDK communication in a cluster[J]. The Journal of Supercomputing, 2022, 78(2): 1852-1884.
- [36] Faltelli M, Belocchi G, Quaglia F, et al. Metronome: Adaptive and Precise Intermittent Packet Retrieval in DPDK[J]. IEEE/ACM Transactions on Networking, 2023, 31(03): 979-993.
- [37] Estan C, Varghese G, Fisk M. Bitmap algorithms for counting active flows on high speed links[C]//Proceedings of the 3rd ACM SIGCOMM conference on Internet measurement. 2003: 153-166.
- [38] Geravand S, Ahmadi M. Bloom filter applications in network security: A state-of-the-art survey[J]. Computer Networks, 2013, 57(18): 4047-4064.
- [39] Zeng Z, Cui L, Qian M, et al. A survey on sliding window sketch for network measurement[J]. Computer Networks, 2023, 226: 109696.
- [40] Drmota M, Kutzelnigg R. A precise analysis of cuckoo hashing[J]. ACM Transactions on Algorithms (TALG), 2012, 8(2): 1-36.
- [41] 许可, 李彦彪, 谢高岗, 张大方. 基于混合计数布隆过滤器的高效数据名查找方法[J]. 计算机研究与发展, 2023, 60(5): 1136-1150.
- [42] Li M, Chen D, Dai H, et al. Seesaw Counting Filter: A Dynamic Filtering Framework for Vulnerable Negative Keys[J].

- IEEE Transactions on Knowledge and Data Engineering, 2023.
- [43] Li M, Xie R, Chen D, et al. A Pareto optimal Bloom filter family with hash adaptivity[J]. The VLDB Journal, 2023, 32(3): 525-548.
- [44] Tang L, Huang Q, Lee P P C. A fast and compact invertible sketch for network-wide heavy flow detection[J]. IEEE/ACM Transactions on Networking, 2020, 28(5): 2350-2363.
- [45] Zhang Y, Liu Z, Wang R, et al. CocoSketch: High-performance sketch-based measurement over arbitrary partial key query[C]//Proceedings of the 2021 ACM SIGCOMM 2021 Conference. 2021: 207-222.
- [46] 熊兵, 刘永青, 夏卓群, 等. RobustSketch: 支持网络流量抖动的大流弹性识别方法[J]. 软件学报, 2024: 1-20.
- [47] Huang Q, Jin X, Lee P P C, et al. Sketchvisor: Robust network measurement for software packet processing[C]//Proceedings of the Conference of the ACM Special Interest Group on Data Communication. 2017: 113-126.
- [48] Liu Z, Ben-Basat R, Einziger G, et al. Nitrosketch: Robust and general sketch-based monitoring in software switches[M]//Proceedings of the ACM Special Interest Group on Data Communication. 2019: 334-350.
- [49] Erlingsson U, Manasse M, McSherry F. A cool and practical alternative to traditional hash tables[C]//Proc. 7th Workshop on Distributed Data and Structures (WDAS'06). 2006.
- [50] Kirsch A, Mitzenmacher M, Wieder U. More robust hashing: Cuckoo hashing with a stash[J]. SIAM Journal on Computing, 2010, 39(4): 1543-1561.
- [51] Gou X, Zhao C, Yang T, et al. Single hash: Use one hash function to build faster hash based data structures[C]//2018 IEEE international conference on big data and smart computing (BigComp). IEEE, 2018: 278-285.
- [52] Luo L, Fu P, Li S, et al. Ark filter: A general and space-efficient sketch for network flow analysis[J]. IEEE/ACM Transactions on Networking, 2023.
- [53] Breslow A D, Zhang D P, Greathouse J L, et al. Horton tables: Fast hash tables for In-Memory Data-Intensive computing[C]//2016 USENIX Annual Technical Conference (USENIX ATC 16). 2016: 281-294.
- [54] 蒋捷, 杨全, 张梦瑜, 等. DCuckoo: 基于片内摘要的高性能散列表[J]. 计算机研究与发展, 2017, 54(11): 2508-2515.
- [55] Scouarnec N L. Cuckoo++ hash tables: High-performance hash tables for networking applications[C]//Proceedings of the 2018 Symposium on Architectures for Networking and Communications Systems. 2018: 41-54.
- [56] Frieze A, Melsted P, Mitzenmacher M. An analysis of random-walk cuckoo hashing[J]. SIAM Journal on Computing, 2011, 40(2): 291-308.
- [57] Sun Y, Hua Y, Jiang S, et al. SmartCuckoo: A Fast and Cost-Efficient Hashing Index Scheme for Cloud Storage Systems[C]//2017 USENIX Annual Technical Conference (USENIX ATC 17). 2017: 553-565.
- [58] Sun Y, Hua Y, Feng D, et al. A collision-mitigation cuckoo hashing scheme for large-scale storage systems[J]. IEEE Transactions on Parallel and Distributed Systems, 2016, 28(3): 619-632.
- [59] Wu H, Wang S, Jin Z, et al. CostCounter: A Better Method for Collision Mitigation in Cuckoo Hashing[J]. ACM Transactions on Storage, 2023, 19(3): 1-24.
- [60] Fan B, Andersen D G, Kaminsky M. MemC3: Compact and concurrent {MemCache} with dumber caching and smarter hashing[C]//10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13). 2013: 371-384.
- [61] Sun Y, Hua Y, Chen Z, et al. Mitigating asymmetric read and write costs in cuckoo hashing for storage systems[C]//2019 USENIX Annual Technical Conference (USENIX ATC 19). 2019: 329-344.
- [62] Reviriego P, Pontarelli S, Levy G. CuCoTrack: Cuckoo filter based connection tracking[J]. Information Processing Letters, 2019, 147: 55-60.
- [63] Yan Y, Chen C, Lin H, et al. Priority-aware per-flow measurement using cuckoo sketch[C]//2020 IFIP Networking Conference (Networking). IEEE, 2020: 622-624.

-
- [64] Shi Q, Xu Y, Qi J, et al. Cuckoo counter: Adaptive structure of counters for accurate frequency and top-k estimation[J]. IEEE/ACM Transactions on Networking, 2023.
- [65] Xiao Q, Li Y, Wu Y. Finding recently persistent flows in high-speed packet streams based on cuckoo filter[J]. Computer Networks, 2023, 237: 110097.
- [66] Fan L, Cao P, Almeida J, et al. Summary cache: a scalable wide-area web cache sharing protocol[J]. IEEE/ACM transactions on networking, 2000, 8(3): 281-293.
- [67] Cormode G, Muthukrishnan S. An improved data stream summary: the count-min sketch and its applications[J]. Journal of Algorithms, 2005, 55(1): 58-75.
- [68] Estan C, Varghese G. New directions in traffic measurement and accounting: Focusing on the elephants, ignoring the mice[J]. ACM Transactions on Computer Systems (TOCS), 2003, 21(3): 270-313.
- [69] Han H, Yan Z, Jing X, et al. Applications of sketches in network traffic measurement: A survey[J]. Information Fusion, 2022, 82: 58-85.
- [70] Woo S, Park K S. Scalable TCP session monitoring with symmetric receive-side scaling[J]. KAIST, Daejeon, Korea, Tech. Rep, 2012, 144.

毕业/学位论文答辩委员会名单

毕业/学位论文题目		基于 DPDK 的高速自适应网络测量方法		
作 者		张家康		
专 业		网络空间安全		
研究方向		加密流量识别		
导 师		程光		
答辩委员会组成	姓 名	职 称	学科专业	工作单位
	刘光杰 (主席)	教授	网络空间安全	南京信息工程大学
	季一木	教授	网络空间安全	南京邮电大学
	吴桦	副教授	网络空间安全	东南大学
	张玉健	副教授	网络空间安全	东南大学
	陈子涵 (秘书)		网络空间安全	东南大学

备注：

- 1、本表格适用于所有研究生。
- 2、本表格排版在终版毕业/学位论文中，附在毕业/学位论文的最后。